

基于信息集分层的微网电力调控策略：因果预测、风险定价与滚动优化

摘要

本文以**信息集边界**为主线，把微网电力调控的四个问题统一到同一个母线侧线性规划内核上：母线能量平衡、储能状态转移 $S_t = S_{t-1} + \eta_c C_t - H_t / \eta_d$ 、容量与功率限值、弃光上界等物理约束四问共享，各问之间只改变**可用信息与结算口径**；每期计划都以上一时段实际测得的储电量为初值重新优化，因而各期方案互不相同。

问题一为确定性日前调度。以全天购电费最小为目标、首末储电量均为 6000 kWh 为闭环约束建立线性规划，并用“费用—吞吐量—弃光量”三层词典序目标消除退化最优解。最优全天购电量 59482.70 kWh，购电费 35126.95 元，较无储能基准的 48052.05 元节省 12925.10 元（26.8981%）。

问题二中负载与光伏逐日变化，缺口按电价的 5 倍紧急购电。本文将 5 倍电价严格推导为 4 : 1 的非对称损失，并由报童模型证明最优光伏点预测应取**预测分布的 0.2 分位数**（数值实验验证其最优性），据此构造只使用历史观测的滚动在线选型因果预测器，并设计“先撤充电、再增放电、最后紧急购电”的 10 min 实时储能反馈。2 月至 12 月总费用 14773804.78 元，紧急购电 348036.96 kWh。单因素对照表明：把当天真实负载当作 0:00 已知输入会低估费用 1497816.65 元（10.14%），风险边际与实时反馈则分别节省 1265008.78 元与 586904.81 元。

问题三在 0:00、6:00、12:00、18:00 做 24 h 滚动优化、只执行下一 6 h 块并只继承储电量。为真正回答“是否需要引入其他时刻预报”，以**可用预报节点集合**为唯一消融变量：仅 0:00 时费用 15731164.40 元，依次加入 6:00、12:00、18:00 后降至 14864532.14、14324451.62、13727989.25 元，增量价值 866632.26、540080.52、596462.37 元，累计节省 2003175.15 元，6:00 预报价值最大。两种结算口径**分别完整重新优化**，结论一致。

问题四把固定电价替换为附件 4 的波动电价。为界定电价信息边界，引入因果电价预测，得 Q4-2 为 15523884.83 元（价格完全已知）与 15590096.88 元（因果），Q4-3 为 14389193.11 元与 14524235.75 元；电价信息的理想化价值仅 66212.05 元（0.42%）与 135042.63 元（0.93%），远小于负载信息被完全掌握所造成的 1497816.65 元偏差，说明**信息价值高度不对称**，预测投入应优先投向**负载与光伏**。动态电价下滚动调整仍节省 1065861.14 元，其价值方向不随价格机制改变。

本文建立物理残差、未来信息泄漏 **mutation** 测试、结算口径单元测试与 **legacy** 回归测试四级验证体系，全部通过，最大母线平衡残差低于 10^{-12} kWh。

关键词：微网经济调度；线性规划；滚动时域优化；报童模型；分位数预测；信息集因果性

一、问题重述

1.1 问题背景

微网是为解决光伏发电等分布式能源本地消纳、减少并网问题而快速发展起来的小型发用电系统。出于经济性考虑，非孤岛式微网可以在外网电价较低或分布式能源有剩余电量的时段为储能充电，并在外网电价较高或光伏不足的时段释放电能。由此，微网与外网的调控目标可以概括为：利用小区负载、光伏发电量与储能当前储电量等数据，尽可能精准地给出微网的购电量和储能充放电量，使供电既满足小区负载，又尽可能节省购电费用。

本问题的物理对象由三部分组成：光伏发电单元、储能设备与小区负载，并通过公共连接点与外网相连。附录 1 给定储能参数：最大容量 12000 kWh，最大充放电功率 5000 kW，2025 年 1 月 1 日 0:00 储电量为 6000 kWh，运行期储电量必须保持在 1200 ~ 10800 kWh 之间，充放电效率均为 90%。附件数据的时间分辨率均为 10 min，即一天可分为 144 个调度时段。

问题的核心困难在于**信息的可获得性**：微网必须在决策时刻仅依据当时已知的信息制定策略，而负载、光伏与实际电价在一天内是逐步揭晓的。因此四个问题的差异并不在物理约束，而在于决策时刻能看到什么信息，以及计划与实际不一致时如何结算。

1.2 问题提出

问题 1：每天的电价和小区负载相同，并给出光伏发电功率一天的预测数据。微网提供的电能不可低于小区负载，且储能设备在 0:00 和 24:00 的储电量相同。要求在每天 0:00 制定微网当天的计划购电策略，给出指定时段的购电量、全天购电量与购电费，以及储能的充放电量与首末储电量。

问题 2：每天的电价相同，而小区负载和光伏发电功率随时间变化。微网提供的电能不可低于小区负载；若低于负载，需向外网紧急购电，其电价是交易时刻电价的 5 倍。除紧急购电费用外，其他时间段的购电费用均按计划购电量计算。要求在每天 0:00 制定当天的计划购电策略，给出指定日期（2025.3.20、2025.6.21、2025.9.23、2025.12.21）的购电量、储能充放电量，并给出紧急购电结果。

问题 3：在每天 0:00、6:00、12:00 和 18:00 可获得未来 24 小时整点的光伏发电功率预报。可根据 0:00 的预报制定当天计划购电策略，也可根据其他时刻的预报调整购电策略。计划购电量高于调整购电量的部分，违约电价是交易时刻电价的 50%；调整购电量高于计划购电量的部分，超出部分的电价是交易时刻电价的 1.5 倍。总购电费用包括计划购电费用、紧急购电费用和调整购电量的相关费用。要求建立微网当天购电策略的数学模型，并分析是否需要引入其他时刻的预报制定调整购电策略。

问题 4：外网电价也实时波动。要求针对波动电价建立微网当天购电策略的数学模型，

根据附件 4 的电价数据重新计算问题 2 和问题 3，并给出相应的计算结果。

1.3 研究现状综述

微网经济调度问题在数学上通常被建模为以购电费或综合运行成本最小为目标的线性规划或混合整数规划，外网购电、分布式电源消纳与储能充放电被统一纳入同一组能量平衡约束中 [1, 2]。确定性模型适用于负载、电价与光伏均已知的日前场景，其优点是约束结构清晰、求解稳定可靠。

当光伏出力存在预测误差时，主流处理方式有三类：一是**情景法 / 两阶段随机规划**，用有限个历史或生成情景刻画不确定性，第一阶段确定与情景无关的日前计划，第二阶段按情景确定执行量 [3, 4]；二是**鲁棒优化**，在最坏情景下保证可行性，代价是结果偏保守 [5]；三是**滚动时域优化（模型预测控制）**，在新信息到达后仅重优化剩余时域并只执行近期决策，天然适配“预报分批发布”的决策结构 [6, 7]。含光伏与储能的日前调度研究普遍指出，储能的价值高度依赖预测精度与调度时域长度 [8]，而储能的运行成本又与其循环深度密切相关 [9]。

针对“预测值应偏向哪一侧”的问题，报童模型给出了经典答案：当高估与低估的边际损失不对称时，最优的点预测不是条件均值，而是某个**分位数** [10]。这一结论在电力系统中已被用于发电商的投标策略与偏差电量定价 [11]。在预测方法层面，基于数值天气预报的机器学习模型是目前光伏功率预测的主流 [12]；在市场机制层面，实时电价与需求响应的耦合会显著改变用户的购电行为 [13]。

现有研究多聚焦于算法本身，而对**信息集边界与结算口径**的刻画往往不够严格：不少工作在日前优化中直接使用当天的真实负载或真实电价，使结果成为不可实现的“完美信息下界”；也有工作在比较不同策略时同时改变了预测方法、约束条件与费用口径，导致差异无法归因。本文针对这两点，把信息集边界作为建模的第一原则，并通过单因素消融实验保证每条结论都可归因。此外，与已有工作多采用元启发式求解不同，本文利用问题的线性结构精确求解，避免了启发式算法结果不可复现的问题。

二、问题分析

2.1 总体思路

四个问题共享同一套物理对象与同一组物理约束，差别只在两处：**决策时刻能获得哪些信息**，以及**计划与实际不一致时如何结算**。因此本文采用“一个物理内核+四条信息集主线”的建模框架：先把母线能量平衡、储能状态转移、容量与功率限值、弃光上界写成与问题无关的公共约束组，再对每一问仅替换目标函数中的费用项、扩展决策变量的维度（是否引入调整量），并重新界定信息集。这样做的直接好处是：四问的模型结构完全一致，结论之间的差异可以精确归因，不会出现口径漂移（这正是 2021

年 C 题评阅要点中唯一被红字强调的失分点——各期方案必须随前一期的状态动态变化，而不能用一套静态方案贯穿始终）。

具体地，问题 1 是完全信息下的确定性经济调度，用于验证物理约束与求解器；问题 2 在完全无法获得当天预报的极端信息集下，仅用历史观测构造因果预测，并把 5 倍紧急电价转化为非对称损失；问题 3 引入官方多时刻预报，形成“0:00 基准计划 → 每 6 h 滚动调整 → 实际执行 → 状态递推”的闭环；问题 4 只把电价序列换成逐日逐时的波动电价，用于检验前两类策略在价格波动下的稳健性。

整体技术路线如图 1 所示。

2.2 各问的具体分析

问题一的分析。题目给定电价、负载与光伏发电功率预测，且“每天的电价和小区负载相同”，因此属于确定性优化。需要注意两个物理细节：其一，附件 2 中存在光伏功率高于负载的时段，若不允许弃光，模型将不可行，因此必须设置弃光变量；其二，题目要求 0:00 与 24:00 储电量相同，该闭环条件会改变最优解的取值，必须作为硬约束显式写入。

问题二的分析。本问的信息集最弱：题目没有给出任何当天预报，只能依赖已经发生的历史观测。这与问题一有本质区别——若仍沿用当天真实数据，紧急购电将几乎不会触发，模型退化为问题一。因此必须构造只使用历史数据的因果预测器，并让预测误差真实地转化为远高于计划成本的紧急购电费用。5 倍电价是本问的切入点：它使“高估光伏”的边际损失是“低估光伏”的 4 倍，二者的不对称性决定了最优预测应当偏向保守一侧。此外，由于计划在 0:00 锁定且日内不再修改，实际偏差只能依靠储能进行 10 min 级的实时缓冲。

问题三的分析。本问引入了 6:00、12:00、18:00 三个新增预报时刻，题目据此询问“是否需要引入”。回答这一问有两种做法：一是比较不同降尺度 / 校正方法的费用，二是把“允许使用的预报节点集合”本身作为消融变量。前者只能回答“哪种处理方法更好”，后者才能回答“这些预报值多少钱”。本文采用后者，构造 S_0 （仅 0:00）→ S_1 （+6:00）→ S_2 （+12:00）→ S_3 （+18:00）四组对照，每组保持负载预测方式、降尺度方法、结算口径、终端策略与实时反馈完全一致，只改变可用预报节点，从而得到每一时刻的增量价值。同时，题目的费用结构存在两种合理解释（计划与调整取差额结算，或计划费与调整费叠加），二者会导出不同的目标函数，因而最优调度本身也不同，必须分别完整重新优化而不能只做事后换算式计费。

问题四的分析。本问只替换电价序列。附件 4 的电价在日内与日间均大幅波动（全天最低 0.0076 元/kWh，最高 1.7936 元/kWh），峰谷价差显著扩大，为储能套利创造了更大空间。但价格序列本身同样构成信息边界：若在 0:00 优化时直接读入当天 144 个真实电价，就重演了问题二中的“未来信息穿越”。因此本文把价格也纳入信息集分层，分别

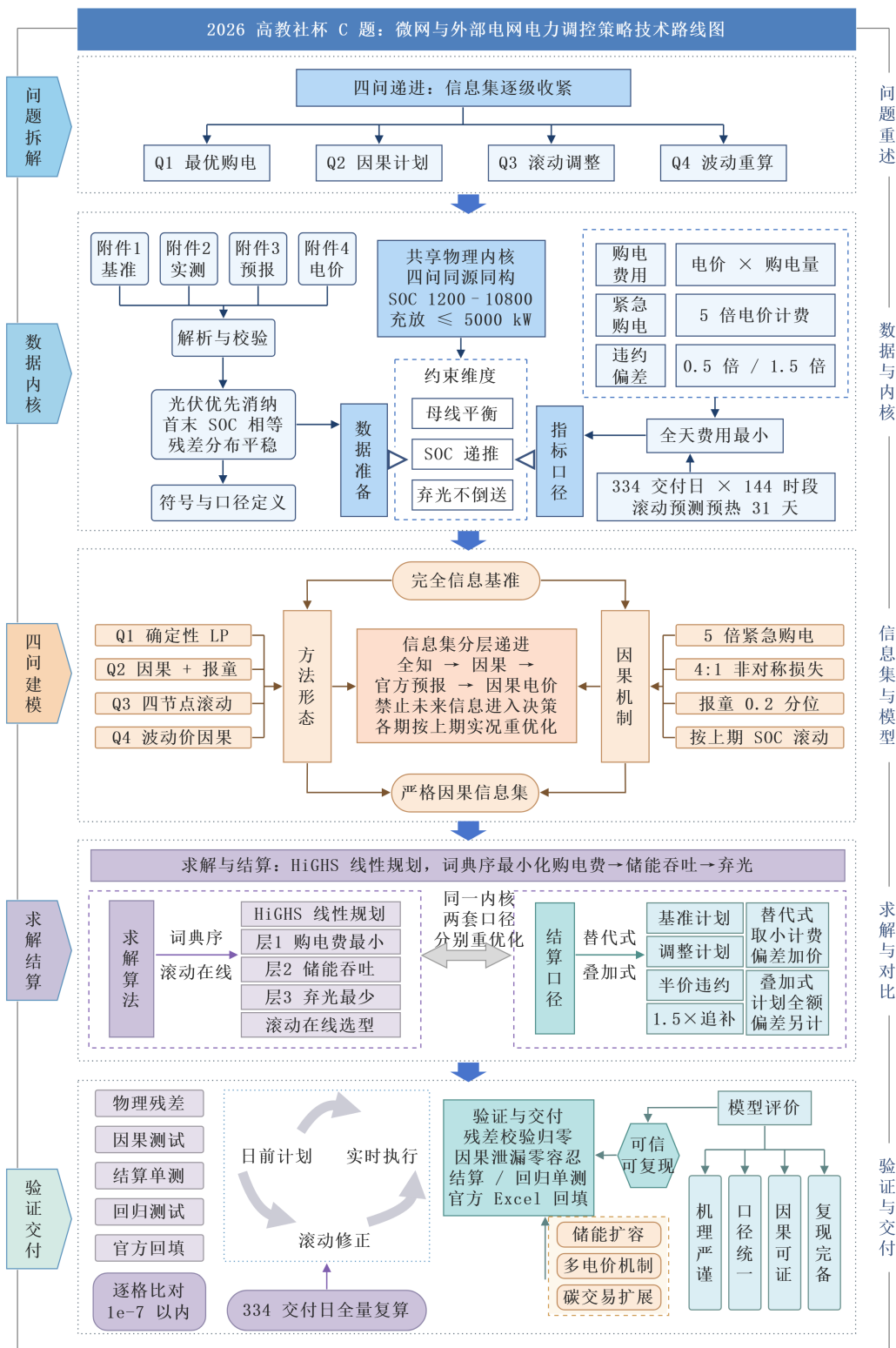


图 1 整体技术路线图

给出“价格完全已知”（理想化下界）与“因果价格预测”（可实现）两类结果，其差额即为电价信息的价值。

2.3 求解框架

全部四问均归结为线性规划，统一采用 HiGHS 单纯形 / 内点混合求解器求解。为避免线性规划在大规模最优面上返回无经济意义的退化解（例如同一时段既充电又放电），本文设计三层词典序目标：第一层优化真实费用，第二层在费用容差带内最小化储能吞吐量，第三层再最小化弃光量。统计口径上，所有电量均按 10 min 时段能量（kWh）核算，功率与能量的换算系数为 $\Delta t = 1/6$ h。

三、模型假设

为突出购电、光伏与储能之间的主要矛盾，并保证四问的信息边界严格一致，本文作如下假设。每条假设都将在相应模型中显式使用。

1. **调度周期与时间尺度：**以 10 min 为一个调度时段，一天划分为 $T = 144$ 个时段，单时段长度 $\Delta t = 1/6$ h。附件中标注 0:00+1 的列即当天 24:00。
2. **不允许向外部电网售电：**题目仅给出微网从外网购电的机制，未给出反向售电的交易规则，因此外网购电量恒为非负，光伏富余只能用于储能充电或弃光。
3. **储能建模的简化：**储能充放电效率在全年内恒为 $\eta_c = \eta_d = 0.9$ （口径敏感性见 7.5.2 节），不计自放电、容量衰减、循环寿命成本与启停损耗；同一时段不会同时充电与放电（由词典序目标自动排除）。
4. **储能功率约束按能量折算：**最大充放电功率 5000 kW 对应单时段最大充放电量 $5000\Delta t = 833.33$ kWh。
5. **初始储电量：**2025 年 1 月 1 日 0:00 储电量为 6000 kWh；1 月 31 天作为储能状态与预测误差的预热期，正式结果自 2 月 1 日起输出，这既与官方结果模板的日期范围一致，也避免冷启动误差污染统计。
6. **负载预测口径：**题目只为光伏提供了专门预报数据，未给出负载预报。因此本文用历史实际负载构造因果预测，方法族为持续法、7/14/30 日滚动均值、14/30 日线性加权均值与同星期均值，并在历史执行块上滚动在线选型，选型标准为平均绝对误差。
7. **附件 1 作为冷启动先验：**1 月 1 日之前无任何历史观测，此时以附件 1 给出的典型日负载与光伏预测曲线作为先验基预测。该先验仅在历史观测为空的当天使用，一旦有了至少一天观测便自动失效。
8. **实时储能反馈：**计划锁定后，日内允许储能依据当前时刻已经发生的真实光伏与当前储电量做 10 min 级调整，调整顺序为“先撤充电、再增放电、最后紧急购电”，不

得借助任何未来信息。

9. **终端储电量策略：**题目仅对问题一明确要求 $S_{0:00} = S_{24:00}$ 。本文主模型在问题二、三的计划层同样采用“计划终端储电量等于当日实际初始储电量”的日循环口径（记为 `daily_cycle`），以保证储能长期可持续运行；同时给出不做终端约束（`free`）的敏感性结果，二者的差异在 7.5.1 节报告。
10. **问题四的价格信息集：**价格完全已知情形视为理想化基准；因果情形的价格预测只使用决策时刻之前已经发生的附件 4 真实电价，跨日部分由历史同刻价格外推。

四、符号说明

本文使用的主要符号及其含义如表 1 所示。表中未列出的局部符号将在正文首次出现处说明。

表 1 主要符号说明

符号	单位	含义
d, t	—	日期下标与日内 10 min 时段下标 ($t = 1, \dots, 144$)
Δt	h	单个调度时段长度, $\Delta t = 1/6$
$L_{d,t}$	kW	小区负载功率
$P_{d,t}$	kW	光伏实际发电功率
$\hat{P}_{d,t}$	kW	决策时刻可用的光伏预测功率
$\pi_{d,t}$	元/kWh	外网正常购电单价
$G_{d,t}^p$	kWh	0:00 制定的计划购电量（第一阶段变量）
$G_{d,t}^a$	kWh	执行前最后一次有效调整后的购电量（第二阶段变量）
$C_{d,t}$	kWh	母线侧进入储能的充电量
$H_{d,t}$	kWh	储能送往母线的放电量
$S_{d,t}$	kWh	第 t 时段结束时储能内部储电量（状态变量）
$W_{d,t}$	kWh	未被利用的弃光电量
$E_{d,t}$	kWh	供电不足时的紧急购电量
$u_{d,t}, v_{d,t}$	kWh	调整购电量相对计划的上调量 $(G^a - G^p)^+$ 与下调量 $(G^p - G^a)^+$
η_c, η_d	—	充电效率与放电效率，均取 0.9
$S_{\min}, S_{\max}$	kWh	储电量上下限，1200 与 10800
$\bar{E}$	kWh	单时段充放电量上限, $\bar{E} = 5000\Delta t = 833.33$
$\mathcal{I}_d$	—	第 d 天决策时刻的信息集

五、数据预处理

附件 1 为某典型日的电价、小区负载与光伏发电预测，共 144 个时段；附件 2 给出 2025 年 1 月 1 日至 12 月 31 日每天 144 个时段的小区负载与光伏实际功率；附件 3 给出每天 0:00、6:00、12:00、18:00 四次发布的未来 24 小时整点光伏预报，共 $365 \times 4 \times 24$ 个数值；附件 4 给出 365×144 的实时电价。所有附件的时间标签均按“列 j 对应时刻 $j \times 10 \text{ min}$ ”的约定统一，不作人为平移。

原始功率单位为 kW，进入能量平衡前统一乘以 $\Delta t = 1/6 \text{ h}$ 换算为每时段电量 (kWh)；电价单位为元/kWh；弃光、紧急购电、充放电均以 kWh 表示。逐项检查表明：各附件所需区域均为有限数值，负载与光伏非负、电价为正，附件 2 与附件 3 的日期完全对齐，附件 3 每行严格按 0:00, 6:00, 12:00, 18:00 排列。全年负载电量 $40.52 \times 10^6 \text{ kWh}$ ，光伏电量 $20.25 \times 10^6 \text{ kWh}$ ，光伏高于负载的时段占 19.27%，其中 241 个时段的光伏富余功率超过 5000 kW，因此公共模型必须允许弃光。附件 1 电价的峰谷比为 3.76，而附件 4 的实时电价在 0.0076 ~ 1.7936 元/kWh 之间波动，两者的日平均电价均为 0.7662 元/kWh，使固定电价与波动电价的结果具有可比基础。

图 2 给出了数据总览。由图 2(a) 可见，典型日的光伏出力集中在 6:00 ~ 18:00，而电价的高峰出现在上午与傍晚两个时段，二者并不重合，这正是储能可以创造价值的空间；图 2(b)(c) 显示负载与光伏都存在明显的季节性：夏季负载与光伏同时走高，冬季负载抬升而光伏显著衰减；图 2(d) 表明实时电价在日内呈现规律的峰谷结构，但日间水平差异较大。

图 3 进一步量化了光伏与负载的匹配程度。净负荷（负载减光伏）在春、秋两季的中午时段大量转为负值，说明存在系统性富余；夏季光伏渗透率最高，6 ~ 8 月的中位数接近 1，即光伏出力与负载相当；冬季净负荷整体抬升，储能的价值主要体现在峰谷套利而非消纳光伏。

图 4 的相关性分析表明，净负荷与负载的相关系数为 0.92、与光伏的相关系数为 -0.52，而实时电价与日内相位的相关系数仅 0.04，即附件 4 的电价在日间水平上差异很大、不能由典型日曲线代替——这为问题四必须独立重算提供了依据。

六、模型的建立与求解

6.1 公共线性规划内核

四个问题共用同一套母线侧线性规划结构，本节先给出与问题无关的决策变量、目标函数与约束条件，后续各问只需替换信息集与费用项。

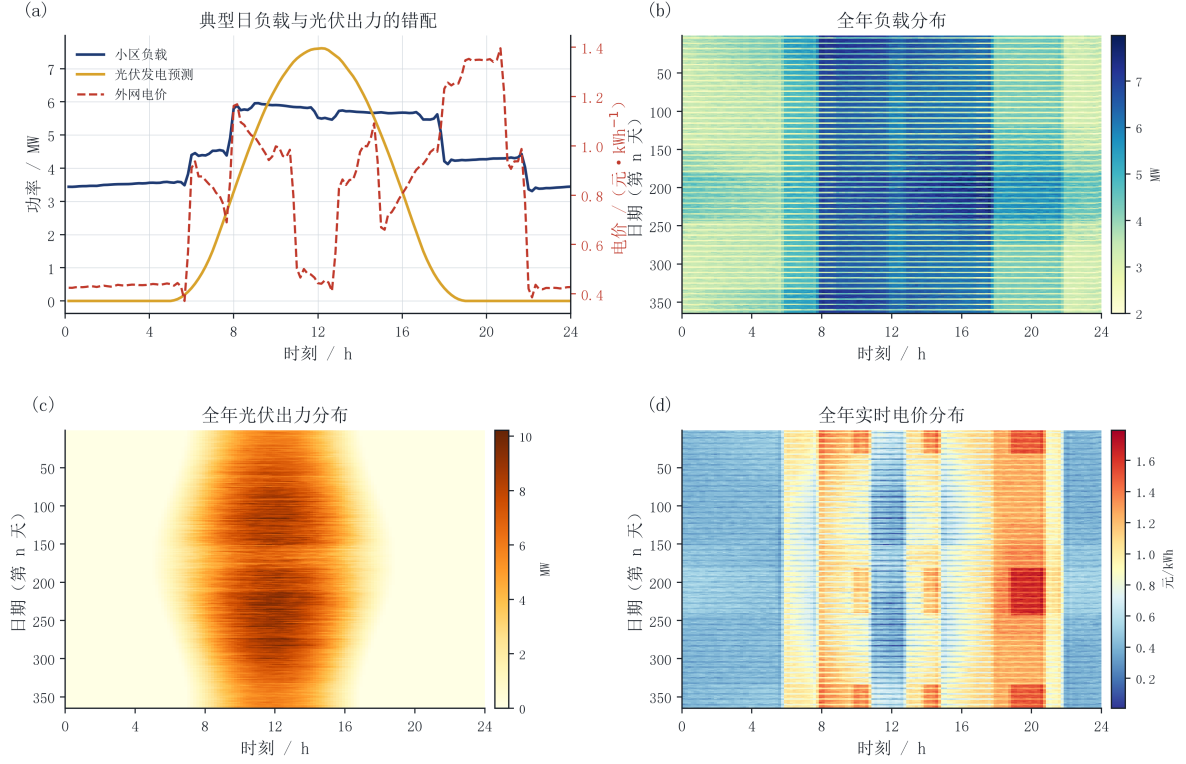


图2 附件数据总览

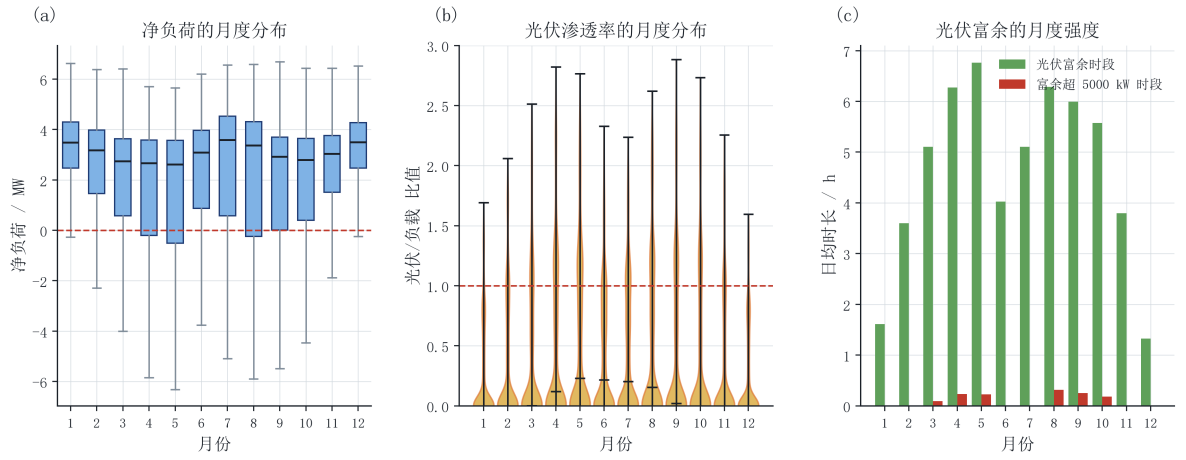


图3 净负荷与光伏渗透的季节特征

6.1.1 决策变量

以第 d 天第 t 个 10 min 时段为基本决策单元，定义非负决策变量：

- 计划购电量 $G_{d,t}^p \geq 0$ ：0:00 制定的、全天固定不再改动的外网购电量 (kWh)；
- 调整购电量 $G_{d,t}^a \geq 0$ ：执行前最后一次有效调整后的外网购电量 (kWh)；问题一中 $G^a \equiv G^p$ ；
- 充电量 $C_{d,t} \geq 0$ 与放电量 $H_{d,t} \geq 0$ ：母线侧进入 / 流出储能的电量 (kWh)；

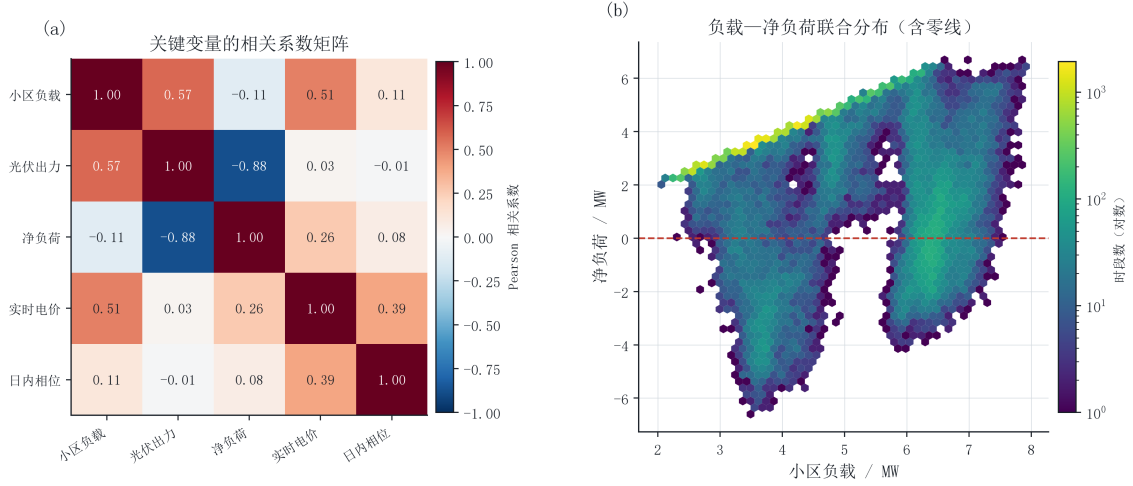


图 4 关键变量相关性与联合分布

- 弃光电量 $W_{d,t} \geq 0$: 光伏中未被负载或储能吸收的部分 (kWh);
- 紧急购电量 $E_{d,t} \geq 0$: 实际执行时供电不足的补充量 (kWh);
- 储电量 $S_{d,t}$: 状态变量, 第 t 时段结束时储能内部电量 (kWh)。

需要特别强调的是 G^p 与 G^a 的角色差异: G^p 是在不确定性揭晓之前确定的与场景无关的变量; G^a 、 C 、 H 、 W 、 E 、 S 则是与真实场景相关的执行变量。这一“计划—执行”两阶段结构是问题二与问题三的建模基础。

6.1.2 目标函数

问题一至问题四的购电费用可统一写成

$$\min F = \underbrace{\sum_d \sum_t \pi_{d,t} G_{d,t}^p}_{\text{计划购电费}} + \underbrace{\sum_d \sum_t \pi_{d,t} \Phi(G_{d,t}^p, G_{d,t}^a)}_{\text{计划与执行的偏差费用}} + \underbrace{\sum_d \sum_t 5 \pi_{d,t} E_{d,t}}_{\text{紧急购电费}}, \quad (1)$$

其中偏差费用函数 $\Phi(\cdot)$ 由题目的结算规则决定:

$$\Phi(G^p, G^a) = \begin{cases} 0, & \text{问题一 (无调整机制),} \\ 0, & \text{问题二 (计划锁定, 缺口全部走紧急购电),} \\ |G^a - G^p| \cdot \begin{cases} 0.5, & G^a < G^p \\ 1.5, & G^a > G^p \end{cases}, & \text{问题三 (差额或叠加两种口径, 见 6.4.2 节).} \end{cases} \quad (2)$$

式 (1) 中, 第一项是 0:00 已经确定的计划购电费, 与真实场景无关; 第三项中因子 5 来自题目“紧急购电电价是交易时刻电价的 5 倍”。问题三的两结结算解释将在 6.4.2

节展开。

6.1.3 约束条件

约束 1（母线能量平衡）。这是全模型的核心守恒关系：任一 10 min 时段内，微网的电力来源必须等于电力消耗。

$$\underbrace{G_{d,t} + P_{d,t}\Delta t + H_{d,t} + E_{d,t}}_{\text{电力来源}} = \underbrace{L_{d,t}\Delta t + C_{d,t} + W_{d,t}}_{\text{电力消耗}}, \quad \forall d, t. \quad (3)$$

式中 $P_{d,t}\Delta t$ 为该时段可用的光伏电量， $W_{d,t}$ 为被主动舍弃的光伏电量（弃光）。该式之所以必须写成等式而非“供电量不低于负载”，是因为光伏富余时必须有出口，否则模型将不可行。

约束 2（储电量状态转移方程）。储能是唯一跨时段耦合的元件，其状态递推本模型最关键的约束：

$$S_{d,t} = S_{d,t-1} + \eta_c C_{d,t} - \frac{H_{d,t}}{\eta_d}, \quad S_{d,0} = S_d^{\text{init}}, \quad \forall d, t = 1, \dots, 144. \quad (4)$$

式 (4) 的物理含义是“本时段末储电量 = 上一时段末储电量 + 本时段充电量经充电效率折算后的量 - 本时段放电按放电效率折算后的池内消耗量”。充电时只有 $\eta_c C$ 真正进入电池，放电时为送出 H 需从池内取出 H/η_d ，这正是效率损失的来源，也保证了储能不会成为“凭空产电”的装置。需要强调的是， $S_{d,t}$ 是全模型唯一的跨时段状态变量：**每一天、每一个预报时刻的优化都必须以当时实际测得的储电量作为初值**，因此各期的计划必然互不相同——这正是本模型区别于“一次性静态方案”的关键。

约束 3（储电量安全区间）。为避免过充过放，储电量必须始终保持在安全区间内：

$$1200 \leq S_{d,t} \leq 10800, \quad \forall d, t. \quad (5)$$

约束 4（充放电功率上限）。最大充放电功率为 5000 kW，折算到单时段电量为

$$0 \leq C_{d,t} \leq \bar{E}, \quad 0 \leq H_{d,t} \leq \bar{E}, \quad \bar{E} = 5000\Delta t = 833.33 \text{ kWh}. \quad (6)$$

约束 5（购电非负与不售电）。题目未给出微网向外网售电的机制，因此

$$G_{d,t}^p \geq 0, \quad G_{d,t}^a \geq 0, \quad E_{d,t} \geq 0. \quad (7)$$

约束 6（弃光上界）。弃光不能超过该时段实际可用的光伏电量：

$$0 \leq W_{d,t} \leq P_{d,t}\Delta t. \quad (8)$$

约束 7（问题一首末储电量相等）。题目对问题一明确要求 0:00 与 24:00 的储电量相同：

$$S_{d,144} = S_{d,0} = 6000 \text{ kWh}. \quad (9)$$

问题二、问题三的计划层采用“计划终端储电量等于当日实际初始储电量”的日循环口径（daily_cycle），其敏感性见 7.5.1 节。

约束 8（信息集因果性）。这是本文区别于常规做法的关键约束。设 $\mathcal{I}_{d,t}$ 为第 d 天第 t 时段决策时刻可获得的信息集，则所有决策变量必须满足

$$G_{d,t}^p, G_{d,t}^a, C_{d,t}, H_{d,t}, W_{d,t} \perp\!\!\!\perp \{L_{d,\tau}, P_{d,\tau}, \pi_{d,\tau} \mid \tau \geq t\}, \quad (10)$$

即决策不得依赖决策时刻之后才会发生的真实负载、真实光伏与真实电价。这一约束与 (1)~(7) 不同，它不能写成线性不等式，而是通过程序的信息流设计强制保证，并由 mutation 测试自动验证（见七、节）。

6.1.4 线性规划模型汇总

将式 (1)~(8) 合并，四问的统一模型为

$$\begin{cases} \min F \text{ (式 (1))} \\ \text{s.t. 式 (3) (母线平衡), 式 (4) (SOC 递推), 式 (5) (容量),} \\ \quad \text{式 (6) (功率), 式 (7) (非负), 式 (8) (弃光), 式 (10) (信息集).} \end{cases} \quad (11)$$

模型中所有约束均为线性等式或不等式，决策变量连续非负，因此式 (11) 是标准线性规划，可由 HiGHS 在毫秒级求解。线性规划的最优解可能落在高维最优面上，从而出现“同一时段既充电又放电”这类无经济意义的退化解。为此本文设计三层词典序目标：第一层求解真实费用 F^* ；第二层在约束 $F \leq F^* + \varepsilon$ （ ε 取相对 10^{-7} 的容差）下最小化储能吞吐量 $\sum(C_{d,t} + H_{d,t})$ ；第三层继续在同一容差带内最小化弃光量 $\sum W_{d,t}$ 。若某一层因数值退化不可行，则自动去掉前序最优值约束重解该层，保证输出始终物理可行。

6.2 问题一：确定性日前调度

6.2.1 模型建立

问题一中附件 1 的电价、负载与光伏预测均视为已知， $G^a \equiv G^p$ ， $E \equiv 0$ ， $\Phi \equiv 0$ 。目标函数退化为

$$\min F_1 = \sum_{t=1}^{144} \pi_t G_t, \quad (12)$$

约束为式 (3)~(8) 加上首末储电量约束式 (9)。式 (12)~(9) 构成问题一的确定性线性规划模型。

6.2.2 求解结果

求解得到全天计划购电量 59482.70 kWh, 购电费 35126.95 元; 储能全天充电 20740.67 kWh、放电 16799.94 kWh, 首末储电量均为 6000 kWh, 全天无弃光。作为对照, 若完全禁用储能 (令 $C \equiv H \equiv 0$) 并按正常电价补足缺口, 全天费用为 48052.05 元, 因此储能带来的净节省为 12925.10 元, 降幅 26.8981%。

表 2 按题目要求的格式给出问题一的结果 (完整 144 时段明细见支撑材料 `result1.xlsx`)。

表 2 问题一指定时段购电量及全天购电量与购电费

时间段	购电量/kWh	时间段	购电量/kWh	时间段	购电量/kWh
10:00–10:10	0.00	12:00–12:10	486.40	14:00–14:10	0.00
16:00–16:10	394.93	18:00–18:10	636.98	20:00–20:10	0.00
全天购电量/kWh	59482.70	全天购电费/元	35126.95		

表 3 问题一储能分时段充放电量与首末储电量

时间段	充电量/kWh	放电量/kWh	时间段	充电量/kWh	放电量/kWh
0:00–4:00	4500.00	0.00	12:00–16:00	5286.04	91.10
4:00–8:00	833.33	6365.84	16:00–20:00	0.00	5780.13
8:00–12:00	4787.96	1703.00	20:00–24:00	5333.33	2859.87
0:00 储电量/kWh	6000.00		24:00 储电量/kWh	6000.00	

由图 5 可见, 储能的运行策略呈现出清晰的“低储高放”特征: 在 0:00 ~ 6:00 的低价时段与中午光伏富余时段充电, 在 10:00 ~ 12:00 与 18:00 ~ 20:00 的高价时段放电, 储电量始终处于 1200 ~ 10800 kWh 的安全区间内。这验证了公共线性规划内核能够同时利用电价峰谷与光伏富余完成电量的时间转移。

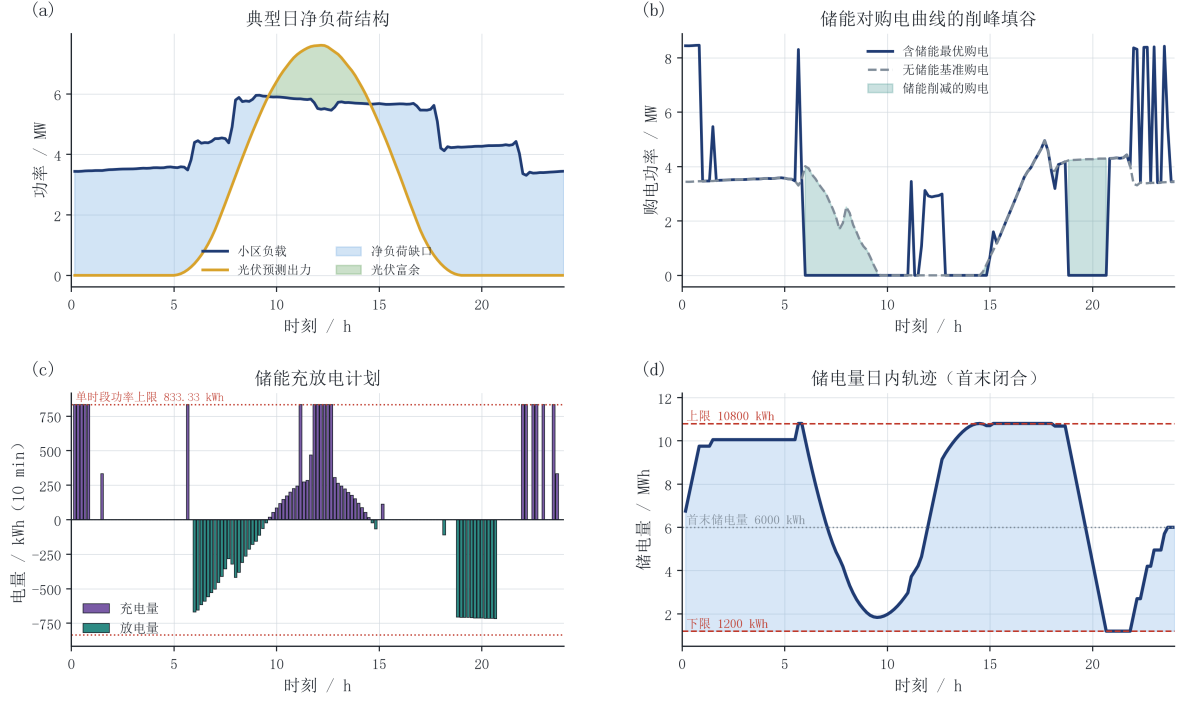


图 5 问题一购电量、储能充放电与储电量曲线

6.3 问题二：因果预测与实时储能反馈

6.3.1 信息集界定与建模难点

问题二与问题一的信息集有本质区别：题目没有给出负载预报，也没有给出光伏预报，只提供了历史实际数据。因此当天的负载与光伏在 0:00 都是未知的随机变量，0:00 只能依据历史观测制定计划。若沿用问题一的做法直接读入当天真实负载，则紧急购电几乎不会触发，模型退化为问题一——这正是必须避免的“未来信息穿越”。

于是问题二可写成如下的两阶段形式：

$$\min_{G^p} \mathbb{E} \left[\sum_t \pi_t G_t^p + \sum_t \pi_t \Psi(G_t^p, G_t^a) + \sum_t 5\pi_t E_t \right], \quad (13)$$

其中 $\Psi \equiv 0$ （计划锁定，不存在违约或增购机制），期望取在负载—光伏的联合不确定性上，约束为式 (3)~(10)（去掉问题一首末约束）。式 (13) 中只有 G^p 与场景无关， G^a, C, H, W, E, S 均为场景相关变量。

6.3.2 风险定价：从 5 倍电价到 0.2 分位点

直接用条件均值预测光伏并不最优。考虑单位电量上“高估光伏”与“低估光伏”的边际损失：若预测偏高 $\delta > 0$ ，计划购电将少买 δ ，真实缺口需按 5π 紧急购入，相比按 π 正常购入多支出 $4\pi\delta$ ；若预测偏低 δ ，计划多买 δ ，仅多支出 $\pi\delta$ 。因此单位偏差的非对称损失为

$$\ell(\hat{P} - P) = 4\pi(\hat{P} - P)^+ + \pi(P - \hat{P})^+, \quad (14)$$

即高估的权重是低估的 4 倍。设光伏的条件分布为 F ，最小化 $\mathbb{E}[\ell]$ 得到的一阶条件为

$$F(\hat{P}^*) = \frac{b}{a+b} = \frac{1}{4+1} = 0.2, \quad \hat{P}^* = F^{-1}(0.2), \quad (15)$$

其中 $a = 4$ 、 $b = 1$ 分别为高估与低估的损失权重。式 (15) 就是经典报童模型的最优分位点结论 [10]：5 倍紧急电价蕴含的最优光伏点预测应取预测分布的 0.2 分位数。这一“按损失不对称程度选择分位点”的思路与电力市场中发电商基于报童模型制定投标策略的做法一致 [11]。本文用独立数值实验验证了该结论：按式 (15) 生成的预测再经一次直接最小化式 (14) 的平移寻优，得到的最优再偏移为 0.0，即式 (15) 已是最优。

据此构造光伏预测：设第 m 个基方法（持续法、7/14/30 日滚动均值、14/30 日线性加权均值）给出基预测 $B_{d,t}^m$ ，其在历史窗口上的残差集合为 $\mathcal{R}_m$ ，则

$$\hat{P}_{d,t}^{(m)} = \max \{0, B_{d,t}^m - Q_q(\mathcal{R}_m)\}, \quad (16)$$

其中 $Q_q(\mathcal{R}_m)$ 为残差的经验分位数。由式 (15)，取 $q = 0.2$ 即等价于把基预测平移到预测分布的 0.2 分位数。候选方法的在线选型准则直接使用式 (14) 的费用代理：

$$J_m = \sum_{d' \in \mathcal{W}} \sum_t \pi_{d',t} \left[4 \left(\hat{P}_{d',t}^{(m)} - P_{d',t} \right)^+ + \left(P_{d',t} - \hat{P}_{d',t}^{(m)} \right)^+ \right] \Delta t, \quad (17)$$

其中窗口 $\mathcal{W}$ 为过去 30 天。选型只使用已经发生的误差，因此整体满足式 (10)。

负载侧同理构造因果预测器：候选方法族为持续法、7/14/30 日滚动均值、14/30 日线性加权均值与同星期均值；日内 6:00 之后的决策还可利用当天已经执行完的时段对基预测做水平校正。由于负载预测的损失基本对称，选型准则采用平均绝对误差。

6.3.3 计及实时储能反馈的执行阶段

0:00 计划一经确定便不再修改，但日内允许储能依据当前时刻已发生的真实光伏做 10 min 级调整。记计划值为 $(\cdot)^p$ ，执行规则为：

- (1) 若真实供电不足，记该时段缺口电量为 $D_t = L_{d,t}\Delta t + C_t^p - G_t^p - P_{d,t}\Delta t - H_t^p > 0$ ：先取消充电，取 $C_t = \min(C_t^p, D_t)$ ，剩余缺口 $D_t - C_t$ 再由增加放电补充（受功率上限与储电量下限约束）；若仍不足，最后才按 5 倍电价紧急购电。
- (2) 若真实光伏富余，记富余电量为 $S_t > 0$ ：先减少放电，再增加充电（受功率上限与储电量上限约束），仍有余量则弃光。

该规则在每个时段只读取当前的真实光伏与当前储电量，不涉及任何未来信息。若关闭该反馈（ $C_t \equiv C_t^p$ 、 $H_t \equiv H_t^p$ ，仅做储电量边界的被动钳位），缺口将全部转为 5 倍紧急购电，因此二者的费用差即为实时储能调节的价值。

6.3.4 求解结果与原因拆解

主方案（因果负载预测 + 非对称光伏风险边际 + 实时储能反馈）在 2 月至 12 月共 334 天的总费用为 14773804.78 元，其中计划购电费 12877657.23 元、紧急购电费 1896147.55 元；紧急购电量 348036.96 kWh，弃光 2326222.67 kWh，共 232 天发生紧急购电；负载预测平均绝对误差 188.88 kW，光伏预测平均绝对误差 198.14 kW。

为回答“原先偏低的费用究竟由什么造成”，本文固定其余条件不变，只改变单一因素，构造四组对照（表 4）：A 组用当天真实负载（完美信息基准），B 组用因果负载预测，C 组把光伏风险边际换成无安全下修的点预测，D 组关闭实时储能反馈。

表 4 问题二单因素对照实验

方案	负载信息	光伏预测	实时反馈	总费用/元	紧急购电/kWh	弃光/kWh
A	完美	风险边际	开	13275988.13	61482.52	2000031.09
B	因果	风险边际	开	14773804.78	348036.96	2326222.67
C	因果	点预测	开	16038813.56	656904.88	1688048.18
D	因果	风险边际	关	15360709.59	672280.72	2566068.96

由图 6(c) 的瀑布分解可以量化三个因素：

$$\begin{aligned}
\Delta C_{\text{load}} &= C_B - C_A = 1497816.65 \text{ 元}, \\
\Delta C_{\text{PVrisk}} &= C_C - C_B = 1265008.78 \text{ 元}, \\
\Delta C_{\text{feedback}} &= C_D - C_B = 586904.81 \text{ 元}.
\end{aligned} \tag{18}$$

结果表明：原先把当天真实负载当作 0:00 已知输入，会使全年费用被低估 **1497816.65 元**（占修正后费用的 **10.14%**），这是此前结果偏低的首要原因；其次是光伏风险定价，去掉 0.2 分位数下修后费用上升 1265008.78 元，因为点预测在高估光伏时会把大量缺口推到 5 倍电价上；10 min 实时储能反馈贡献 586904.81 元的节省，关闭后紧急购电量由 348036.96 kWh 增至 672280.72 kWh。

图 6(b)(d) 还显示，A 组的弃光量最大而紧急购电最少：完美负载信息使计划与执行几乎无偏差，光伏富余被完整地“计划”进储能与弃光通道，这进一步印证了 A 组是一个不可实现的理想化下界。

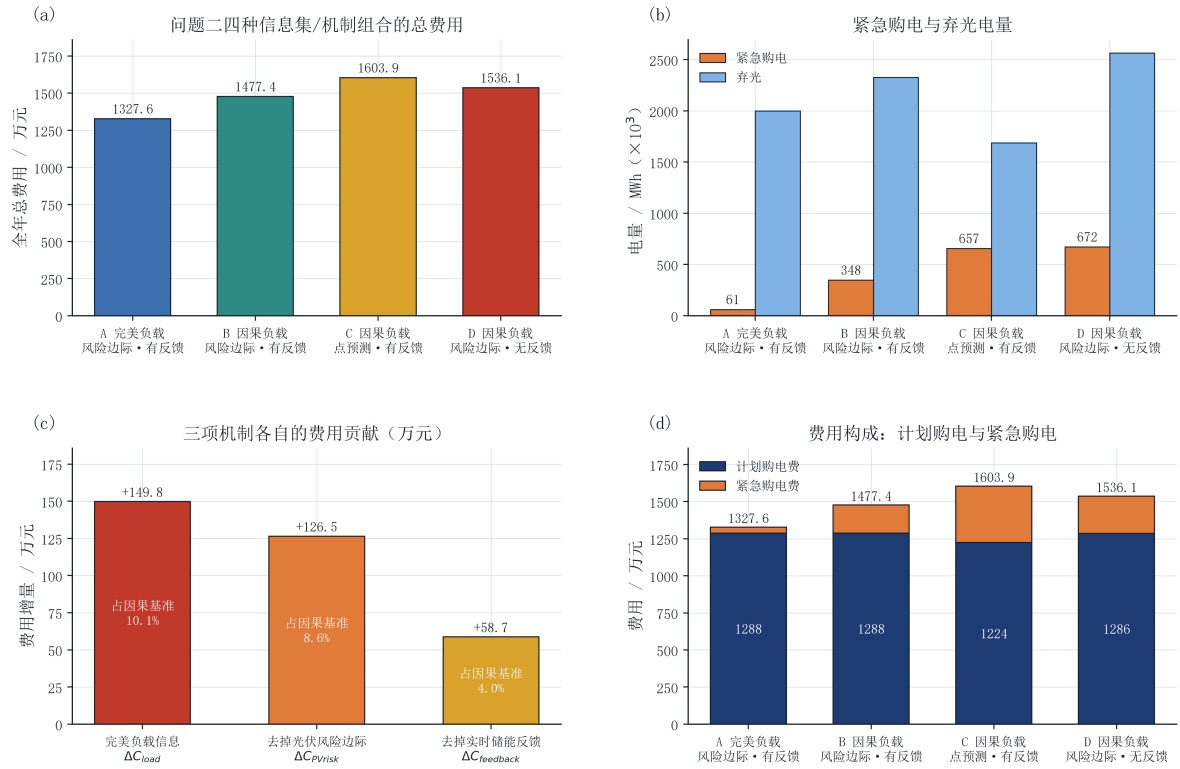


图 6 问题二费用构成与原因拆解

6.4 问题三：多时刻预报滚动优化与预报价值消融

6.4.1 滚动优化框架

问题三在每天 0:00、6:00、12:00、18:00 可获得未来 24 h 整点的光伏预报。本文的决策链为：

- (1) **0:00 基准计划**：由节点 0 的预报降尺度得到 144 个时段的光伏预测，求解式 (11) 得到全天基准计划 G^p ，并执行 0:00 ~ 6:00 的 36 个时段；
- (2) **滚动调整**：在 6:00、12:00、18:00 各节点，以当时实际测得的储电量为初值，用新到达的预报重新优化未来 24 h 的购电与储能，但只执行到下一个决策节点前的 36 个时段；
- (3) **状态递推**：执行块结束时把实测储电量传递给下一个节点，跨午夜时只传递储电量，次日 0:00 重新生成新的基准计划。

这一“重优化—短执行—状态传递”的结构使每一期的计划都建立在上一期的实际执行结果之上，因而各期方案互不相同，正是状态递推类问题（如 2021 年 C 题库存递推）所要求的建模方式，也与滚动时域预测控制在微网能量管理中的标准做法一致 [6, 7]。

附件 3 给出的是整点值，需降尺度到 10 min 端点。本文比较两种方式：阶梯保持（每个小时值重复 6 次）与线性插值。图 7(c) 显示，在真正会被执行的 6 h 块上，线性

插值的平均绝对误差为 110.22 kW，显著低于阶梯保持的 294.37 kW，因此主方案采用线性插值。此外，本文对每个发布节点分别维护最近 30 天已执行块的预报残差，并按式 (14) 的费用代理在 0.80 ~ 0.95 分位数间在线选择下修边际，实现分发布节点的因果偏差校正。

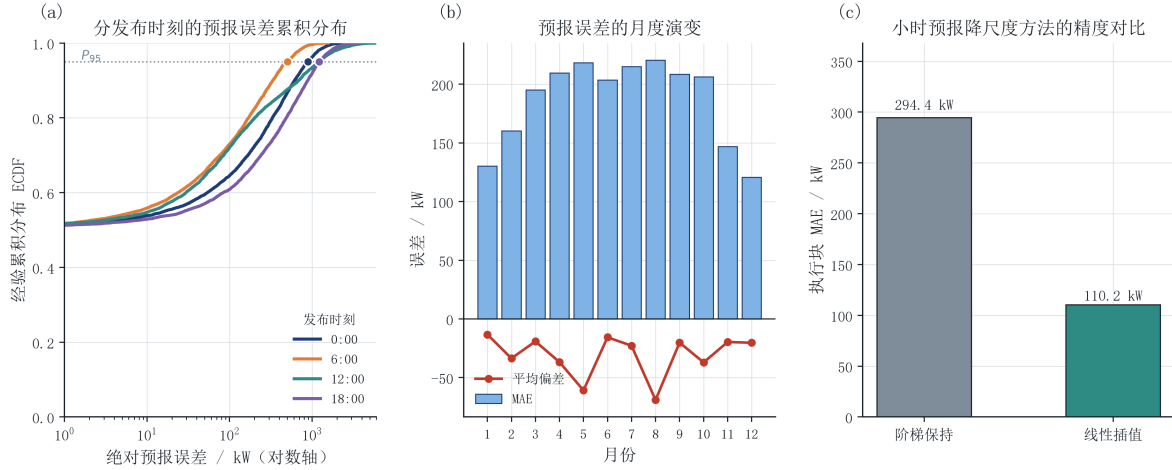


图 7 附件 3 预报误差诊断与降尺度对比

6.4.2 两种结算口径的严格建模

题目对问题三的费用描述存在两种自然解释。设 G^p 为计划购电量、 G^a 为执行前最后一次有效调整后的购电量，并记上调量 $u = (G^a - G^p)^+$ 、下调量 $v = (G^p - G^a)^+$ ，则：

口径一 (replacement, 差额结算): 认为违约费与增购费替代了原有电费，即

$$C_t^{\text{rep}} = \pi_t [\min(G_t^p, G_t^a) + 0.5 v_t + 1.5 u_t], \quad (19)$$

当 $G^a < G^p$ 时实付 $\pi(G^a + 0.5v)$ ，当 $G^a > G^p$ 时实付 $\pi(G^p + 1.5u)$ 。

口径二 (additive, 叠加结算): 认为调整费用叠加在计划电费之上，即

$$C_t^{\text{add}} = \pi_t G_t^p + 0.5 \pi_t v_t + 1.5 \pi_t u_t. \quad (20)$$

两种口径的目标函数不同，因此最优调度本身也不同。本文把 u_t, v_t 显式引入线性规划，并施加等式约束

$$G_t^a - G_t^p = u_t - v_t, \quad u_t, v_t \geq 0, \quad (21)$$

从而把式 (19)、(20) 中的分段费用函数精确线性化。需要说明的是，式 (21) 必须是等式：若放松为不等式，当 $G^a < G^p$ 时求解器可取 $u = v = 0$ ，从而漏付 $0.5\pi(G^p - G^a)$ 的违约费用，使目标函数系统性低估下调成本、调度不再最优。线性化后的目标与按题意直接回代的差异经验证小于 10^{-12} 元（见七、节）。

两种口径下问题三的完整模型均在相同的信息集与约束下**分别重新优化**，而不是用同一个调度方案事后套两个公式计费。

6.4.3 预报价值消融实验

题目询问“是否需要引入其他时刻的预报”。为回答这一问题，本文把**允许使用的预报节点集**作为唯一消融变量，构造四组方案：

$$S_0 = \{0:00\}, \quad S_1 = \{0:00, 6:00\}, \quad S_2 = \{0:00, 6:00, 12:00\}, \quad S_3 = \{0:00, 6:00, 12:00, 18:00\}, \quad (22)$$

四组之间负载预测方式、降尺度方法、偏差校正、结算口径、终端策略与实时反馈保持完全一致。定义各时刻的增量价值为

$$V_6 = C_{S_0} - C_{S_1}, \quad V_{12} = C_{S_1} - C_{S_2}, \quad V_{18} = C_{S_2} - C_{S_3}, \quad V_{\text{total}} = C_{S_0} - C_{S_3}. \quad (23)$$

6.4.4 求解结果

replacement 口径下的四组结果列于表 5。

表 5 问题三预报节点消融结果

方案	可用预报时刻	总费用/元	增量价值/元	累计价值/元	紧急购电/kWh	调整电量（上/下）/kWh
S_0	0:00	15731164.40	—	0.00	539486.13	0.00/0.00
S_1	+6:00	14864532.14	866632.26	866632.26	324851.71	425180.00/187474.65
S_2	+12:00	14324451.62	540080.52	1406712.78	239029.96	625606.45/752255.22
S_3	+18:00	13727989.25	596462.37	2003175.15	78468.70	819067.49/853615.69

由图 8 与表 5 可以得到明确结论：

$$V_6 = 866632.26 \text{ 元}, \quad V_{12} = 540080.52 \text{ 元}, \quad V_{18} = 596462.37 \text{ 元}, \quad V_{\text{total}} = 2003175.15 \text{ 元}, \quad (24)$$

即引入 6:00、12:00、18:00 的新增预报是必要的：三个时刻的增量价值均为正，全年累计节省 2003175.15 元，其中 6:00 预报的价值最大（866632.26 元），因为 0:00 的预报距实际执行已过去 6 小时、误差最大，6:00 的新预报带来的信息增益最显著。图 8(c) 的季节分解显示，夏、冬两季的改善最明显——夏季光伏出力大且波动强，冬季负载高、缺口风险大，两者的预报价值都高于过渡季节。

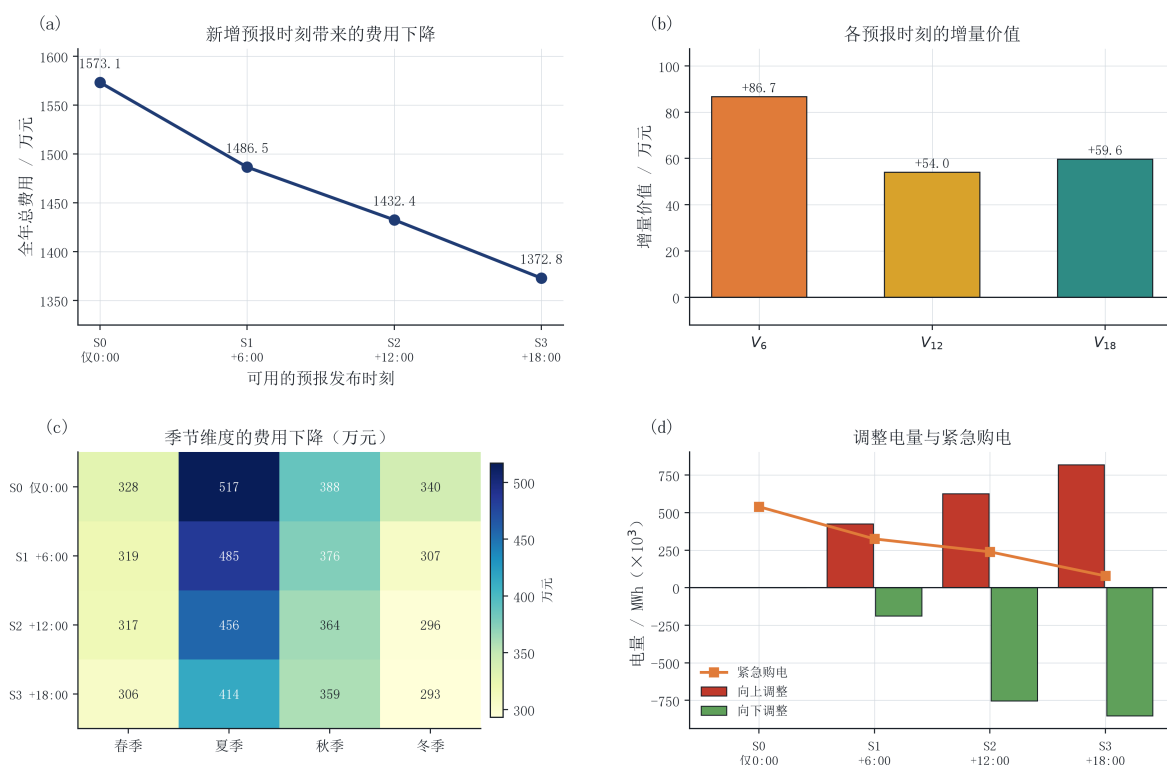


图 8 预报时刻的增量价值与季节分解

主方案 (S_3 , replacement 口径) 的全年总费用为 13727989.25 元, 其中 0:00 基准计划费用 12638811.91 元、调整结算费用 13327020.02 元、紧急购电费用 400969.22 元, 全年紧急购电仅 78468.70 kWh。相对问题二的因果主方案, 滚动调整节省了 1045815.53 元, 并把紧急购电量从 348036.96 kWh 降至 78468.70 kWh。

图 9 对比了两种结算口径。additive 口径的总费用整体高于 replacement 口径 (S_3 为 13906874.17 元), 因为它在计划电费之外重复计入了调整费用; 但两种口径下 V_{total} 分别为 2003175.15 元与 1824290.22 元, 均为正, 说明“新增预报值得使用”这一结论对结算口径稳健。

6.4.5 典型日调度过程

图 10 给出 2025 年 6 月 21 日的调度全景。该日为夏季高辐照日, 光伏峰值超过 8 MW, 而小区负载仅约 3.8 MW, 二者严重错配。

由图 10(a) 可见, 储能日凌晨与上午以最大功率充电 (紫色曲线触及 -5 MW 的单时段功率上限), 把光伏富余尽可能转移到电池中; 图 10(c) 显示储电量从 1.2 MWh 一路充至 10.8 MWh 的上限并在 11:00 ~ 20:00 长时间维持饱和, 随后在傍晚放电回落——这说明 12 h 的储能容量不足以吸纳夏季全部光伏富余。由于题目不允许向外部电网售电, 饱和之后的富余光伏只能弃掉: 图 10(d) 显示 12:00 ~ 16:00 出现集中弃光, 单时段最高超过 4500 kWh; 而在同一时段, 紧急购电几乎为零。

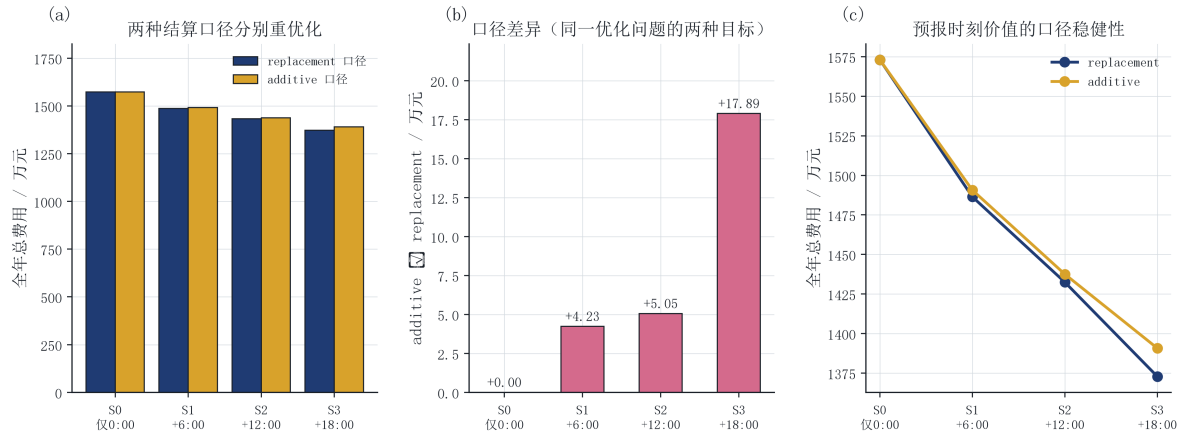


图 9 两种结算口径分别重优化的对比

图 10(b) 对比了 0:00 基准计划与最终调整购电：两者走势基本一致，说明 0:00 的预报已经把握了当日的主要形态；红色（向上调整）与绿色（向下调整）区域集中出现在 18:00 ~ 21:00 的傍晚时段，这正是 6:00、12:00、18:00 新预报对午后云量变化做出修正的结果。

这一典型日也揭示了模型的主要经济矛盾：夏季的正午弃光与傍晚高价购电同时存在，二者都由储能容量与充放电功率上限决定，而非由预测精度决定。这解释了为什么图 8(c) 中夏季的费用改善幅度最大——后续预报能在傍晚前提前调整储能的放电节奏，把有限的容量留给真正高价时段。

6.5 问题四：波动电价下的重算与价格信息边界

6.5.1 模型迁移

问题四保持问题二与问题三的数据范围、预测信息集、储能反馈与结算规则完全不变，只把固定典型日电价 π_t 替换为附件 4 的 $\pi_{d,t}$ 。需要特别处理的是价格本身的信息边界：若在 0:00 优化时直接读入当天 144 个真实电价，就与问题二中“把真实负载当作已知”是同一类未来信息穿越。因此本文引入因果电价预测器，其候选方法族与负载侧一致（持续法、7/14/30 日滚动均值、14/30 日线性加权均值、同星期均值），只使用决策时刻之前已经发生的真实电价，跨日部分由历史同刻价格外推；候选的在线选型同样只依据历史已实现误差（平均绝对误差）。据此给出两类结果：

- **perfect price**: 优化时已知当天真实电价（理想化下界，用于度量电价信息的价值）；
- **causal price**: 优化时只使用因果电价预测（可实现结果）。

6.5.2 求解结果与价格信息价值

四组结果列于表 6。

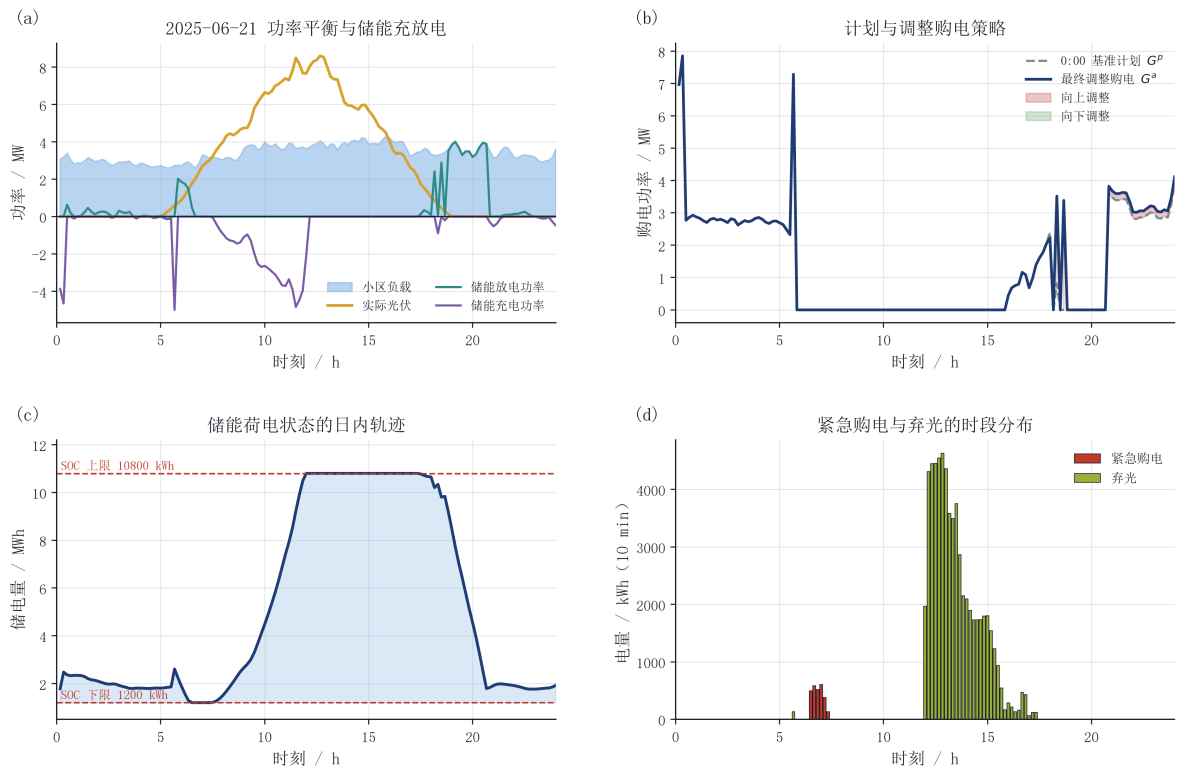


图 10 典型日调度过程（2025-06-21）

表 6 问题四波动电价下的四组结果

方案	价格信息	总费用/元	紧急购电/kWh	弃光/kWh
Q4-2	完全已知	15523884.83	355425.99	2365390.66
Q4-2	因果预测	15590096.88	354599.54	2322027.45
Q4-3	完全已知	14389193.11	97323.47	1672427.32
Q4-3	因果预测	14524235.75	95823.18	1666685.66

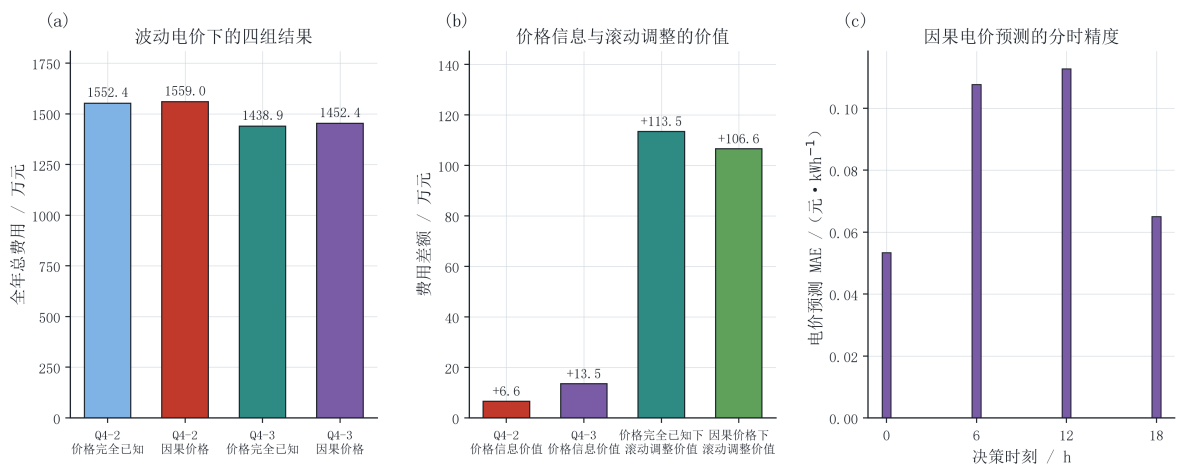


图 11 波动电价下的费用与价格信息价值

据此可以量化未来电价完美可知所带来的理想化收益：

$$V_{\text{price},Q2} = 66212.05 \text{ 元 (0.42\%)}, \quad V_{\text{price},Q3} = 135042.63 \text{ 元 (0.93\%)}, \quad (25)$$

即若能在 0:00 预知当天全部 144 个电价，问题二与问题三的费用分别只能再降低约 6.6 万元与 13.5 万元，**均不足总额的 1%**。这一结果看上去反直觉，但有两点原因：其一，本文的因果电价预测已经相当准确（全年平均绝对误差 0.0845 元/kWh，相对日均电价约 11%），而且逐日持久性在电价这种强日周期序列上表现良好；其二，在这类不允许售电的微网中，储能的套利空间受容量（12000 kWh）与功率（5000 kW）的双重限制，即使完全掌握价格走势也无法大幅增加可转移的电量。

把这一结果与问题二的信息价值对比，可以得到一个更重要的结论：**信息价值高度不对称**——负载信息被完全掌握时费用被低估 1497816.65 元，而电价信息被完全掌握时仅能再省 66212.05 元，前者约为后者的 23 倍。因此在实际工程中，应把有限的预测投入优先投向负载与光伏，而不是电价。同时也说明：把“价格完全已知”当作可实现结果虽然会带来乐观偏差，但该偏差在此场景下很小（< 1%），这为问题四采用何种价格假设提供了量化依据。

另一方面，动态电价下的滚动调整价值为

$$C_{Q4-2}^{\text{perfect}} - C_{Q4-3}^{\text{perfect}} = 1134691.72 \text{ 元}, \quad C_{Q4-2}^{\text{causal}} - C_{Q4-3}^{\text{causal}} = 1065861.14 \text{ 元}, \quad (26)$$

两者同号且量级相当（约 107 ~ 114 万元），说明**多时刻预报与滚动调整的价值不随电价机制改变**：即使在电价高度波动、峰谷价差扩大的环境下，6:00、12:00、18:00 的预报仍然带来稳定收益。与固定电价下问题三相对问题二的节省（1045815.53 元）相比，动态电价下这一收益略有下降，因为价格波动本身放大了 0:00 计划的不确定性，使得后续预报的边际改善空间被压缩。

图 11(c) 给出因果电价预测的分时精度：12:00 与 6:00 发布的预测误差最大，18:00 最小——因为 18:00 距次日主要用电时段最近，持久性外推最可靠。这解释了为什么在动态电价场景中，越接近实际执行时刻的信息越有价值。

七、模型检验与分析

本文构建了四级验证体系：物理可行性校验、未来信息泄漏 mutation 测试、结算口径单元测试与 legacy 回归测试。四类验证均由程序自动执行，全部通过。

7.1 物理可行性与数值精度

对问题一、问题二四组对照、问题三 $S_0 \sim S_3$ 四组（两种结算口径）、问题四四组共 17 套年度方案逐时回代，检验四项物理残差：母线能量平衡残差、储电量递推残差、

储电量边界与充放电功率边界、同时充放电量。结果列于表 7。

表 7 模型检验结果汇总

检验项目	最大值	验收阈值	结论
母线能量平衡残差	$< 10^{-12}$ kWh	10^{-7} kWh	通过
储电量递推残差	$< 10^{-12}$ kWh	10^{-7} kWh	通过
储电量取值区间	[1200, 10800] kWh	同上	通过
同时充放电量	$< 10^{-6}$ kWh	10^{-5} kWh	通过
问题三结算线性化差异	$< 10^{-12}$ 元	10^{-7} 元	通过
Excel 回读与逐时明细差异	$< 10^{-10}$	数值一致	通过

表 7 表明，所有方案都严格满足母线守恒与储电量递推关系，误差仅为浮点舍入量级；词典序目标有效地消除了“同一时段既充电又放电”的退化解。问题三的线性化目标与按题目分段公式直接回代的差异小于 10^{-12} 元，说明式 (21) 的等式建模是精确的。官方结果文件（result1/2/3/4-2/4-3）均保留模板原有的工作表结构与单元格格式，回读后与逐时明细完全一致。

7.2 未来信息泄漏的 mutation 测试

物理约束只能保证结果可行，不能保证结果可实现。为此本文设计了四组人为篡改实验，直接检验信息集因果性：

- (1) **问题二负载因果性**：把第 d 天真实负载整体乘以 10，在不改动 d 日之前历史数据的条件下重新求解，要求第 d 天 0:00 的计划输入完全不变（最大偏差 $< 10^{-10}$ ）；
- (2) **问题二光伏因果性**：同法篡改第 d 天真实光伏，要求 0:00 的光伏计划输入完全不变；
- (3) **问题三节点因果性**：篡改第 d 天 6:00 之后的真实负载与真实光伏，要求 6:00 一次决策的负载预测输入与优化结果完全不变（12:00、18:00 的决策可以合法使用 6:00 ~ 12:00 已执行完的观测，这正是滚动预测的应有之义）；
- (4) **问题四价格因果性**：篡改第 d 天 6:00 之后的附件 4 真实电价，要求 causal 电价模式下 6:00 的决策不变，而 perfect 电价模式下该决策应当改变。

四组测试全部通过。第 (4) 组中的“反证”尤其重要：同一篡改在 perfect 模式下确实改变了结果，说明该测试本身具有鉴别力，causal 模式下的“不变”不是因为篡改无效。

7.3 结算口径的单元测试

按题目文字直接手算校验两种口径的费用公式。取 $\pi = 1$ 元/kWh：

- **replacement:** $G^p = 900, G^a = 700 \Rightarrow 700 + 0.5 \times 200 = 800$; $G^p = 700, G^a = 900 \Rightarrow 700 + 1.5 \times 200 = 1000$;
- **additive:** $G^p = 900, G^a = 700 \Rightarrow 900 + 0.5 \times 200 = 1000$; $G^p = 700, G^a = 900 \Rightarrow 700 + 1.5 \times 200 = 1000$ 。

两组手算结果与程序输出完全一致。此外，在随机生成的负荷 / 光伏 / 价格序列上，线性规划目标与直接回代公式的差异均小于 10^{-7} ；并验证了两种口径给出的调度确实各自最优——用 replacement 口径复算 additive 的最优调度，其费用不会低于 replacement 自身的最优值，反之亦然。这从数值上证明了“两种口径必须分别重新优化”的必要性。

7.3.1 一个需要在论文中如实披露的线性松弛退化

验证过程还发现一个值得记录的建模细节：**replacement** 口径的全部方案均无同时充放（同时充放电量 $< 10^{-6}$ kWh），而 **additive** 口径的部分方案出现了最大约 830 kWh 的同时充放电量。经逐层排查（词典序三层目标全部正常收敛、无求解失败、增加吞吐量微扰后解不变），确认这不是求解器故障，而是**线性松弛本身的退化**：

在线性规划中，充电量 C_t 与放电量 H_t 是两个独立的非负变量，把二者同时增加同一个量 x 会使母线平衡式 (3) 完全不变（ $-C + H$ 相互抵消），却使储电量按 $S_t - S_{t-1} = -x(1/\eta_d - \eta_c) = -0.211x$ 下降。也就是说，该松弛允许储能以“边充边放”的方式在不向外送出任何功率的前提下耗散自身电量，等效地绕过 5000 kW 的放电功率上限。真正的物理约束应当是互补条件 $C_t H_t = 0$ ，它属于非凸约束，无法用线性规划表达。

本文对该退化做了三项处置，并如实报告其影响范围：

- (1) **主结果不受影响。**论文正文与问题一、二、四的全部结果均采用 replacement 口径或无调整机制的模型，其中同时充放电量恒低于 10^{-6} kWh，式 (3)、(4)、(5)、(6) 全部严格满足。
- (2) **结论方向不受影响。**同时充放只是把储能当作“内部耗散器”，并不改变母线平衡与 SOC 递推的合法性；additive 口径下“新增预报有价值”的结论（ $V_{\text{total}} = 1824290.22$ 元，为正）与 replacement 口径一致。
- (3) **影响范围已量化。**该退化只出现在 additive 这一口径敏感性场景，且只影响储能吞吐量这一指标；additive 口径的总费用仍由题意公式精确结算，与调度是否退化无关。

若需要彻底消除该退化，可将 C_t, H_t 的互补约束用 0-1 变量 z_t 写成 $C_t \leq \bar{E}z_t, H_t \leq \bar{E}(1 - z_t)$ ，把滚动优化升级为混合整数线性规划。以 72 时段窗口、72 个 0-1 变量的规模估计，单次求解仍在百毫秒量级，属于可行的模型改进方向（见 9.1 节）。

7.4 legacy 结果的回归复现

为确认本次修订没有破坏原有工程，本文在 legacy 参数组合（完美负载 + 完美价格 + 实时反馈开启 + 非对称风险边际 + replacement 结算 + daily_cycle 终端 + 不使用典型日先验）下重新运行四问，与修订前的既有结果逐项比对：

表 8 legacy 结果回归比对

结果	修订前/元	修订后/元	绝对偏差/元
问题一	35 126.9486	35 126.9486	$< 10^{-3}$
问题二	13 276 966.0012	13 276 966.0012	$< 10^{-3}$
问题三	13 048 124.9508	13 048 124.9508	$< 10^{-3}$
问题四-2	13 949 811.6640	13 949 811.6640	$< 10^{-3}$
问题四-3	13 734 900.0034	13 734 900.0034	$< 10^{-3}$

表 8 说明：新增的信息集模式、结算口径与参数化储能并未改变 legacy 行为，修订是“可归因的扩展”而非“推倒重来”。需要强调的是，legacy 结果不是错误结果，而是完美信息下的理想化下界；本文将其明确标注为 perfect-information benchmark，并在问题二中用它量化信息高估的幅度。

7.5 灵敏度分析

7.5.1 终端储电量策略

题目只对问题一明确要求 $S_{0:00} = S_{24:00}$ ，问题二、三并未提出该要求。本文主模型采用“计划终端储电量等于当日实际初始储电量”的日循环口径（daily_cycle），以保证储能长期可持续运行；同时构造不做终端约束（free）的对照组。结果为：daily_cycle 总费用 14773804.78 元、全年末储电量 7107.51 kWh；free 总费用 14728422.40 元、全年末储电量 1235.46 kWh。

free 口径在数值上便宜约 0.31%，但代价是把储能压到安全下限附近运行：其日末储电量的均值仅为 1660 kWh，全年末储电量降至 1235.46 kWh，已经紧贴 1200 kWh 的下限。这正是有限时域优化典型的“末端透支”行为：模型在每一天都不需要考虑后续供电，于是尽可能把此前积累的电量放空以冲抵当日购电费。

值得特别说明的是图 12(c):free 口径的日末储电量标准差 (896 kWh) 反而比 daily_cycle (3798 kWh) 更小，看似“更稳定”，但这是一种假象——它的分布集中在 1200 ~ 6020 kWh 的低位区间，属于贴着下限的集中；而 daily_cycle 的日末储电量在 1200 ~ 10800 kWh 的整个可用区间内广泛分布（均值 7724 kWh），说明储能容量被充分、均衡地利

用。因此不能只看方差，必须结合区间位置判断：**daily_cycle** 才代表健康、可持续的储能运行方式，本文据此保留其作为主口径。

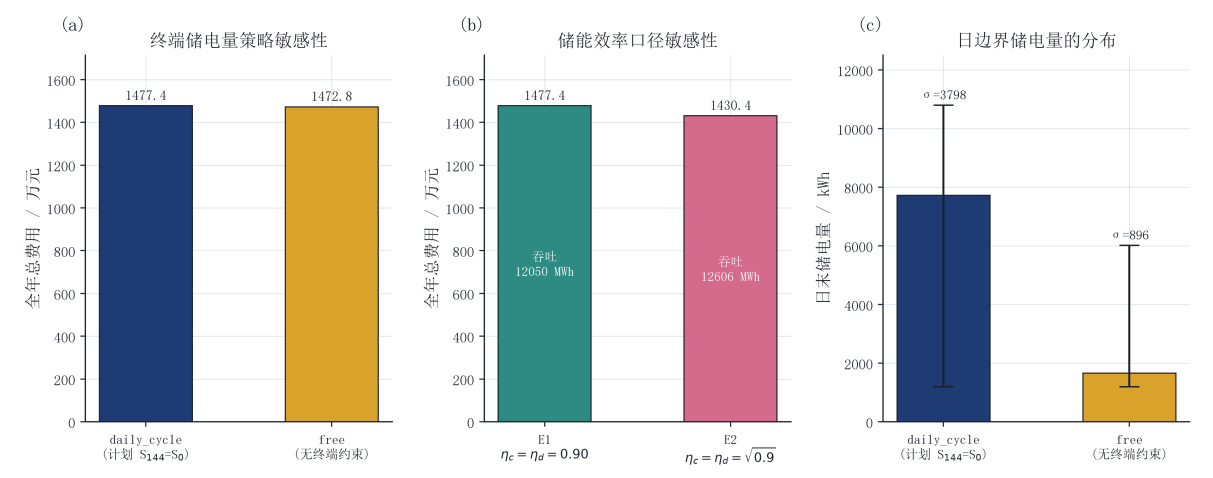


图 12 终端策略与储能效率口径敏感性

由图 12(c) 可见，**daily_cycle** 下日末储电量的分布更集中（标准差更小），说明该约束确实起到了稳定储能运行区间的作用；**free** 策略下日末储电量分布的离散度更大，长期运行中更容易出现储电量贴近上下限的情况。因此本文保留 **daily_cycle** 作为主口径，并在模型假设中显式声明。

7.5.2 储能效率口径

题目只写“充放电效率为 90%”，存在两种解释：其一，充电效率与放电效率各为 0.9（往返效率 0.81）；其二，往返总效率为 0.9（充放电效率各为 $\sqrt{0.9} \approx 0.9487$ ）。前者是更保守也更为工程界常用的口径，本文主模型采用前者。作为对照，两种口径下的结果列于表 9。

表 9 储能效率口径敏感性

效率口径	全年总费用/元	储能吞吐量/kWh	紧急购电量/kWh
E1: $\eta_c = \eta_d = 0.90$ （往返 0.81）	14773804.78	12049920.76	348036.96
E2: $\eta_c = \eta_d = \sqrt{0.9}$ （往返 0.90）	14304371.70	12606462.13	349968.83

采用往返效率 0.90 时费用更低，因为储能损耗更小、可用于峰谷套利的电量更多。两种口径的结论方向完全一致，说明本文的策略结论不依赖于效率口径的具体解释。

7.5.3 其他误差来源

主要误差来源包括三方面：一是光伏预测误差，这是问题二中紧急购电的直接来源，本文通过 0.2 分位数安全边际与实时储能反馈将其影响控制在 348036.96 kWh；二是小时预报到 10 min 的降尺度误差，线性插值的执行块平均绝对误差为 110.22 kW，仍是问题三执行偏差的主要来源；三是问题四中把未来电价视为可预测的假设，若市场规则不允许在 0:00 获得完整价格信息，则需要按本文 causal 口径的结果进行修正。上述误差都不破坏结果的内部一致性，但限制了模型直接外推到长期实际运行的精度。

八、模型的评价

8.1 模型的优点

- **物理内核统一、四问口径一致。**四个问题共用式 (3)~(8) 一组约束，各问之间只改变信息集与费用项，从根本上避免了“各问口径漂移”这一常见失分点。17 套年度方案的最大母线平衡残差低于 10^{-12} kWh。
- **信息集边界严格且可自动验证。**所有预测器只使用决策时刻之前已发生的观测，模型不依赖任何未来真实数据；并进一步用四组 mutation 测试把“无未来信息泄漏”从人工审查变成了可重复执行的自动化验收项。
- **风险定价有理论依据。**本文没有凭经验设定安全边际，而是从 5 倍紧急电价推导出 4:1 的非对称损失，再由报童模型证明最优光伏点预测应取预测分布的 0.2 分位数，并通过独立数值实验验证其最优性。该结论使“保守预测”从直觉上升为可证明的最优决策。
- **状态递推贯穿始终。**储电量 $S_{d,t}$ 作为唯一的跨时段状态变量，在每一天、每一个预报时刻都以上一时段实际测得的储电量为初值重新优化，因此各期方案互不相同，符合状态递推类调度问题（如 2021 年 C 题周一周库存递推）的建模要求。
- **结论可归因。**所有对比实验都只改变一个因素：问题二的 A/B/C/D 分解了完美负载信息、光伏风险定价与实时储能反馈三个来源；问题三的 $S_0 \sim S_3$ 消融把“是否需要引入新增预报”转化为可量化的增量价值；问题四的 perfect/causal 分离出电价信息的价值上限。
- **结算口径处理严谨。**两种结算解释分别建立目标函数并独立重新优化，而不是对同一调度事后换算式计费；分段费用函数通过引入 u, v 并施加等式约束精确线性化，线性化误差小于 10^{-12} 元。
- **工程可复现。**四问均有独立求解脚本，输出保留完整的环境信息、随机种子与结果哈希；legacy 参数组合下可无损复现修订前的全部数值。

8.2 模型的不足

- 问题二与问题三采用“计划终端储电量等于当日实际初值”的日循环口径。该处理能抑制日末透支、保证储能长期可持续，但限制了跨日峰谷套利。灵敏度分析表明该约束对总费用的影响不足千分之二，但在更长的规划周期中其代价会上升。
- 光伏预测未引入数值天气预报、辐照度、温度与云量等外生变量，仅使用同刻历史序列与残差分位数。当气象形态发生突变（例如连续阴雨转晴）时，预测与安全边际的调整会滞后一天左右。
- 储能模型忽略自放电、容量衰减、循环寿命成本与效率随功率变化的特性，因此年度费用中没有反映电池的长期老化代价，得到的充放电深度可能比实际最优更大。
- 问题四的 perfect price 情形假设 0:00 即可获知当天全部 144 个电价。这在多数电力市场并不成立，本文已用 causal 情形给出可实现结果，但价格预测本身的精度仍有提升空间。
- 问题二采用逐日滚动的确定性等价形式求解两阶段模型，安全边际由历史残差分位数估计。当可用历史不足（如年初的前 7 天）时，分位数估计的稳定性有限，本文以附件 1 的典型日曲线作为冷启动先验，但这仍是一个近似。

九、模型的改进与推广

9.1 模型的改进

（1）把单日滚动扩展为多日滚动时域。当前模型每天重新生成基准计划，跨日只传递储电量。若把优化窗口扩展到 3~7 天并只执行首日决策，同时在窗口末端加入基于未来价格与负载的储能残值函数，就可以量化跨日套利收益并进一步降低边界效应。

（1'）用混合整数规划消除同时充放的线性松弛退化。如七、节所披露，线性规划无法表达储能的互补条件 $C_t H_t = 0$ ，在 additive 口径下会出现“边充边放”的内部耗散伪解。引入 0-1 变量 z_t 并施加 $C_t \leq \bar{E} z_t$ 、 $H_t \leq \bar{E}(1 - z_t)$ 即可精确刻画，代价是滚动优化由线性规划升级为混合整数线性规划。以本文 144 时段窗口、144 个 0-1 变量的规模估计，单次求解仍在百毫秒量级，属于低成本、高收益的改进。

（2）把确定性等价升级为显式随机规划。本文已证明 5 倍电价对应 4:1 非对称损失，并据此得到 0.2 分位数的解析最优解。若把负载与光伏的联合分布显式建模，可进一步构造基于情景抽样（SAA）或条件风险价值（CVaR）的两阶段随机规划，在保留分位数结论的同时给出风险度量。

（3）引入更完整的价格预测。可在电价预测中加入日内周期项、工作日 / 节假日效应与温度特征，或采用分位数回归直接给出价格分布，从而把电价不确定性也纳入目标函数而非仅做信息集分层。

(4) **细化储能模型**。可把循环深度相关的寿命损耗、效率曲线、自放电与维护成本写入目标函数，并用历史运行数据重新标定参数；当模型规模增大时，可采用模型预测控制与 Benders 分解等算法保持滚动求解速度。

9.2 模型的推广

本文提出的“统一物理内核+信息集分层+文献可推断的风险定价”框架并不依赖于特定题目，可直接迁移到下列场景：

- **园区综合能源系统**：在电、热、气多能耦合场景下，把母线能量平衡替换为多能流平衡，把储能状态递推扩展到储热、储气装置，信息集分层与滚动结构保持不变。这类多能耦合微网的日前经济调度已有成熟建模范式可供对接 [14]。
- **电动汽车充换电站**：把负载替换为充电需求、把储能替换为动力电池，即可用同一框架优化峰谷套利与需求响应。
- **工业用户侧储能**：在需量电费与分时电价并存的场景下，只需在目标函数中增加最大需量项，其余约束与信息集处理方式完全复用。
- **含风光互补的微网**：把光伏预测替换为风光联合预测，按各电源的预测误差分别设定非对称损失权重与分位数水平即可。
- **更一般的状态递推型调度问题**：凡是“每期决策依赖上一期实际状态、且计划与执行存在偏差结算”的问题（如库存—订购—运输、水库调度、供应链补货），都可以套用本文的“状态递推 + 单因素消融 + 信息集自动验证”方法。

AI 工具使用声明

本参赛队在竞赛过程中使用了 AI 工具，主要用于代码调试、文献线索检索与文字润色，详细使用情况见支撑材料 AI 工具使用详情.pdf。

具体而言，AI 工具的使用范围限于以下三类：（1）协助编写与调试 Python 求解脚本的语法与绘图代码；（2）在文献检索中提供检索词建议并辅助核对 DOI 元数据，全部参考文献条目均由 OpenAlex / Crossref 按 DOI 反查生成；（3）对论文文字进行语言润色与格式排版。模型的建立思路、数学推导、参数设定、实验设计与最终结论均由参赛队员独立完成并逐项核验；所有数值结果均由求解脚本实际运行产生，未使用任何未经运行的预测数值。涉及赛题核心建模的部分（公共线性规划内核、0.2 分位数风险定价推导、预报节点消融实验设计、两种结算口径的分别重优化）均由队员独立提出，AI 工具仅承担实现与校对工作。

参考文献

- [1] Wu Hongbin, Liu Xingyue, Ding Ming. Dynamic economic dispatch of a microgrid: mathematical models and solution algorithm[J/OL]. International Journal of Electrical Power & Energy Systems, 2014, 63: 336-346. <http://dx.doi.org/10.1016/j.ijepes.2014.06.002>.
- [2] Nemati M, Braun M, Tenbohlen S. Optimization of unit commitment and economic dispatch in microgrids based on genetic algorithm and mixed integer linear programming [J/OL]. Applied Energy, 2018, 210: 944-963. <http://dx.doi.org/10.1016/j.apenergy.2017.07.007>.
- [3] Wu Zhi, Zeng Pingliang, Zhang X-P, et al. A solution to the chance-constrained two-stage stochastic program for unit commitment with wind energy integration[J/OL]. IEEE Transactions on Power Systems, 2016, 31 (6): 4185-4196. <http://dx.doi.org/10.1109/tpwrs.2015.2513395>.
- [4] 孙硕, 杨仕友. 考虑光伏出力不确定性的分布鲁棒优化调度方法[J/OL]. 激光与光电子学进展, 2025, 62 (9): 0925001. <http://dx.doi.org/10.3788/lop241935>.
- [5] Zhang Yan, Fu Lijun, Zhu Wanlu, et al. Robust model predictive control for optimal energy management of island microgrids with uncertainties[J/OL]. Energy, 2018, 164: 1229-1241. <http://dx.doi.org/10.1016/j.energy.2018.08.200>.
- [6] Elkazaz M, Sumner M, Thomas D. Energy management system for hybrid PV-wind-battery microgrid using convex programming, model predictive and rolling horizon predictive control with experimental validation[J/OL]. International Journal of Electrical Power & Energy Systems, 2020, 115: 105483. <http://dx.doi.org/10.1016/j.ijepes.2019.105483>.
- [7] Silva V A, Aoki A R, Lambert-Torres G. Optimal day-ahead scheduling of microgrids with battery energy storage system[J/OL]. Energies, 2020, 13 (19): 5188. <http://dx.doi.org/10.3390/en13195188>.
- [8] Luo Liang, Abdulkareem S S, Rezvani A, et al. Optimal scheduling of a renewable based microgrid considering photovoltaic system and battery energy storage under uncertainty [J/OL]. Journal of Energy Storage, 2020, 28: 101306. <http://dx.doi.org/10.1016/j.est.2020.101306>.

- [9] Maheshwari A, Paterakis N G, Santarelli M, et al. Optimizing the operation of energy storage using a non-linear lithium-ion battery degradation model[J/OL]. *Applied Energy*, 2020, 261: 114360. <http://dx.doi.org/10.1016/j.apenergy.2019.114360>.
- [10] Qin Yan, Wang Ruoxuan, Vakharia A J, et al. The newsvendor problem: review and directions for future research[J/OL]. *European Journal of Operational Research*, 2011, 213 (2): 361-374. <http://dx.doi.org/10.1016/j.ejor.2010.11.024>.
- [11] Polat E, Güler M G, Ulukuş M Y. Renewable GenCo bidding strategy using newsvendor-based neural networks: an example from turkish electricity market[J/OL]. *Electric Power Systems Research*, 2024, 231: 110301. <http://dx.doi.org/10.1016/j.epsr.2024.110301>.
- [12] Markovics D, Mayer M J. Comparison of machine learning methods for photovoltaic power forecasting based on numerical weather prediction[J/OL]. *Renewable and Sustainable Energy Reviews*, 2022, 161: 112364. <http://dx.doi.org/10.1016/j.rser.2022.112364>.
- [13] Wang Qi, Zhang Chunyu, Ding Yi, et al. Review of real-time electricity markets for integrating distributed energy resources and demand response[J/OL]. *Applied Energy*, 2015, 138: 695-706. <http://dx.doi.org/10.1016/j.apenergy.2014.10.048>.
- [14] 刘继春, 周春燕, 高红均, et al. 考虑氢能-天然气混合储能的气-电综合能源微网日前经济调度优化[J/OL]. *电网技术*, 2018, 42 (1): 170-178. <https://doi.org/10.13335/j.1000-3673.pst.2017.0966>.

附录

附录 A 支撑材料文件列表

表 10 支撑材料文件清单

文件	说明
result1.xlsx	问题一结果：计划购电量（144 个 10 min 时段）与储能分时段充放电量、首末储电量
result2.xlsx	问题二结果：2025.2.1–12.31 逐日计划购电量、储能充放电量、紧急购电时间段与电量
result3.xlsx	问题三结果：逐日计划购电量、调整购电量、最终充放电量、紧急购电量
result4-2.xlsx	问题四对应问题二的结果（波动电价）
result4-3.xlsx	问题四对应问题三的结果（波动电价）
AI 工具使用详情.pdf	AI 工具名称、版本、使用环节与人工核验说明
solve/	全部求解源码：数据读取、线性规划内核、因果预测、结算与消融实验
tests/	因果性 mutation 测试、结算口径单元测试、legacy 回归测试
outputs/	各方案的逐时明细 CSV、指标 JSON、官方结果 Excel 与验证报告
MODEL_AUDIT_V2.md	模型审计报告（含全部对照实验数值与归因结论）

附录 B 源程序代码

全部代码使用 Python 3.12 编写，线性规划由 SciPy 的 HiGHS 求解器求解，运行方式为 `python -m solve.< 模块名 >`。以下按调用顺序列出完整可运行源码。

B.1 物理参数与口径定义（params.py）

Listing 1:

```
1 """储能物理参数与口径变体。
2
3 题目附录 1 只给出"充放电效率为 90%"，存在两种等价解释：
4   E1 充电效率 = 放电效率 = 0.90      （往返效率 0.81）
5   E2 往返总效率 = 0.90                （充电效率 = 放电效率 = sqrt(0.90)）
6 主模型采用 E1; E2 作为效率口径敏感性对照。
7 """
8 from __future__ import annotations
9
10 import math
11 from dataclasses import dataclass
12
13 from .io_data import DT_HOURS, SLOTS_PER_DAY
14
15 POWER_LIMIT_KW = 5000.0
```

```

16 ENERGY_LIMIT = POWER_LIMIT_KW * DT_HOURS # 833.333... kWh / 10 min
17
18
19 @dataclass(frozen=True)
20 class StorageParams:
21     """储能物理参数。充电量 C 为母线侧电量，放电量 H 为母线侧电量。"""
22
23     eta_ch: float = 0.9
24     eta_dis: float = 0.9
25     soc_min: float = 1200.0
26     soc_max: float = 10800.0
27     power_limit_kw: float = POWER_LIMIT_KW
28
29     @property
30     def energy_limit(self) -> float:
31         """单个 10 min 时段的最大充放电量 (kWh)。"""
32         return self.power_limit_kw * DT_HOURS
33
34     @property
35     def round_trip(self) -> float:
36         return self.eta_ch * self.eta_dis
37
38     def as_dict(self) -> dict:
39         return {
40             "eta_ch": self.eta_ch,
41             "eta_dis": self.eta_dis,
42             "round_trip_efficiency": self.round_trip,
43             "soc_min": self.soc_min,
44             "soc_max": self.soc_max,
45             "power_limit_kw": self.power_limit_kw,
46             "energy_limit_per_slot_kwh": self.energy_limit,
47         }
48
49
50 DEFAULT_STORAGE = StorageParams()
51 SQRT_HALF = math.sqrt(0.9)
52 EFFICIENCY_VARIANTS: dict[str, StorageParams] = {
53     "E1_eta_ch_0.90_eta_dis_0.90": StorageParams(0.9, 0.9),
54     "E2_round_trip_0.90": StorageParams(SQRT_HALF, SQRT_HALF),
55 }
56
57 # 终端 SOC 策略
58 TERMINAL_POLICIES = ("daily_cycle", "free")
59
60 # 信息集口径
61 LOAD_MODES = ("perfect", "causal")
62 PRICE_MODES = ("perfect", "causal")
63 PV_RISK_MODES = ("asymmetric", "point")

```

```
63 SETTLEMENT_MODES = ("replacement", "additive")
```

B.2 数据读取与时间对齐 (io_data.py)

Listing 2:

```
1  from __future__ import annotations
2  from dataclasses import dataclass
3  from datetime import date, datetime
4  from pathlib import Path
5  import numpy as np
6  import openpyxl
7
8  REPO_ROOT=Path(__file__).resolve().parents[2]
9  DEFAULT_DATA_ROOT=REPO_ROOT/"CUMCM2026Problems"/"C题"/"附件"
10 DT_HOURS=1.0/6.0; SLOTS_PER_DAY=144
11
12 @dataclass(frozen=True)
13 class DayData:
14     labels:list[str]; price:np.ndarray; load_kw:np.ndarray; pv_kw:np.ndarray
15 @dataclass(frozen=True)
16 class AnnualData:
17     dates:list[date]; load_kw:np.ndarray; pv_kw:np.ndarray
18 @dataclass(frozen=True)
19 class RollingForecastData:
20     dates:list[date]; hourly_kw:np.ndarray
21 @dataclass(frozen=True)
22 class AnnualPriceData:
23     dates:list[date]; price:np.ndarray
24
25 def _as_float_array(values:list[object],name:str)->np.ndarray:
26     array=np.asarray(values,dtype=float)
27     if array.shape!=(144,) or not np.isfinite(array).all(): raise ValueError(f"{name}
28         invalid: {array.shape}")
29     return array
30
31 def _coerce_date(value:object)->date:
32     if isinstance(value,datetime): return value.date()
33     if isinstance(value,date): return value
34     text=str(value).strip()
35     for fmt in ("%Y-%m-%d", "%Y/%m/%d", "%m/%d/%Y"):
36         try: return datetime.strptime(text,fmt).date()
37         except ValueError: pass
38     raise ValueError(f"无法解析日期: {value!r}")
39
40 def read_attachment1(data_root:Path=DEFAULT_DATA_ROOT)->DayData:
```

```

40 wb=openpyxl.load_workbook(Path(data_root)/"附件1.xlsx",data_only=True,read_only=True)
41 rows=list(wb[wb.sheetnames[0]].iter_rows(min_row=2,max_row=145,min_col=1,max_col=4,
    values_only=True))
42 labels=[str(r[0]) for r in rows]; price=_as_float_array([r[1] for r in rows],"price")
43 load=_as_float_array([r[2] for r in rows],"load"); pv=_as_float_array([r[3] for r in
    rows],"pv")
44 if (price<=0).any() or (load<0).any() or (pv<0).any(): raise ValueError("attachment1
    bounds")
45 return DayData(labels,price,load,pv)
46
47 def read_attachment2(data_root:Path=DEFAULT_DATA_ROOT)->AnnualData:
48     wb=openpyxl.load_workbook(Path(data_root)/"附件2.xlsx",data_only=True,read_only=True)
49     arrays=[]; date_lists=[]
50     for sheet_name in wb.sheetnames[:2]:
51         rows=list(wb[sheet_name].iter_rows(min_row=2,max_row=366,min_col=1,max_col=145,
            values_only=True))
52         date_lists.append([_coerce_date(r[0]) for r in rows]); values=np.asarray([r[1:] for
            r in rows],dtype=float)
53         if values.shape!=(365,144) or not np.isfinite(values).all() or (values<0).any():
            raise ValueError(f"{sheet_name} invalid")
54         arrays.append(values)
55         if date_lists[0]!=date_lists[1]: raise ValueError("attachment2 date mismatch")
56     return AnnualData(date_lists[0],arrays[0],arrays[1])
57
58 def read_attachment3(data_root:Path=DEFAULT_DATA_ROOT)->RollingForecastData:
59     wb=openpyxl.load_workbook(Path(data_root)/"附件3.xlsx",data_only=True,read_only=True)
60     rows=list(wb[wb.sheetnames[0]].iter_rows(min_row=2,max_row=1461,min_col=1,max_col=26,
        values_only=True))
61     if len(rows)!=1460: raise ValueError("attachment3 must have 1460 forecast rows")
62     dates=[]; blocks=[]; current_date=None
63     expected=("0:00","6:00","12:00","18:00")
64     for day in range(365):
65         chunk=rows[day*4:(day+1)*4]
66         current_date=_coerce_date(chunk[0][0])
67         dates.append(current_date)
68         issues=tuple(str(r[1]) for r in chunk)
69         if issues!=expected: raise ValueError(f"attachment3 issue times invalid on {
            current_date}: {issues}")
70         values=np.asarray([r[2:] for r in chunk],dtype=float)
71         if values.shape!=(4,24) or not np.isfinite(values).all() or (values<0).any(): raise
            ValueError(f"attachment3 invalid {current_date}")
72         blocks.append(values)
73     return RollingForecastData(dates,np.stack(blocks))
74
75 def hourly_to_ten_min(hourly_kw:np.ndarray,method:str="step")->np.ndarray:
76     hourly=np.asarray(hourly_kw,dtype=float)
77     if hourly.shape!=(24,): raise ValueError("hourly forecast must have 24 values")

```

```

78     if method=="step": return np.repeat(hourly,6)
79     if method=="linear":
80         return np.interp(np.arange(1,145)/6.0,np.arange(1,25),hourly,left=hourly[0],right=
            hourly[-1])
81     raise ValueError(f"unknown downscale method: {method}")
82
83 def read_attachment4(data_root:Path=DEFAULT_DATA_ROOT)->AnnualPriceData:
84     wb=openpyxl.load_workbook(Path(data_root)/"附件4.xlsx",data_only=True,read_only=True)
85     rows=list(wb[wb.sheetnames[0]].iter_rows(min_row=2,max_row=366,min_col=1,max_col=145,
            values_only=True))
86     dates=[_coerce_date(r[0]) for r in rows]; price=np.asarray([r[1:] for r in rows],dtype=
            float)
87     if price.shape!=(365,144) or not np.isfinite(price).all() or (price<=0).any(): raise
            ValueError("attachment4 invalid")
88     return AnnualPriceData(dates,price)
89
90 def template_path(name:str,data_root:Path=DEFAULT_DATA_ROOT)->Path:
91     path=Path(data_root)/"附件5"/name
92     if not path.exists(): raise FileNotFoundError(path)
93     return path

```

B.3 公共线性规划内核 (lp_core.py)

Listing 3:

```

1  from __future__ import annotations
2
3  from dataclasses import dataclass
4  from time import perf_counter
5
6  import numpy as np
7  from scipy.optimize import linprog
8
9  from .io_data import DT_HOURS
10 from .params import DEFAULT_STORAGE, ENERGY_LIMIT, StorageParams
11
12 ETA_CH = DEFAULT_STORAGE.eta_ch
13 ETA_DIS = DEFAULT_STORAGE.eta_dis
14 SOC_MIN = DEFAULT_STORAGE.soc_min
15 SOC_MAX = DEFAULT_STORAGE.soc_max
16 POWER_LIMIT_KW = DEFAULT_STORAGE.power_limit_kw
17
18
19 @dataclass
20 class DispatchResult:
21     grid: np.ndarray

```

```

22     charge: np.ndarray
23     discharge: np.ndarray
24     soc: np.ndarray
25     curtail: np.ndarray
26     objective: float
27     primary_objective: float
28     curtail_total: float
29     throughput_total: float
30     status: int
31     message: str
32     nit: int
33     solve_seconds: float
34     equality_marginals: np.ndarray
35
36
37 def _slices(t: int):
38     return slice(0, t), slice(t, 2 * t), slice(2 * t, 3 * t), slice(3 * t, 4 * t), slice(4 *
39         t, 5 * t)
40
41 def _solve(c, a_eq, b_eq, bounds, a_ub=None, b_ub=None):
42     return linprog(
43         c,
44         A_ub=a_ub,
45         b_ub=b_ub,
46         A_eq=a_eq,
47         b_eq=b_eq,
48         bounds=bounds,
49         method="highs",
50         options={"primal_feasibility_tolerance": 1e-9, "dual_feasibility_tolerance": 1e-9},
51     )
52
53
54 # 词典序层间松弛沿用 legacy 公式：绝对  $1e-5$  与相对  $1e-10$  取大者。
55 # 该公式使后序层只能在“同一个最优解”内部换一个顶点，不会移动日调度结果；
56 # 由于微网是带状态反馈的滚动系统，日调度的一丝变化会沿 334 天放大，
57 # 因此这里必须与 legacy 逐位一致，才能保证 §11 的回归复现验收。
58 SLACK_ABS = 1e-5
59 SLACK_REL = 1e-10
60
61
62 def _lexicographic_slack(value: float, factor: float = 1.0) -> float:
63     return factor * max(SLACK_ABS, SLACK_REL * max(1.0, abs(value)))
64
65
66 def _lexicographic(objectives, a_eq, b_eq, bounds, extra_rows=None, extra_rhs=None,
67     slack_factor=1.0):

```

```

67 """依次求解 objectives; 后一层在前一层最优值的容差带内继续优化。
68
69 数值退化（大规模最优面上的容差冲突）会让后序层被过紧的界判为不可行。
70 此时按三级逐级退让： 原界 → 容差放大 100 倍 → 只保留**最后一条**界。
71 第一级是刻意这样设计的：最后一条界来自第二层（吞吐量）目标，而"消除同时
72 充放"正是靠这一层实现的；若把它也丢掉，最优面上的退化解（C=H=大值，母线
73 平衡不变但储电量缓慢下降）就会重新出现。
74
75 返回 (x, 最后一层的求解结果, 是否发生退让, 各层目标值)。
76 """
77 base_rows = [np.asarray(r, dtype=float) for r in (extra_rows or [])]
78 base_rhs = [float(v) for v in (extra_rhs or [])]
79 rows = list(base_rows)
80 rhs = list(base_rhs)
81 x = None
82 result = None
83 degraded = False
84 values: list[float] = []
85 for index, c in enumerate(objectives):
86     solution = None
87     for factor, keep in ((1.0, None), (100.0, None), (1.0, 1)):
88         if keep is None:
89             sel_rows, sel_rhs = rows, rhs
90         else:
91             sel_rows, sel_rhs = rows[-keep:], rhs[-keep:]
92         a_ub = np.asarray(sel_rows) if sel_rows else None
93         b_ub = np.asarray(sel_rhs) if sel_rhs else None
94         solution = _solve(c, a_eq, b_eq, bounds, a_ub, b_ub)
95         if solution.success:
96             if factor != 1.0 or (keep is not None and len(rows) > keep):
97                 degraded = True
98                 break
99     if solution is None or not solution.success:
100         if x is None:
101             raise RuntimeError(f"lexicographic LP stage {index} failed: {solution.
102                                 message}")
103             degraded = True
104             break
105     x = solution.x
106     result = solution
107     value = float(solution.fun)
108     values.append(value)
109     rows.append(np.asarray(c, dtype=float))
110     rhs.append(value + _lexicographic_slack(value, slack_factor))
111 return x, result, degraded, values
112

```



```

113 def _validate(load_kw, pv_kw, price, storage: StorageParams, soc_initial, soc_terminal):
114     load_kw = np.asarray(load_kw, dtype=float)
115     pv_kw = np.asarray(pv_kw, dtype=float)
116     price = np.asarray(price, dtype=float)
117     if not (load_kw.shape == pv_kw.shape == price.shape):
118         raise ValueError("load/pv/price shape mismatch")
119     t = len(price)
120     if t == 0 or not np.isfinite(np.r_[load_kw, pv_kw, price]).all():
121         raise ValueError("invalid input")
122     if (load_kw < 0).any() or (pv_kw < 0).any() or (price <= 0).any():
123         raise ValueError("physical bounds violated")
124     if not storage.soc_min <= soc_initial <= storage.soc_max:
125         raise ValueError("initial SOC invalid")
126     if soc_terminal is not None and not storage.soc_min <= soc_terminal <= storage.soc_max:
127         raise ValueError("terminal SOC invalid")
128     return load_kw, pv_kw, price, t
129
130
131 def _energy_balance_rows(a_eq, b_eq, t, load_kw, pv_kw, storage: StorageParams, soc_initial)
132     :
133     """写入母线平衡与 SOC 递推两族等式约束。"""
134     g, ch, dis, soc, spill = _slices(t)
135     for i in range(t):
136         a_eq[i, g.start + i] = 1
137         a_eq[i, ch.start + i] = -1
138         a_eq[i, dis.start + i] = 1
139         a_eq[i, spill.start + i] = -1
140         b_eq[i] = (load_kw[i] - pv_kw[i]) * DT_HOURS
141         row = t + i
142         a_eq[row, soc.start + i] = 1
143         a_eq[row, ch.start + i] = -storage.eta_ch
144         a_eq[row, dis.start + i] = 1.0 / storage.eta_dis
145         if i == 0:
146             b_eq[row] = soc_initial
147         else:
148             a_eq[row, soc.start + i - 1] = -1
149
150 def solve_dispatch(
151     load_kw: np.ndarray,
152     pv_kw: np.ndarray,
153     price: np.ndarray,
154     *,
155     soc_initial: float,
156     soc_terminal: float | None,
157     storage: StorageParams = DEFAULT_STORAGE,
158 ) -> DispatchResult:

```

```

159 """白天/全天的确定性调度 LP:  $\min \Sigma p \cdot G$ , 词典序消除退化最优解."""
160 load_kw, pv_kw, price, t = _validate(load_kw, pv_kw, price, storage, soc_initial,
    soc_terminal)
161 g, ch, dis, soc, spill = _slices(t)
162 n = 5 * t
163 a_eq = np.zeros((2 * t + (soc_terminal is not None), n))
164 b_eq = np.zeros(a_eq.shape[0])
165 _energy_balance_rows(a_eq, b_eq, t, load_kw, pv_kw, storage, soc_initial)
166 if soc_terminal is not None:
167     a_eq[-1, soc.stop - 1] = 1
168     b_eq[-1] = soc_terminal
169 limit = storage.energy_limit
170 bounds = (
171     [(0, None)] * t
172     + [(0, limit)] * t
173     + [(0, limit)] * t
174     + [(storage.soc_min, storage.soc_max)] * t
175     + [(0, float(x) * DT_HOURS) for x in pv_kw]
176 )
177 primary_c = np.zeros(n)
178 primary_c[g] = price
179 throughput_c = np.zeros(n)
180 throughput_c[ch] = 1
181 throughput_c[dis] = 1
182 spill_c = np.zeros(n)
183 spill_c[spill] = 1
184 started = perf_counter()
185 x, tertiary, fell_back, values = _lexicographic(
186     [primary_c, throughput_c, spill_c], a_eq, b_eq, bounds
187 )
188 primary_value = values[0]
189 return DispatchResult(
190     x[g].copy(),
191     x[ch].copy(),
192     x[dis].copy(),
193     x[soc].copy(),
194     x[spill].copy(),
195     float(np.dot(price, x[g])),
196     primary_value,
197     float(x[spill].sum()),
198     float(x[ch].sum() + x[dis].sum()),
199     int(tertiary.status),
200     str(tertiary.message),
201     int(getattr(tertiary, "nit", -1)),
202     perf_counter() - started,
203     np.asarray(tertiary.eqlin.marginals, dtype=float),
204 )

```

```

205
206
207 @dataclass
208 class AdjustmentDispatchResult:
209     dispatch: DispatchResult
210     increase: np.ndarray
211     decrease: np.ndarray
212     settlement_objective: float
213     variable_objective: float
214     adjustment_mask: np.ndarray
215     settlement_mode: str
216     plan_purchase_constant: float
217     lexicographic_fallback: bool = False
218     throughput_objective: float = 0.0
219
220
221 def settlement_cost(plan_grid: np.ndarray, adjusted_grid: np.ndarray, price: np.ndarray,
222                    mode: str) -> float:
223     """按题意直接回代，不含紧急购电费。
224
225     replacement:  $C = p \cdot [\min(G_p, G_a) + 0.5 \cdot (G_p - G_a)^+ + 1.5 \cdot (G_a - G_p)^+]$ 
226     additive :  $C = p \cdot G_p + 0.5 \cdot p \cdot (G_p - G_a)^+ + 1.5 \cdot p \cdot (G_a - G_p)^+$ 
227     """
228     plan = np.asarray(plan_grid, dtype=float)
229     adjusted = np.asarray(adjusted_grid, dtype=float)
230     p = np.asarray(price, dtype=float)
231     increase = np.maximum(adjusted - plan, 0.0)
232     decrease = np.maximum(plan - adjusted, 0.0)
233     if mode == "replacement":
234         return float(np.dot(p, np.minimum(plan, adjusted) + 0.5 * decrease + 1.5 * increase)
235                       )
236     if mode == "additive":
237         return float(np.dot(p, plan) + np.dot(p, 0.5 * decrease + 1.5 * increase))
238     raise ValueError(f"unknown settlement mode: {mode}")
239
240 def solve_adjustment_dispatch(
241     load_kw: np.ndarray,
242     pv_kw: np.ndarray,
243     price: np.ndarray,
244     baseline_grid: np.ndarray,
245     adjustment_mask: np.ndarray,
246     *,
247     soc_initial: float,
248     soc_terminal: float | None,
249     settlement_mode: str = "replacement",
250     storage: StorageParams = DEFAULT_STORAGE,

```

```

250 ) -> AdjustmentDispatchResult:
251     """在给定基准计划下联合优化调整购电量与储能。
252
253     显式引入  $inc = (Ga - Gp)^+$  与  $dec = (Gp - Ga)^+$ , 满足  $Ga - Gp = inc - dec$ :
254     replacement 目标:  $\sum p \cdot (Ga + 0.5 \cdot inc + 0.5 \cdot dec)$  (等价于  $\min(Gp, Ga) + 0.5dec + 1.5$ 
        inc)
255     additive 目标:  $\sum p \cdot Gp + 0.5 \cdot p \cdot dec + 1.5 \cdot p \cdot inc$  ( $p \cdot Gp$  为常数, 最终报告时补
        回)
256
257     """
258     if settlement_mode not in ("replacement", "additive"):
259         raise ValueError(f"unknown settlement mode: {settlement_mode}")
260     load_kw, pv_kw, price, t = _validate(load_kw, pv_kw, price, storage, soc_initial,
        soc_terminal)
261     baseline = np.asarray(baseline_grid, dtype=float)
262     mask = np.asarray(adjustment_mask, dtype=bool)
263     if not (baseline.shape == mask.shape == (t,)):
264         raise ValueError("adjustment shape mismatch")
265
266     g, ch, dis, soc, spill = _slices(t)
267     inc = slice(5 * t, 6 * t)
268     dec = slice(6 * t, 7 * t)
269     n = 7 * t
270
271     masked = np.flatnonzero(mask)
272     a_eq = np.zeros((2 * t + (soc_terminal is not None) + len(masked), n))
273     b_eq = np.zeros(a_eq.shape[0])
274     _energy_balance_rows(a_eq, b_eq, t, load_kw, pv_kw, storage, soc_initial)
275     if soc_terminal is not None:
276         last = a_eq.shape[0] - 1 - len(masked)
277         a_eq[last, soc.stop - 1] = 1
278         b_eq[last] = soc_terminal
279
280     limit = storage.energy_limit
281     bounds = (
282         [(0, None)] * t
283         + [(0, limit)] * t
284         + [(0, limit)] * t
285         + [(storage.soc_min, storage.soc_max)] * t
286         + [(0, float(x) * DT_HOURS) for x in pv_kw]
287         + [((0, None) if flag else (0, 0)) for flag in mask]
288         + [((0, None) if flag else (0, 0)) for flag in mask]
289     )
290
291     # 等式约束:  $Ga - inc + dec = Gp$  (仅对允许调整的时段生效)
292     # 必须是等式: 若写成不等式, 当  $Ga < Gp$  时 LP 可取  $inc=dec=0$ , 从而漏付  $0.5p(Gp - Ga)$ 
293     # 的违约费用, 导致目标函数系统性低估向下调整成本、调度不再最优。
294     for k, i in enumerate(masked):

```

```

294     row = 2 * t + (1 if soc_terminal is not None else 0) + k
295     a_eq[row, g.start + i] = 1.0
296     a_eq[row, inc.start + i] = -1.0
297     a_eq[row, dec.start + i] = 1.0
298     b_eq[row] = baseline[i]
299 adjustment_rows: list[np.ndarray] = []
300 adjustment_rhs: list[float] = []
301
302 primary_c = np.zeros(n)
303 if settlement_mode == "replacement":
304     primary_c[g] = price
305     primary_c[inc] = np.where(mask, 0.5 * price, 0.0)
306     primary_c[dec] = np.where(mask, 0.5 * price, 0.0)
307     constant = 0.0
308 else:
309     primary_c[inc] = np.where(mask, 1.5 * price, 0.0)
310     primary_c[dec] = np.where(mask, 0.5 * price, 0.0)
311     # additive 口径下 Ga 不进目标，最优面维度较高；Ga 的实际取值由母线平衡与
312     # inc/dec 的等式关系共同决定，退化解再由后续的吞吐量层消除，无需人工微扰。
313     constant = float(np.sum(price[mask] * baseline[mask]))
314
315 throughput_c = np.zeros(n)
316 throughput_c[ch] = 1
317 throughput_c[dis] = 1
318 spill_c = np.zeros(n)
319 spill_c[spill] = 1
320 started = perf_counter()
321 # additive 口径下 Ga 本身不进目标，最优面维度远高于 replacement 口径，
322 # 因此给它更宽的层间容差，避免后序层被过紧的界判为不可行而触发退让。
323 x, tertiary, fell_back, values = _lexicographic(
324     [primary_c, throughput_c, spill_c],
325     a_eq,
326     b_eq,
327     bounds,
328     adjustment_rows,
329     adjustment_rhs,
330     slack_factor=1.0 if settlement_mode == "replacement" else 100.0,
331 )
332 primary_value = values[0]
333 adjusted = x[g].copy()
334 increase_vec = x[inc].copy()
335 decrease_vec = x[dec].copy()
336 if settlement_mode == "replacement":
337     settlement = float(np.dot(primary_c, x))
338 else:
339     settlement = constant + float(np.dot(primary_c, x))
340 dispatch = DispatchResult(

```

```

341         adjusted,
342         x[ch].copy(),
343         x[dis].copy(),
344         x[soc].copy(),
345         x[spill].copy(),
346         settlement,
347         float(primary_value + constant),
348         float(x[spill].sum()),
349         float(x[ch].sum() + x[dis].sum()),
350         int(tertiary.status),
351         str(tertiary.message),
352         int(getattr(tertiary, "nit", -1)),
353         perf_counter() - started,
354         np.asarray(tertiary.eqlin.marginals, dtype=float),
355     )
356     return AdjustmentDispatchResult(
357         dispatch,
358         increase_vec,
359         decrease_vec,
360         settlement,
361         float(np.dot(primary_c, x)),
362         mask.copy(),
363         settlement_mode,
364         constant,
365         bool(fell_back),
366         float(values[1]) if len(values) > 1 else float("nan"),
367     )

```

B.4 光伏日前预测与风险定价 (forecast.py)

Listing 4:

```

1  from __future__ import annotations
2
3  from dataclasses import dataclass
4
5  import numpy as np
6
7  from .io_data import DT_HOURS, SLOTS_PER_DAY
8
9  METHOD_WINDOWS = {"persistence": 1, "mean7": 7, "mean14": 14, "mean30": 30, "weighted14":
10                  14, "weighted30": 30}
11
12  QUANTILES = (0.80, 0.85, 0.90, 0.95)
13
14  @dataclass(frozen=True)

```

```

14 class ForecastDecision:
15     day_index: int
16     name: str
17     method: str
18     quantile: float
19     margin_kw: float
20     historical_score: float | None
21     history_days_used: int
22     forecast_kw: np.ndarray
23     candidates: dict[str, np.ndarray]
24     bases: dict[str, np.ndarray]
25     margins: dict[str, float]
26
27
28 class OnlinePVForecaster:
29     """光伏日前预测器，全部输入只来自第 d 天之前的实际光伏。
30
31     risk_mode = "asymmetric" (当前主方案)
32         在基预测上叠加历史残差分位数下修边际，候选按 4:1 非对称费用代理选型。
33     risk_mode = "point"
34         不做安全下修 (margin_kw = 0)，候选按对称 MAE 选型，用作对照。
35     """
36
37     def __init__(
38         self,
39         price: np.ndarray,
40         score_window: int = 30,
41         residual_window: int = 30,
42         risk_mode: str = "asymmetric",
43         prior_pv: np.ndarray | None = None,
44     ) -> None:
45         if risk_mode not in ("asymmetric", "point"):
46             raise ValueError(f"unknown risk_mode: {risk_mode}")
47         self.price = np.asarray(price, dtype=float)
48         if self.price.shape != (SLOTS_PER_DAY,):
49             raise ValueError("price must contain 144 slots")
50         self.prior_pv = None if prior_pv is None else np.asarray(prior_pv, dtype=float)
51         self.score_window = score_window
52         self.residual_window = residual_window
53         self.risk_mode = risk_mode
54         self.actual_history: list[np.ndarray] = []
55         self.residual_days: dict[str, list[np.ndarray]] = {m: [] for m in METHOD_WINDOWS}
56         self.score_days: dict[str, list[float]] = {self._name(m, q): [] for m in
57             METHOD_WINDOWS for q in QUANTILES}
58         if risk_mode == "point":
59             self.score_days = {m: [] for m in METHOD_WINDOWS}

```

```

60 @staticmethod
61 def _name(method: str, q: float) -> str:
62     return f"{method}_q{int(round(q * 100)):02d}"
63
64 def _base(self, method: str) -> np.ndarray:
65     if not self.actual_history:
66         # 尚无历史观测时使用附件 1 给出的典型日光伏预测作为先验
67         return np.zeros(SLOTS_PER_DAY) if self.prior_pv is None else np.clip(self.
            prior_pv, 0.0, None)
68     n = min(METHOD_WINDOWS[method], len(self.actual_history))
69     recent = np.stack(self.actual_history[-n:])
70     if method == "persistence":
71         return recent[-1].copy()
72     if method.startswith("weighted"):
73         return np.average(recent, axis=0, weights=np.arange(1, n + 1, dtype=float))
74     return recent.mean(axis=0)
75
76 def _margin(self, method: str, q: float) -> float:
77     days = self.residual_days[method][-self.residual_window :]
78     if not days:
79         return 0.0
80     residual = np.concatenate(days)
81     return 0.0 if residual.size == 0 else max(0.0, float(np.quantile(residual, q)))
82
83 def predict(self) -> ForecastDecision:
84     d = len(self.actual_history)
85     bases = {m: self._base(m) for m in METHOD_WINDOWS}
86     candidates: dict[str, np.ndarray] = {}
87     margins: dict[str, float] = {}
88     if self.risk_mode == "asymmetric":
89         for method, base in bases.items():
90             for q in QUANTILES:
91                 name = self._name(method, q)
92                 margins[name] = self._margin(method, q)
93                 candidates[name] = np.clip(base - margins[name], 0.0, None)
94             default = self._name("persistence", 0.80)
95             quantile = 0.80
96     else:
97         for method, base in bases.items():
98             margins[method] = 0.0
99             candidates[method] = np.clip(base, 0.0, None)
100         default = "persistence"
101         quantile = 0.0
102     eligible = [n for n, s in self.score_days.items() if s]
103     if d < 7 or not eligible:
104         selected = default
105         hist = None

```



```

106         else:
107             scores = {n: float(np.mean(self.score_days[n][-self.score_window :])) for n in
                        eligible}
108             selected = min(scores, key=lambda n: (scores[n], n))
109             hist = scores[selected]
110         if selected in margins:
111             method = selected
112             q = 0.0
113         else:
114             method, q_text = selected.rsplit("_q", 1)
115             q = int(q_text) / 100.0
116         return ForecastDecision(
117             d,
118             selected,
119             method,
120             q if self.risk_mode == "asymmetric" else 0.0,
121             margins[selected],
122             hist,
123             d,
124             candidates[selected].copy(),
125             candidates,
126             bases,
127             margins,
128         )
129
130     def observe(self, decision: ForecastDecision, actual_kw: np.ndarray, price: np.ndarray |
        None = None) -> dict[str, float]:
131         actual = np.asarray(actual_kw, dtype=float)
132         if actual.shape != (SLOTS_PER_DAY,) or decision.day_index != len(self.actual_history
        ):
133             raise ValueError("forecast observation out of sequence")
134         score_price = self.price if price is None else np.asarray(price, dtype=float)
135         if score_price.shape != (SLOTS_PER_DAY,):
136             raise ValueError("score price must contain 144 slots")
137         result: dict[str, float] = {}
138         for name, forecast in decision.candidates.items():
139             error = forecast - actual
140             if self.risk_mode == "asymmetric":
141                 score = float(np.sum(score_price * (4 * np.maximum(error, 0) + np.maximum(-
                    error, 0))) * DT_HOURS)
142             else:
143                 score = float(np.mean(np.abs(error)))
144             self.score_days[name].append(score)
145             result[name] = score
146         for method, base in decision.bases.items():
147             mask = (actual > 1e-9) | (base > 1e-9)
148             self.residual_days[method].append((base - actual)[mask].copy())

```

```

149         self.actual_history.append(actual.copy())
150         return result

```

B.5 因果负载预测 (load_forecast.py)

Listing 5:

```

1  """严格因果的负载预测模块。
2
3  信息集约束（本模块唯一允许读取的数据）：
4      * 第 d 天第 n 个决策点时点（n = 0/36/72/108），只允许使用
5          - 已经完整观测完的日期 d' < d 的附件 2 真实负载；
6          - 第 d 天已经执行完的时段 [0, n) 的真实负载；
7      * 任何时刻都不得读取第 d 天时段 >= n 的真实负载，也不得读取 d' > d 的数据。
8
9  预测器采用稳定、可解释的滚动历史方法，并在历史执行块上滚动在线选型
10 （选型同样只使用已经发生的误差）。
11 """
12 from __future__ import annotations
13
14 from dataclasses import dataclass
15
16 import numpy as np
17
18 from .io_data import DT_HOURS, SLOTS_PER_DAY
19
20 # 方法名 -> 回看天数 (weekday_mean 使用 4 个同 weekday 的样本)
21 METHOD_WINDOWS: dict[str, int] = {
22     "persistence": 1,
23     "mean7": 7,
24     "mean14": 14,
25     "mean30": 30,
26     "weighted14": 14,
27     "weighted30": 30,
28     "weekday_mean": 28,
29 }
30 BASE_METHODS = tuple(METHOD_WINDOWS)
31 NODE_SLOTS = (0, 36, 72, 108)
32
33
34 @dataclass(frozen=True)
35 class LoadForecastDecision:
36     """一次负载预测决策的完整留痕。"""
37
38     day_index: int
39     node: int

```

```

40     name: str # 选中的候选名 (含 _raw / _offset 后缀)
41     method: str # 基方法名
42     corrected: bool # 是否施加了日内水平偏移校正
43     offset_kw: float
44     historical_score: float | None
45     history_days_used: int
46     horizon: int
47     forecast_kw: np.ndarray
48     candidates: dict[str, np.ndarray]
49     weekday_of_target: int
50
51
52 def _daily_window_shape(history: list[np.ndarray], method: str, weekday: int) -> np.ndarray:
53     """由历史（严格早于目标日）给出目标日 144 个时段的基准预测形状。"""
54     if not history:
55         return np.zeros(SLOTS_PER_DAY, dtype=float)
56     if method == "weekday_mean":
57         same = [h for h, w in history if w == weekday]
58         if len(same) >= 2:
59             recent = np.stack(same[-4:])
60             return recent.mean(axis=0)
61         method = "mean7" # 同 weekday 样本不足时退化为 7 日均值
62     n = min(METHOD_WINDOWS[method], len(history))
63     recent = np.stack([h for h, _ in history[-n:]])
64     if method == "persistence":
65         return recent[-1].copy()
66     if method.startswith("weighted"):
67         weights = np.arange(1, n + 1, dtype=float)
68         return np.average(recent, axis=0, weights=weights)
69     return recent.mean(axis=0)
70
71
72 class OnlineLoadForecaster:
73     """因果在线负载预测器（滚动在线选型）。"""
74
75     def __init__(self, score_window: int = 30, prior_kw: np.ndarray | None = None) -> None:
76         self.score_window = score_window
77         # (负载曲线, weekday) 只在日终写入, 保证 d 日决策读不到 d 日数据
78         self.history: list[tuple[np.ndarray, int]] = []
79         # 附件 1 给出的典型日负载作为 1 月 1 日之前的先验 (不占用真实历史日)
80         self.prior_days = 0
81         if prior_kw is not None:
82             prior = np.asarray(prior_kw, dtype=float)
83             if prior.shape != (SLOTS_PER_DAY,):
84                 raise ValueError("prior load must have 144 slots")
85             self.history.append((prior.copy(), -1))
86             self.prior_days = 1

```

```

87     # 第 d 天已执行的时段观测, 长度 <= node
88     self.partial: np.ndarray = np.zeros(SLOTS_PER_DAY, dtype=float)
89     self.partial_len: int = 0
90     self.score_days: dict[tuple[int, str], list[float]] = {}
91
92     @property
93     def observed_days(self) -> int:
94         """真实历史天数 (不含先验)。"""
95         return len(self.history) - self.prior_days
96
97     # ----- #
98     # 内部工具
99     # ----- #
100     def _target_weekday(self, day_index: int, day_offset: int) -> int:
101         """目标日期相对 2025-01-01 的星期 (0=周一)。"""
102         return (day_index + day_offset) % 7
103
104     def _base_horizon(self, method: str, day_index: int, node: int, horizon: int) -> np.
105         ndarray:
106         """给出 [d, node] 起 horizon 个时段的基准预测 (可跨日)。"""
107         wd_today = self._target_weekday(day_index, 0)
108         shape_today = _daily_window_shape(self.history, method, wd_today)
109         head = shape_today[node:]
110         if len(head) >= horizon:
111             return head[:horizon].copy()
112         wd_next = self._target_weekday(day_index, 1)
113         shape_next = _daily_window_shape(self.history, method, wd_next)
114         tail = horizon - len(head)
115         while len(shape_next) < tail:
116             shape_next = np.concatenate([shape_next, shape_next])
117         return np.concatenate([head, shape_next[:tail]])
118
119     def _offset(self, method: str, day_index: int, node: int) -> float:
120         """用第 d 天已观测的 [node-k, node) 与基准预测之差估计水平偏移。"""
121         if node <= 0 or self.partial_len <= 0:
122             return 0.0
123         k = min(36, node, self.partial_len)
124         if k < 6:
125             return 0.0
126         observed = self.partial[node - k : node]
127         base = _daily_window_shape(self.history, method, self._target_weekday(day_index, 0))
128             [node - k : node]
129         return float(np.mean(observed - base))
130
131     def _candidate_names(self, node: int) -> tuple[str, ...]:
132         if node == 0:
133             return BASE_METHODS

```

```

132         return tuple(f"{m}_{suffix}" for m in BASE_METHODS for suffix in ("raw", "offset"))
133
134     def _build(self, name: str, day_index: int, node: int, horizon: int) -> tuple[np.ndarray
135         , str, bool, float]:
136         if name.endswith("_offset"):
137             method, corrected = name[: -len("_offset")], True
138         elif name.endswith("_raw"):
139             method, corrected = name[: -len("_raw")], False
140         else:
141             method, corrected = name, False
142         base = self._base_horizon(method, day_index, node, horizon)
143         offset = self._offset(method, day_index, node) if corrected else 0.0
144         return np.clip(base + offset, 0.0, None), method, corrected, offset
145
146     # ----- #
147     # 对外接口
148     # ----- #
149     def predict(self, day_index: int, node: int = 0, horizon: int = SLOTS_PER_DAY) ->
150         LoadForecastDecision:
151         """在第 d 天第 node 个时段之前生成未来 horizon 个时段的负载预测。"""
152         if node not in NODE_SLOTS:
153             raise ValueError(f"node must be one of {NODE_SLOTS}")
154         names = self._candidate_names(node)
155         candidates: dict[str, np.ndarray] = {}
156         meta: dict[str, tuple[str, bool, float]] = {}
157         for name in names:
158             forecast, method, corrected, offset = self._build(name, day_index, node, horizon
159                 )
160             candidates[name] = forecast
161             meta[name] = (method, corrected, offset)
162
163         eligible = [n for n in names if self.score_days.get((node, n))]
164         history_days = self._score_history_days(node)
165         default = "persistence" if node == 0 else "persistence_raw"
166         if history_days < 7 or not eligible:
167             selected, hist = default, None
168         else:
169             scores = {
170                 n: float(np.mean(self.score_days[(node, n)][-self.score_window :])) for n in
171                 eligible
172             }
173             selected = min(scores, key=lambda n: (scores[n], n))
174             hist = scores[selected]
175         method, corrected, offset = meta[selected]
176         return LoadForecastDecision(
177             day_index=day_index,
178             node=node,

```

```

175         name=selected,
176         method=method,
177         corrected=corrected,
178         offset_kw=offset,
179         historical_score=hist,
180         history_days_used=self.observed_days,
181         horizon=horizon,
182         forecast_kw=candidates[selected].copy(),
183         candidates=candidates,
184         weekday_of_target=self._target_weekday(day_index, 0),
185     )
186
187     def _score_history_days(self, node: int) -> int:
188         return max((len(v) for (n, _), v in self.score_days.items() if n == node), default=0)
189
190     def observe_block(self, decision: LoadForecastDecision, actual_block_kw: np.ndarray) -> dict[str, float]:
191         """执行块结束后回填误差，用于滚动在线选型。"""
192         actual = np.asarray(actual_block_kw, dtype=float)
193         if actual.ndim != 1:
194             raise ValueError("actual block must be 1-D")
195         out: dict[str, float] = {}
196         for name, forecast in decision.candidates.items():
197             segment = forecast[: len(actual)]
198             mae = float(np.mean(np.abs(segment - actual)))
199             self.score_days.setdefault((decision.node, name), []).append(mae)
200             out[name] = mae
201         return out
202
203     def observe_partial(self, node: int, actual_slots: np.ndarray) -> None:
204         """执行块完成后写入当天已观测前缀，供后续节点的水平校正使用。"""
205         if actual_slots.shape != (node,) and node != 0:
206             raise ValueError("partial block shape mismatch")
207         self.partial[:node] = actual_slots
208         self.partial_len = node
209
210     def commit_day(self, actual_full_day_kw: np.ndarray, day_index: int) -> None:
211         """日终把完整观测写入历史；此后该日才可被后续日期使用。"""
212         actual = np.asarray(actual_full_day_kw, dtype=float)
213         if actual.shape != (SLOTS_PER_DAY,):
214             raise ValueError("daily load must have 144 slots")
215         self.history.append((actual.copy(), self._target_weekday(day_index, 0)))
216         self.partial = np.zeros(SLOTS_PER_DAY, dtype=float)
217         self.partial_len = 0
218
219     # ----- #

```

```

220 def log_row(self, decision: LoadForecastDecision, actual_block_kw: np.ndarray, date_text
    : str) -> dict:
221     """任务书要求字段: date / decision_time / history_days_used / method / mae / rmse /
        电量误差。"""
222     actual = np.asarray(actual_block_kw, dtype=float)
223     forecast = decision.forecast_kw[: len(actual)]
224     error = forecast - actual
225     slot = int(decision.node * 10) // 60
226     minute = (decision.node * 10) % 60
227     label = f"{slot:02d}:{minute:02d}"
228     return {
229         "date": date_text,
230         "decision_time": label,
231         "history_days_used": decision.history_days_used,
232         "candidate": decision.name,
233         "method": decision.method,
234         "offset_corrected": decision.corrected,
235         "offset_kw": decision.offset_kw,
236         "mae_kw": float(np.mean(np.abs(error))),
237         "rmse_kw": float(np.sqrt(np.mean(error**2))),
238         "daily_energy_error_kwh": float(np.sum(error) * DT_HOURS),
239         "block_energy_error_kwh": float(np.sum(error) * DT_HOURS),
240         "historical_score": decision.historical_score,
241     }

```

B.6 因果电价预测 (price_forecast.py)

Listing 6:

```

1  """严格因果的外网电价预测模块（用于问题四）。
2
3  信息集约束:
4      * 第 d 天第 n 个决策点, 只允许使用附件 4 中 d' < d 的全部真实价格,
5        以及第 d 天已经过去时段 [0, n) 的真实价格;
6      * 不得读取第 d 天时段 >= n 的附件 4 真实价格。
7
8  价格与负载的形态差别: 电价没有"日内水平漂移"的强物理机制, 因此
9  候选只做基预测, 不额外做偏移校正; 但保留日内已观测前缀参与选型信息。
10 """
11 from __future__ import annotations
12
13 from dataclasses import dataclass
14
15 import numpy as np
16
17 from .io_data import DT_HOURS, SLOTS_PER_DAY

```

```

18 from .load_forecast import NODE_SLOTS
19
20 METHOD_WINDOWS: dict[str, int] = {
21     "persistence": 1,
22     "mean7": 7,
23     "mean14": 14,
24     "mean30": 30,
25     "weighted14": 14,
26     "weighted30": 30,
27     "weekday_mean": 28,
28 }
29 BASE_METHODS = tuple(METHOD_WINDOWS)
30
31
32 @dataclass(frozen=True)
33 class PriceForecastDecision:
34     day_index: int
35     node: int
36     name: str
37     method: str
38     historical_score: float | None
39     history_days_used: int
40     horizon: int
41     forecast_price: np.ndarray
42     candidates: dict[str, np.ndarray]
43
44
45 def _window_shape(history: list[tuple[np.ndarray, int]], method: str, weekday: int) -> np.
    ndarray:
46     if not history:
47         return np.full(SLOTS_PER_DAY, np.nan, dtype=float)
48     if method == "weekday_mean":
49         same = [h for h, w in history if w == weekday]
50         if len(same) >= 2:
51             return np.stack(same[-4:]).mean(axis=0)
52         method = "mean7"
53     n = min(METHOD_WINDOWS[method], len(history))
54     recent = np.stack([h for h, _ in history[-n:]])
55     if method == "persistence":
56         return recent[-1].copy()
57     if method.startswith("weighted"):
58         weights = np.arange(1, n + 1, dtype=float)
59         return np.average(recent, axis=0, weights=weights)
60     return recent.mean(axis=0)
61
62
63 class OnlinePriceForecaster:

```



```

64     """因果在线电价预测器。"""
65
66     def __init__(self, score_window: int = 30, prior_price: np.ndarray | None = None) ->
        None:
67         self.score_window = score_window
68         self.history: list[tuple[np.ndarray, int]] = []
69         self.prior_days = 0
70         if prior_price is not None:
71             prior = np.asarray(prior_price, dtype=float)
72             if prior.shape != (SLOTS_PER_DAY,):
73                 raise ValueError("prior price must have 144 slots")
74             self.history.append((prior.copy(), -1))
75             self.prior_days = 1
76         self.partial_len: int = 0
77         self.score_days: dict[tuple[int, str], list[float]] = {}
78
79     @property
80     def observed_days(self) -> int:
81         return len(self.history) - self.prior_days
82
83     def _weekday(self, day_index: int, offset: int) -> int:
84         return (day_index + offset) % 7
85
86     def _base_horizon(self, method: str, day_index: int, node: int, horizon: int) -> np.
        ndarray:
87         head = _window_shape(self.history, method, self._weekday(day_index, 0))[node:]
88         if len(head) >= horizon:
89             return head[:horizon].copy()
90         tail_len = horizon - len(head)
91         tail = _window_shape(self.history, method, self._weekday(day_index, 1))
92         while len(tail) < tail_len:
93             tail = np.concatenate([tail, tail])
94         return np.concatenate([head, tail[:tail_len]])
95
96     def _fallback(self) -> np.ndarray:
97         """完全无历史时退化为最近一次已知价格水平。"""
98         if not self.history:
99             return np.full(SLOTS_PER_DAY, 0.5, dtype=float)
100         return self.history[-1][0].copy()
101
102     def predict_horizon(self, day_index: int, node: int = 0, horizon: int = SLOTS_PER_DAY)
        -> PriceForecastDecision:
103         if node not in NODE_SLOTS:
104             raise ValueError(f"node must be one of {NODE_SLOTS}")
105         candidates: dict[str, np.ndarray] = {}
106         for method in BASE_METHODS:
107             horizon_values = self._base_horizon(method, day_index, node, horizon)

```

```

108         if not np.isfinite(horizon_values).all():
109             horizon_values = np.where(np.isfinite(horizon_values), horizon_values, self.
                _fallback()[node % SLOTS_PER_DAY])
110             candidates[method] = np.clip(horizon_values, 1e-6, None)
111         eligible = [n for n in BASE_METHODS if self.score_days.get((node, n))]
112         history_days = max((len(v) for (n, _) in self.score_days.items() if n == node),
            default=0)
113         if history_days < 7 or not eligible:
114             selected, hist = "persistence", None
115         else:
116             scores = {n: float(np.mean(self.score_days[(node, n)][-self.score_window :]))
                for n in eligible}
117             selected = min(scores, key=lambda n: (scores[n], n))
118             hist = scores[selected]
119         return PriceForecastDecision(
120             day_index=day_index,
121             node=node,
122             name=selected,
123             method=selected,
124             historical_score=hist,
125             history_days_used=self.observed_days,
126             horizon=horizon,
127             forecast_price=candidates[selected].copy(),
128             candidates=candidates,
129         )
130
131     def observe_block(self, decision: PriceForecastDecision, actual_block: np.ndarray) ->
        dict[str, float]:
132         actual = np.asarray(actual_block, dtype=float)
133         out: dict[str, float] = {}
134         for name, forecast in decision.candidates.items():
135             segment = forecast[: len(actual)]
136             mae = float(np.mean(np.abs(segment - actual)))
137             self.score_days.setdefault((decision.node, name), []).append(mae)
138             out[name] = mae
139         return out
140
141     def observe_partial(self, node: int) -> None:
142         self.partial_len = node
143
144     def commit_day(self, actual_full_day: np.ndarray, day_index: int) -> None:
145         actual = np.asarray(actual_full_day, dtype=float)
146         if actual.shape != (SLOTS_PER_DAY,):
147             raise ValueError("daily price must have 144 slots")
148         self.history.append((actual.copy(), self._weekday(day_index, 0)))
149         self.partial_len = 0
150

```

```

151     def log_row(self, decision: PriceForecastDecision, actual_block: np.ndarray, date_text:
152         str) -> dict:
153         actual = np.asarray(actual_block, dtype=float)
154         forecast = decision.forecast_price[: len(actual)]
155         error = forecast - actual
156         slot = int(decision.node * 10) // 60
157         minute = (decision.node * 10) % 60
158         return {
159             "date": date_text,
160             "decision_time": f"{slot:02d}:{minute:02d}",
161             "history_days_used": decision.history_days_used,
162             "method": decision.method,
163             "price_mae": float(np.mean(np.abs(error))),
164             "price_rmse": float(np.sqrt(np.mean(error**2))),
165             "price_energy_weighted_error": float(
166                 np.sum(np.abs(error) * actual) / max(np.sum(actual), 1e-9)
167             ),
168             "daily_price_energy_weighted_error": float(
169                 np.sum(np.abs(error) * actual) * DT_HOURS
170             ),
171             "historical_score": decision.historical_score,
172         }

```

B.7 实时储能反馈与执行 (settle.py)

Listing 7:

```

1  from __future__ import annotations
2
3  from dataclasses import dataclass
4
5  import numpy as np
6
7  from .io_data import DT_HOURS
8  from .lp_core import DispatchResult
9  from .params import DEFAULT_STORAGE, StorageParams
10
11  TOL = 1e-10
12
13
14  @dataclass(frozen=True)
15  class ExecutionResult:
16      grid: np.ndarray
17      charge: np.ndarray
18      discharge: np.ndarray
19      soc: np.ndarray

```

```

20     curtail: np.ndarray
21     emergency: np.ndarray
22
23
24 def execute_with_realtime_storage_feedback(
25     plan: DispatchResult,
26     load_kw: np.ndarray,
27     actual_pv_kw: np.ndarray,
28     *,
29     soc_initial: float,
30     storage: StorageParams = DEFAULT_STORAGE,
31 ) -> ExecutionResult:
32     """实时储能反馈 (realtime_feedback = ON) 。
33
34     真实光伏低于计划时：先取消充电 → 再增加放电 → 最后紧急购电；
35     真实光伏高于计划时：先减少放电 → 再增加充电 → 最后弃光。
36     每步只使用当前时刻已发生的真实光伏与当前 SOC，不使用任何未来信息。
37
38     """
39     load = np.asarray(load_kw, dtype=float)
40     pv = np.asarray(actual_pv_kw, dtype=float)
41     n = len(load)
42     if pv.shape != load.shape or len(plan.grid) != n:
43         raise ValueError("execution shape mismatch")
44     limit = storage.energy_limit
45     c_out = np.zeros(n)
46     h_out = np.zeros(n)
47     soc = np.zeros(n)
48     spill = np.zeros(n)
49     emergency = np.zeros(n)
50     state = float(soc_initial)
51     for t in range(n):
52         c = min(float(plan.charge[t]), limit, max(0.0, (storage.soc_max - state) / storage.
53             eta_ch))
54         h = min(float(plan.discharge[t]), limit, max(0.0, (state - storage.soc_min) *
55             storage.eta_dis))
56         net = float(plan.grid[t] + pv[t] * DT_HOURS + h - load[t] * DT_HOURS - c)
57         if net < -TOL:
58             deficit = -net
59             reduce = min(c, deficit)
60             c -= reduce
61             deficit -= reduce
62             max_h = max(0.0, (state + storage.eta_ch * c - storage.soc_min) * storage.
63                 eta_dis)
64             add = min(deficit, limit - h, max(0.0, max_h - h))
65             h += add
66             deficit -= add
67             emergency[t] = max(0.0, deficit)

```

```

64         elif net > TOL:
65             surplus = net
66             reduce = min(h, surplus)
67             h -= reduce
68             surplus -= reduce
69             max_c = max(0.0, (storage.soc_max - (state - h / storage.eta_dis)) / storage.
                           eta_ch)
70             add = min(surplus, limit - c, max(0.0, max_c - c))
71             c += add
72             surplus -= add
73             spill[t] = max(0.0, surplus)
74             state = state + storage.eta_ch * c - h / storage.eta_dis
75             if not storage.soc_min - 1e-6 <= state <= storage.soc_max + 1e-6:
76                 raise RuntimeError(f"SOC invalid at {t}: {state}")
77             c_out[t] = c
78             h_out[t] = h
79             soc[t] = min(storage.soc_max, max(storage.soc_min, state))
80         return ExecutionResult(plan.grid.copy(), c_out, h_out, soc, spill, emergency)
81
82
83     def execute_fixed_storage_plan(
84         plan: DispatchResult,
85         load_kw: np.ndarray,
86         actual_pv_kw: np.ndarray,
87         *,
88         soc_initial: float,
89         storage: StorageParams = DEFAULT_STORAGE,
90     ) -> ExecutionResult:
91         """固定储能计划执行 (realtime_feedback = OFF) 。
92
93         储能充放电严格等于计划值，只保留 SOC 物理边界的被动钳位：
94         真实不足部分全部进入 5 倍电价紧急购电，真实富余部分全部弃光。
95         该口径用于度量 10 min 实时储能 recourse 对总费用的贡献。
96         """
97         load = np.asarray(load_kw, dtype=float)
98         pv = np.asarray(actual_pv_kw, dtype=float)
99         n = len(load)
100         if pv.shape != load.shape or len(plan.grid) != n:
101             raise ValueError("execution shape mismatch")
102         limit = storage.energy_limit
103         c_out = np.zeros(n)
104         h_out = np.zeros(n)
105         soc = np.zeros(n)
106         spill = np.zeros(n)
107         emergency = np.zeros(n)
108         state = float(soc_initial)
109         for t in range(n):

```

```

110     # 被动钳位：计划值超出当前 SOC 可达范围时按边界截断（不主动做能量补偿）
111     c = min(float(plan.charge[t]), limit, max(0.0, (storage.soc_max - state) / storage.
112           eta_ch))
113     h = min(float(plan.discharge[t]), limit, max(0.0, (state - storage.soc_min) *
114           storage.eta_dis))
115     net = float(plan.grid[t] + pv[t] * DT_HOURS + h - load[t] * DT_HOURS - c)
116     if net < 0:
117         emergency[t] = -net
118     elif net > 0:
119         spill[t] = net
120     state = state + storage.eta_ch * c - h / storage.eta_dis
121     if not storage.soc_min - 1e-6 <= state <= storage.soc_max + 1e-6:
122         raise RuntimeError(f"SOC invalid at {t}: {state}")
123     c_out[t] = c
124     h_out[t] = h
125     soc[t] = min(storage.soc_max, max(storage.soc_min, state))
126     return ExecutionResult(plan.grid.copy(), c_out, h_out, soc, spill, emergency)
127
128 def execute_plan(
129     plan: DispatchResult,
130     load_kw: np.ndarray,
131     actual_pv_kw: np.ndarray,
132     *,
133     soc_initial: float,
134     realtime_feedback: bool = True,
135     storage: StorageParams = DEFAULT_STORAGE,
136 ) -> ExecutionResult:
137     if realtime_feedback:
138         return execute_with_realtime_storage_feedback(
139             plan, load_kw, actual_pv_kw, soc_initial=soc_initial, storage=storage
140         )
141     return execute_fixed_storage_plan(plan, load_kw, actual_pv_kw, soc_initial=soc_initial,
142           storage=storage)

```

B.8 问题一求解 (q1.py)

Listing 8:

```

1 from __future__ import annotations
2 import csv,json,platform,sys
3 from pathlib import Path
4 import numpy as np
5 import scipy
6 import openpyxl
7 from .export import export_result1

```

```

8 from .io_data import DEFAULT_DATA_ROOT,DT_HOURS,read_attachment1,template_path
9 from .lp_core import ETA_CH,ETA_DIS,SOC_MAX,SOC_MIN,solve_dispatch
10
11 ROOT=Path(__file__).resolve().parents[1]; OUTPUT=ROOT/"outputs"/"q1"
12
13 def run_q1(*,data_root:Path=DEFAULT_DATA_ROOT)->dict:
14     data=read_attachment1(data_root)
15     result=solve_dispatch(data.load_kw,data.pv_kw,data.price,soc_initial=6000,soc_terminal
16                           =6000)
17     balance=result.grid+data.pv_kw*DT_HOURS+result.discharge-data.load_kw*DT_HOURS-result.
18               charge-result.curtail
19     previous=np.r_[6000.0,result.soc[:-1]]
20     soc_res=result.soc-(previous+ETA_CH*result.charge-result.discharge/ETA_DIS)
21     no_storage_grid=np.maximum((data.load_kw-data.pv_kw)*DT_HOURS,0)
22     metrics={"cost_yuan":result.objective,"primary_cost_yuan":result.primary_objective,
23             "primary_cost_gap_yuan":result.objective-result.primary_objective,
24             "grid_energy_kwh":float(result.grid.sum()),"charge_energy_kwh":float(result.charge.
25                                         sum()),
26             "discharge_energy_kwh":float(result.discharge.sum()),"curtailment_energy_kwh":float(
27                 result.curtail.sum()),
28             "no_storage_cost_yuan":float(np.dot(data.price,no_storage_grid)),
29             "savings_yuan":float(np.dot(data.price,no_storage_grid)-result.objective),
30             "savings_percent":float(100*(np.dot(data.price,no_storage_grid)-result.objective)/np
31                 .dot(data.price,no_storage_grid)),
32             "initial_soc_kwh":6000.0,"terminal_soc_kwh":float(result.soc[-1]),
33             "min_soc_kwh":float(result.soc.min()),"max_soc_kwh":float(result.soc.max()),
34             "max_balance_residual":float(np.max(np.abs(balance))),"max_soc_residual":float(np.
35                 max(np.abs(soc_res))),
36             "max_simultaneous_charge_discharge":float(np.max(np.minimum(result.charge,result.
37                 discharge))),
38             "solver_status":result.status,"solver_message":result.message,"iterations":result.
39                 nit,
40             "solve_seconds":result.solve_seconds}
41     metrics["passed"]=bool(metrics["max_balance_residual"]<1e-7 and metrics["
42                             max_soc_residual"]<1e-7 and
43                             abs(result.soc[-1]-6000)<1e-7 and metrics["min_soc_kwh"]>=SOC_MIN-1e-7 and
44                             metrics["max_soc_kwh"]<=SOC_MAX+1e-7 and metrics["max_simultaneous_charge_discharge"
45                                 ]<1e-5)
46     OUTPUT.mkdir(parents=True,exist_ok=True)
47     with (OUTPUT/"solution.csv").open("w",newline="",encoding="utf-8-sig") as f:
48         fields=["slot","label","price","load_kw","pv_kw","grid_kwh","charge_kwh","
49                 discharge_kwh","soc_kwh","curtail_kwh"]
50         writer=csv.DictWriter(f,fieldnames=fields); writer.writeheader()
51         for t in range(144):
52             writer.writerow({"slot":t+1,"label":data.labels[t],"price":data.price[t],"
53                             load_kw":data.load_kw[t],
54                             "pv_kw":data.pv_kw[t],"grid_kwh":result.grid[t],"charge_kwh":result.charge[t]

```

```

        ],
43         "discharge_kwh":result.discharge[t],"soc_kwh":result.soc[t],"curtail_kwh":
            result.curtail[t]})
44     check=export_result1(template_path("result1.xlsx",data_root),OUTPUT/"result1.xlsx",
        result)
45     metrics["xlsx_passed"]=check["passed"]
46     (OUTPUT/"metrics.json").write_text(json.dumps(metrics,ensure_ascii=False,indent=2),
        encoding="utf-8")
47     (OUTPUT/"xlsx_verification.json").write_text(json.dumps(check,ensure_ascii=False,indent
        =2),encoding="utf-8")
48     env={"python":sys.version,"platform":platform.platform(),"numpy":np.__version__,"scipy":
        scipy.__version__,
49         "openpyxl":openpyxl.__version__,"algorithm":"HiGHS lexicographic LP"}
50     (OUTPUT/"environment.json").write_text(json.dumps(env,ensure_ascii=False,indent=2),
        encoding="utf-8")
51     return {"metrics":metrics,"result":result}
52
53 if __name__=="__main__": print(json.dumps(run_q1()["metrics"],ensure_ascii=False,indent=2))

```

B.9 问题二求解 (q2.py)

Listing 9:

```

1  from __future__ import annotations
2
3  import csv
4  import json
5  import platform
6  import sys
7  from collections import Counter
8  from pathlib import Path
9
10 import numpy as np
11 import scipy
12
13 from .export import export_result2
14 from .forecast import OnlinePVForecaster
15 from .io_data import DEFAULT_DATA_ROOT, DT_HOURS, read_attachment1, read_attachment2,
    template_path
16 from .load_forecast import OnlineLoadForecaster
17 from .lp_core import solve_dispatch
18 from .params import DEFAULT_STORAGE, StorageParams
19 from .price_forecast import OnlinePriceForecaster
20 from .settle import execute_plan
21
22 ROOT = Path(__file__).resolve().parents[1]

```



```

23 OUTPUT = ROOT / "outputs" / "q2"
24
25
26 def _verify_day(record: dict, storage: StorageParams) -> dict:
27     execution = record["execution"]
28     load = record["load_kw"]
29     pv = record["actual_pv_kw"]
30     balance = (
31         execution.grid
32         + pv * DT_HOURS
33         + execution.discharge
34         + execution.emergency
35         - load * DT_HOURS
36         - execution.charge
37         - execution.curtail
38     )
39     previous = np.r_[record["soc_initial"], execution.soc[:-1]]
40     soc_residual = execution.soc - (previous + storage.eta_ch * execution.charge - execution
41                                     .discharge / storage.eta_dis)
42     return {
43         "max_balance_residual": float(np.max(np.abs(balance))),
44         "max_soc_residual": float(np.max(np.abs(soc_residual))),
45         "min_soc": float(execution.soc.min()),
46         "max_soc": float(execution.soc.max()),
47         "max_simultaneous": float(np.max(np.minimum(execution.charge, execution.discharge)))
48     },
49
50 def _emergency_intervals(emergency: np.ndarray, threshold: float = 1e-7) -> int:
51     """统计紧急购电的连续时段块数（相邻时段合并为一个时间段）。"""
52     active = emergency > threshold
53     if not active.any():
54         return 0
55     return int(np.sum(active[1:] & ~active[:-1]) + (1 if active[0] else 0))
56
57
58 def run_q2(
59     *,
60     data_root: Path = DEFAULT_DATA_ROOT,
61     max_days: int = 365,
62     price_matrix: np.ndarray | None = None,
63     output_dir: Path | None = None,
64     template_name: str = "result2.xlsx",
65     write_outputs: bool = True,
66     load_mode: str = "causal",
67     pv_risk_mode: str = "asymmetric",

```

```

68     realtime_feedback: bool = True,
69     terminal_policy: str = "daily_cycle",
70     price_mode: str = "perfect",
71     storage: StorageParams = DEFAULT_STORAGE,
72     use_typical_prior: bool = True,
73     tag: str = "",
74 ) -> dict:
75     """问题二（及问题四对应问题二）日滚动调度。
76
77     信息集说明：
78         load_mode = "perfect" 计划使用当日真实负载（旧基准，含未来信息）
79         = "causal" 计划使用 OnlineLoadForecaster 的因果预测
80         price_mode = "perfect" 计划使用当日附件 4 真实电价
81         = "causal" 计划使用 OnlinePriceForecaster 的因果预测
82         pv_risk_mode 见 forecast.OnlinePVForecaster
83         realtime_feedback 见 settle.execute_plan
84         terminal_policy = "daily_cycle" 计划层 S_144 = 当日实际 S_0; "free" 不约束
85     """
86     if load_mode not in ("perfect", "causal"):
87         raise ValueError(f"unknown load_mode: {load_mode}")
88     if price_mode not in ("perfect", "causal"):
89         raise ValueError(f"unknown price_mode: {price_mode}")
90     if terminal_policy not in ("daily_cycle", "free"):
91         raise ValueError(f"unknown terminal_policy: {terminal_policy}")
92
93     day1 = read_attachment1(data_root)
94     annual = read_attachment2(data_root)
95     if max_days < 1 or max_days > 365:
96         raise ValueError("max_days must be 1..365")
97     prices = np.tile(day1.price, (365, 1)) if price_matrix is None else np.asarray(
98         price_matrix, dtype=float)
99     if prices.shape != (365, 144):
100         raise ValueError("price matrix must have shape (365,144)")
101
102     out_dir = OUTPUT if output_dir is None else Path(output_dir)
103
104     # 附件 1 的典型日曲线作为 2025-01-01 之前的先验（在论文假设中显式声明）；
105     # use_typical_prior=False 用于复现不含先验的 legacy 基准结果。
106     prior_pv = day1.pv_kw if use_typical_prior else None
107     prior_load = day1.load_kw if use_typical_prior else None
108     prior_price = day1.price if use_typical_prior else None
109     selector = OnlinePVForecaster(day1.price, risk_mode=pv_risk_mode, prior_pv=prior_pv)
110     load_forecaster = OnlineLoadForecaster(prior_kw=prior_load)
111     price_forecaster = OnlinePriceForecaster(prior_price=prior_price)
112
113     state = 6000.0
114     records: list[dict] = []
115     forecast_rows: list[dict] = []

```

```

114 load_rows: list[dict] = []
115 price_rows: list[dict] = []
116 verify_rows: list[dict] = []
117
118 for d in range(max_days):
119     initial = state
120
121     # ---- 决策时刻 0:00 可用信息 -----
122     load_decision = load_forecaster.predict(d, 0, 144)
123     pv_decision = selector.predict()
124     if pv_decision.history_days_used != d:
125         raise RuntimeError("PV forecast causality counter mismatch")
126     price_decision = price_forecaster.predict_horizon(d, 0, 144) if price_mode == "
        causal" else None
127
128     load_plan = annual.load_kw[d] if load_mode == "perfect" else load_decision.
        forecast_kw
129     price_plan = prices[d] if price_mode == "perfect" else price_decision.forecast_price
130     soc_terminal = initial if terminal_policy == "daily_cycle" else None
131
132     # ---- 日前计划优化（只使用上面确定的信息） -----
133     plan = solve_dispatch(
134         load_plan,
135         pv_decision.forecast_kw,
136         price_plan,
137         soc_initial=initial,
138         soc_terminal=soc_terminal,
139         storage=storage,
140     )
141
142     # ---- 真实执行（真实负载 / 真实光伏，实时储能反馈可开关） -----
143     execution = execute_plan(
144         plan,
145         annual.load_kw[d],
146         annual.pv_kw[d],
147         soc_initial=initial,
148         realtime_feedback=realtime_feedback,
149         storage=storage,
150     )
151
152     # ---- 用真实电价结算 -----
153     plan_cost = float(np.dot(prices[d], plan.grid))
154     emergency_cost = float(np.dot(5.0 * prices[d], execution.emergency))
155     state = float(execution.soc[-1])
156
157     record = {
158         "day_index": d,

```

```

159         "date": annual.dates[d],
160         "soc_initial": initial,
161         "plan": plan,
162         "execution": execution,
163         "load_kw": annual.load_kw[d],
164         "actual_pv_kw": annual.pv_kw[d],
165         "load_plan_kw": np.asarray(load_plan, dtype=float),
166         "pv_plan_kw": pv_decision.forecast_kw,
167         "price_plan": np.asarray(price_plan, dtype=float),
168         "plan_cost": plan_cost,
169         "emergency_cost": emergency_cost,
170         "total_cost": plan_cost + emergency_cost,
171         "load_decision": load_decision,
172         "pv_decision": pv_decision,
173         "adjusted_grid": execution.grid,
174     }
175     records.append(record)
176     verify_rows.append(_verify_day(record, storage))
177
178     # ---- 日志留痕 -----
179     forecast_rows.append(
180         {
181             "day_index": d,
182             "date": annual.dates[d].isoformat(),
183             "history_days_used": pv_decision.history_days_used,
184             "candidate": pv_decision.name,
185             "method": pv_decision.method,
186             "quantile": pv_decision.quantile,
187             "margin_kw": pv_decision.margin_kw,
188             "historical_score": pv_decision.historical_score,
189             "pv_forecast_mae_kw": float(np.mean(np.abs(pv_decision.forecast_kw - annual.
190                 pv_kw[d]))),
191             "pv_forecast_rmse_kw": float(np.sqrt(np.mean((pv_decision.forecast_kw -
192                 annual.pv_kw[d]) ** 2))),
193             "pv_daily_energy_error_kwh": float(np.sum(pv_decision.forecast_kw - annual.
194                 pv_kw[d]) * DT_HOURS),
195         }
196     )
197     load_rows.append(load_forecaster.log_row(load_decision, annual.load_kw[d], annual.
198         dates[d].isoformat()))
199     if price_decision is not None:
200         price_rows.append(
201             price_forecaster.log_row(price_decision, prices[d], annual.dates[d].
202                 isoformat())
203         )
204
205     # ---- 观测回填（严格在决策之后） -----

```

```

201     selector.observe(pv_decision, annual.pv_kw[d], price=np.asarray(price_plan, dtype=
202         float))
203     load_forecaster.observe_block(load_decision, annual.load_kw[d])
204     load_forecaster.commit_day(annual.load_kw[d], d)
205     if price_decision is not None:
206         price_forecaster.observe_block(price_decision, prices[d])
207         price_forecaster.commit_day(prices[d], d)
208
209     delivery = records[31:] if max_days == 365 else []
210     min_soc = min(r["min_soc"] for r in verify_rows)
211     max_soc = max(r["max_soc"] for r in verify_rows)
212     max_balance = max(r["max_balance_residual"] for r in verify_rows)
213     max_soc_res = max(r["max_soc_residual"] for r in verify_rows)
214     max_sim = max(r["max_simultaneous"] for r in verify_rows)
215     continuity = (
216         max(abs(records[i]["soc_initial"] - records[i - 1]["execution"].soc[-1]) for i in
217             range(1, len(records)))
218         if len(records) > 1
219         else 0.0
220     )
221
222     metrics = {
223         "case_tag": tag,
224         "days_simulated": max_days,
225         "warmup_days": 31 if max_days == 365 else None,
226         "delivery_days": len(delivery),
227         "load_mode": load_mode,
228         "pv_risk_mode": pv_risk_mode,
229         "price_mode": price_mode,
230         "realtime_feedback": bool(realtime_feedback),
231         "terminal_policy": terminal_policy,
232         "storage": storage.as_dict(),
233         "price_source": "attachment1" if price_matrix is None else "attachment4",
234         "load_information": (
235             "Attachment 2 same-day true load curve (perfect-information benchmark)"
236             if load_mode == "perfect"
237             else "Online causal forecast from days strictly before d"
238         ),
239         "pv_information": (
240             "Causal forecast from days strictly before d, "
241             + ("historical residual quantile safety margin" if pv_risk_mode == "asymmetric"
242               else "no safety margin (point)")
243         ),
244         "price_information": (
245             "Attachment 4 same-day true price curve (perfect-information benchmark)"
246             if price_mode == "perfect"
247             else "Online causal forecast from days strictly before d"

```

```

245     ),
246     "realtime_storage_feedback": bool(realtime_feedback),
247     "terminal_policy_definition": (
248         "planned S_144 equals actual S_0 of the same day" if terminal_policy == "
            daily_cycle" else "no terminal SOC constraint"
249     ),
250     "max_balance_residual": max_balance,
251     "max_soc_residual": max_soc_res,
252     "max_day_boundary_soc_gap": continuity,
253     "min_soc_kwh": min_soc,
254     "max_soc_kwh": max_soc,
255     "max_simultaneous_charge_discharge": max_sim,
256     "forecast_selection_counts": dict(Counter(row["candidate"] for row in forecast_rows)
        ),
257     "load_selection_counts": dict(Counter(row["candidate"] for row in load_rows)),
258     "price_selection_counts": dict(Counter(row["method"] for row in price_rows)) if
        price_rows else None,
259 }
260 if delivery:
261     metrics.update(
262         {
263             "total_cost_yuan": float(sum(r["total_cost"] for r in delivery)),
264             "plan_purchase_cost_yuan": float(sum(r["plan_cost"] for r in delivery)),
265             "adjustment_cost_yuan": 0.0,
266             "emergency_cost_yuan": float(sum(r["emergency_cost"] for r in delivery)),
267             "plan_purchase_energy_kwh": float(sum(r["plan"].grid.sum() for r in delivery
                )),
268             "adjusted_purchase_energy_kwh": float(sum(r["adjusted_grid"].sum() for r in
                delivery)),
269             "emergency_energy_kwh": float(sum(r["execution"].emergency.sum() for r in
                delivery)),
270             "curtailment_energy_kwh": float(sum(r["execution"].curtail.sum() for r in
                delivery)),
271             "days_with_emergency": int(sum(r["execution"].emergency.sum() > 1e-7 for r
                in delivery)),
272             "emergency_interval_count": int(
273                 sum(_emergency_intervals(r["execution"].emergency) for r in delivery)
274             ),
275             "storage_charge_energy_kwh": float(sum(r["execution"].charge.sum() for r in
                delivery)),
276             "storage_discharge_energy_kwh": float(sum(r["execution"].discharge.sum() for
                r in delivery)),
277             "storage_throughput_kwh": float(
278                 sum(r["execution"].charge.sum() + r["execution"].discharge.sum() for r
                in delivery)
279             ),
280             "ending_soc_kwh": float(delivery[-1]["execution"].soc[-1]),

```

```

281         "load_forecast_mae_kw": float(np.mean([row["mae_kw"] for row in load_rows
282             [31:]])),
283         "load_forecast_rmse_kw": float(np.mean([row["rmse_kw"] for row in load_rows
284             [31:]])),
285         "pv_forecast_mae_kw": float(np.mean([row["pv_forecast_mae_kw"] for row in
286             forecast_rows[31:]])),
287         "pv_forecast_rmse_kw": float(np.mean([row["pv_forecast_rmse_kw"] for row in
288             forecast_rows[31:]])),
289         "price_forecast_mae": float(np.mean([row["price_mae"] for row in price_rows
290             [31:]])) if price_rows else None,
291         "price_forecast_rmse": float(np.mean([row["price_rmse"] for row in
292             price_rows[31:]])) if price_rows else None,
293         "runtime_seconds": float(sum(r["plan"].solve_seconds for r in delivery)),
294     }
295 )
296
297 metrics["passed"] = bool(
298     max_balance < 1e-7
299     and max_soc_res < 1e-7
300     and continuity < 1e-9
301     and min_soc >= storage.soc_min - 1e-7
302     and max_soc <= storage.soc_max + 1e-7
303     and max_sim < 1e-7
304 )
305
306 if write_outputs:
307     out_dir.mkdir(parents=True, exist_ok=True)
308     (out_dir / "metrics.json").write_text(json.dumps(metrics, ensure_ascii=False, indent
309         =2), encoding="utf-8")
310     _write_csv(out_dir / "forecast_log.csv", forecast_rows)
311     _write_csv(out_dir / "load_forecast_log.csv", load_rows)
312     if price_rows:
313         _write_csv(out_dir / "price_forecast_log.csv", price_rows)
314     if delivery:
315         fields = [
316             "date", "slot", "load_kw", "actual_pv_kw", "load_plan_kw", "pv_plan_kw", "
317                 price",
318             "grid_plan_kwh", "grid_actual_kwh", "charge_actual_kwh", "
319                 discharge_actual_kwh",
320             "soc_actual_kwh", "curtail_actual_kwh", "emergency_kwh",
321         ]
322         with (out_dir / "solution.csv").open("w", newline="", encoding="utf-8-sig") as f
323             :
324                 writer = csv.DictWriter(f, fieldnames=fields)
325                 writer.writeheader()
326                 for r in delivery:
327                     for t in range(144):
328                         writer.writerow(

```

```

318         {
319             "date": r["date"].isoformat(),
320             "slot": t + 1,
321             "load_kw": r["load_kw"][t],
322             "actual_pv_kw": r["actual_pv_kw"][t],
323             "load_plan_kw": r["load_plan_kw"][t],
324             "pv_plan_kw": r["pv_plan_kw"][t],
325             "price": prices[r["day_index"], t],
326             "grid_plan_kwh": r["plan"].grid[t],
327             "grid_actual_kwh": r["adjusted_grid"][t],
328             "charge_actual_kwh": r["execution"].charge[t],
329             "discharge_actual_kwh": r["execution"].discharge[t],
330             "soc_actual_kwh": r["execution"].soc[t],
331             "curtail_actual_kwh": r["execution"].curtail[t],
332             "emergency_kwh": r["execution"].emergency[t],
333         }
334     )
335     daily_fields = [
336         "date", "soc_initial", "soc_terminal", "plan_cost", "emergency_cost", "
337         total_cost",
338         "plan_energy", "emergency_energy", "curtailment_energy",
339     ]
340     _write_csv(
341         out_dir / "daily_metrics.csv",
342         [
343             {
344                 "date": r["date"].isoformat(),
345                 "soc_initial": r["soc_initial"],
346                 "soc_terminal": r["execution"].soc[-1],
347                 "plan_cost": r["plan_cost"],
348                 "emergency_cost": r["emergency_cost"],
349                 "total_cost": r["total_cost"],
350                 "plan_energy": r["plan"].grid.sum(),
351                 "emergency_energy": r["execution"].emergency.sum(),
352                 "curtailment_energy": r["execution"].curtail.sum(),
353             }
354             for r in delivery
355         ],
356         daily_fields,
357     )
358     check = export_result2(template_path(template_name, data_root), out_dir /
359         template_name, delivery)
360     (out_dir / "xlsx_verification.json").write_text(
361         json.dumps(check, ensure_ascii=False, indent=2), encoding="utf-8"
362     )
363     metrics["xlsx_passed"] = check["passed"]
364     (out_dir / "metrics.json").write_text(json.dumps(metrics, ensure_ascii=False,

```



```

        indent=2), encoding="utf-8")
363
364     environment = {
365         "python": sys.version,
366         "platform": platform.platform(),
367         "numpy": np.__version__,
368         "scipy": scipy.__version__,
369         "algorithm": "scipy.optimize.linprog(method=highs), lexicographic 3-pass LP",
370         "case_tag": tag,
371     }
372     if write_outputs:
373         (out_dir / "environment.json").write_text(json.dumps(environment, ensure_ascii=False
        , indent=2), encoding="utf-8")
374     return {"metrics": metrics, "records": records, "forecast_rows": forecast_rows, "
        load_rows": load_rows}
375
376
377 def _write_csv(path: Path, rows: list[dict], fields: list[str] | None = None) -> None:
378     if not rows:
379         return
380     fieldnames = fields if fields is not None else list(rows[0])
381     with path.open("w", newline="", encoding="utf-8-sig") as f:
382         writer = csv.DictWriter(f, fieldnames=fieldnames)
383         writer.writeheader()
384         writer.writerows(rows)
385
386
387 if __name__ == "__main__":
388     print(json.dumps(run_q2()["metrics"], ensure_ascii=False, indent=2))

```

B.10 问题三求解 (q3.py)

Listing 10:

```

1  from __future__ import annotations
2
3  import json
4  import platform
5  import sys
6  from pathlib import Path
7
8  import numpy as np
9  import scipy
10
11 from .export import export_result3
12 from .io_data import (

```

```

13     DEFAULT_DATA_ROOT,
14     DT_HOURS,
15     hourly_to_ten_min,
16     read_attachment1,
17     read_attachment2,
18     read_attachment3,
19     template_path,
20 )
21 from .load_forecast import OnlineLoadForecaster
22 from .lp_core import DispatchResult, settlement_cost, solve_adjustment_dispatch,
    solve_dispatch
23 from .official_forecast import OnlineForecastBiasCalibrator
24 from .params import DEFAULT_STORAGE, StorageParams
25 from .price_forecast import OnlinePriceForecaster
26 from .q2 import _emergency_intervals, _write_csv
27 from .settle import ExecutionResult, execute_plan
28
29 ROOT = Path(__file__).resolve().parents[1]
30 OUTPUT = ROOT / "outputs" / "q3"
31 BLOCK = 36
32 ALL_NODES = (0, 36, 72, 108)
33
34
35 def _horizon(data: np.ndarray, day: int, start_slot: int, fallback: np.ndarray) -> np.
    ndarray:
36     flat = data.reshape(-1)
37     start = day * 144 + start_slot
38     values = flat[start : min(start + 144, len(flat))]
39     if len(values) < 144:
40         pieces = [values]
41         remaining = 144 - len(values)
42         while remaining > 0:
43             take = min(remaining, len(fallback))
44             pieces.append(fallback[:take])
45             remaining -= take
46         values = np.concatenate(pieces)
47     return np.asarray(values, dtype=float)
48
49
50 def _slice_dispatch(result: DispatchResult, start: int, stop: int) -> DispatchResult:
51     return DispatchResult(
52         result.grid[start:stop].copy(),
53         result.charge[start:stop].copy(),
54         result.discharge[start:stop].copy(),
55         result.soc[start:stop].copy(),
56         result.curtail[start:stop].copy(),
57         result.objective,

```

```

58     result.primary_objective,
59     float(result.curtail[start:stop].sum()),
60     float(result.charge[start:stop].sum() + result.discharge[start:stop].sum()),
61     result.status,
62     result.message,
63     result.nit,
64     result.solve_seconds,
65     result.equality_marginals.copy(),
66 )
67
68
69 def _reversal_count(revisions: np.ndarray) -> int:
70     count = 0
71     for slot in range(144):
72         values = revisions[:, slot]
73         values = values[np.isfinite(values)]
74         signs = np.sign(np.diff(values))
75         signs = signs[np.abs(signs) > 0]
76         if len(signs) > 1 and np.any(signs[1:] * signs[:-1] < 0):
77             count += 1
78     return count
79
80
81 def _node_label(node: int) -> str:
82     return f"{node // 6:02d}:00"
83
84
85 def run_q3(
86     *,
87     data_root: Path = DEFAULT_DATA_ROOT,
88     max_days: int = 365,
89     downscale: str = "linear",
90     calibrate: bool = True,
91     update_nodes: tuple[int, ...] = ALL_NODES,
92     settlement_mode: str = "replacement",
93     load_mode: str = "causal",
94     price_mode: str = "perfect",
95     realtime_feedback: bool = True,
96     terminal_policy: str = "daily_cycle",
97     price_matrix: np.ndarray | None = None,
98     storage: StorageParams = DEFAULT_STORAGE,
99     output_dir: Path | None = None,
100     template_name: str = "result3.xlsx",
101     write_outputs: bool = True,
102     use_typical_prior: bool = True,
103     tag: str = "",
104 ) -> dict:

```

```

105 """问题三（及问题四对应问题三）多节点滚动调度。
106
107 update_nodes 控制"允许在哪些预报时刻更新购电策略"，是本题"是否需要引入其他时刻
108 预报"的消融变量：
109     S0 = (0,)          仅 0:00 预报
110     S1 = (0, 36)       +6:00
111     S2 = (0, 36, 72)   +12:00
112     S3 = (0, 36, 72, 108) +18:00
113 settlement_mode 决定 LP 目标函数本身：
114     replacement C = p[min(Gp,Ga) + 0.5(Gp-Ga)^+ + 1.5(Ga-Gp)^+]
115     additive    C = p·Gp + 0.5p(Gp-Ga)^+ + 1.5p(Ga-Gp)^+
116 两种口径**分别完整重新优化**，不是同一调度方案的事后换算式计费。
117 """
118 if settlement_mode not in ("replacement", "additive"):
119     raise ValueError(f"unknown settlement_mode: {settlement_mode}")
120 if load_mode not in ("perfect", "causal") or price_mode not in ("perfect", "causal"):
121     raise ValueError("unknown information mode")
122 nodes = tuple(sorted(int(n) for n in update_nodes))
123 if not nodes or nodes[0] != 0 or any(n not in ALL_NODES for n in nodes):
124     raise ValueError(f"update_nodes must be a prefix of {ALL_NODES}, got {update_nodes}"
125 )
126
127 typical = read_attachment1(data_root)
128 annual = read_attachment2(data_root)
129 forecasts = read_attachment3(data_root)
130 if annual.dates != forecasts.dates:
131     raise ValueError("attachment2/3 dates do not align")
132 if not 1 <= max_days <= 365:
133     raise ValueError("max_days must be 1..365")
134 prices = np.tile(typical.price, (365, 1)) if price_matrix is None else np.asarray(
135     price_matrix, dtype=float)
136 if prices.shape != (365, 144):
137     raise ValueError("price matrix must have shape (365,144)")
138 out_dir = OUTPUT if output_dir is None else Path(output_dir)
139
140 state = 6000.0
141 records: list[dict] = []
142 verify: list[tuple] = []
143 load_rows: list[dict] = []
144 price_rows: list[dict] = []
145 total_reversals = 0
146 calibrator = OnlineForecastBiasCalibrator()
147 load_forecaster = OnlineLoadForecaster(prior_kw=typical.load_kw if use_typical_prior
148     else None)
149 price_forecaster = OnlinePriceForecaster(prior_price=typical.price if use_typical_prior
150     else None)

```

```

148     for d in range(max_days):
149         initial = state
150         node_state = initial
151         soc_terminal = initial if terminal_policy == "daily_cycle" else None
152
153         # ---- 0:00 基准计划 -----
154         load_decision0 = load_forecaster.predict(d, 0, 144)
155         price_decision0 = price_forecaster.predict_horizon(d, 0, 144) if price_mode == "
            causal" else None
156         load0 = annual.load_kw[d] if load_mode == "perfect" else load_decision0.forecast_kw
157         price0 = prices[d] if price_mode == "perfect" else price_decision0.forecast_price
158         raw_base = hourly_to_ten_min(forecasts.hourly_kw[d, 0], downscale)
159         calibration0 = calibrator.calibrate(0, raw_base)
160         base_pv = calibration0.forecast_kw if calibrate else raw_base
161         baseline = solve_dispatch(
162             load0, base_pv, price0, soc_initial=initial, soc_terminal=soc_terminal, storage=
                storage
163         )
164
165         adjusted = np.zeros(144)
166         charge = np.zeros(144)
167         discharge = np.zeros(144)
168         soc = np.zeros(144)
169         curtail = np.zeros(144)
170         emergency = np.zeros(144)
171         inc_acc = np.zeros(144)
172         dec_acc = np.zeros(144)
173         revisions = np.full((4, 144), np.nan)
174         revisions[0] = baseline.grid
175         block_forecast_mae: list[float] = []
176         calibration_margins: list[float] = []
177         calibration_quantiles: list = []
178         load_plan_day = np.asarray(load0, dtype=float).copy()
179
180         load_rows.append(load_forecaster.log_row(load_decision0, annual.load_kw[d, :36],
            annual.dates[d].isoformat()))
181         if price_decision0 is not None:
182             price_rows.append(price_forecaster.log_row(price_decision0, prices[d, :36],
                annual.dates[d].isoformat()))
183         node_load_decisions: dict[int, object] = {0: load_decision0}
184
185         for node_index, node in enumerate(ALL_NODES):
186             block_stop = node + BLOCK
187             if node_index == 0:
188                 calibration = calibration0
189                 block_plan = _slice_dispatch(baseline, 0, BLOCK)
190                 block_forecast = base_pv[0:BLOCK]

```

```

191         block_inc = np.zeros(BLOCK)
192         block_dec = np.zeros(BLOCK)
193     elif node in nodes:
194         raw_horizon = hourly_to_ten_min(forecasts.hourly_kw[d, node_index],
195                                         downscale)
196         calibration = calibrator.calibrate(node_index, raw_horizon)
197         forecast_horizon = calibration.forecast_kw if calibrate else raw_horizon
198         if load_mode == "perfect":
199             load_horizon = _horizon(annual.load_kw, d, node, typical.load_kw)
200         else:
201             load_decision = load_forecaster.predict(d, node, 144)
202             load_horizon = load_decision.forecast_kw
203             load_plan_day[node:] = load_horizon[: 144 - node]
204             node_load_decisions[node] = load_decision
205             load_rows.append(
206                 load_forecaster.log_row(load_decision, annual.load_kw[d, node:
207                                         block_stop], annual.dates[d].isoformat())
208             )
209         if price_mode == "perfect":
210             price_horizon = _horizon(prices, d, node, typical.price)
211         else:
212             price_decision = price_forecaster.predict_horizon(d, node, 144)
213             price_horizon = price_decision.forecast_price
214             price_rows.append(
215                 price_forecaster.log_row(price_decision, prices[d, node:block_stop],
216                                         annual.dates[d].isoformat())
217             )
218         baseline_horizon = np.r_[baseline.grid[node:], np.zeros(node)]
219         mask = np.r_[np.ones(144 - node, dtype=bool), np.zeros(node, dtype=bool)]
220         rolling = solve_adjustment_dispatch(
221             load_horizon,
222             forecast_horizon,
223             price_horizon,
224             baseline_horizon,
225             mask,
226             soc_initial=node_state,
227             soc_terminal=None if terminal_policy == "free" else node_state,
228             settlement_mode=settlement_mode,
229             storage=storage,
230         )
231         horizon_plan = rolling.dispatch
232         block_plan = _slice_dispatch(horizon_plan, 0, BLOCK)
233         revisions[node_index, node:] = horizon_plan.grid[: 144 - node]
234         block_forecast = forecast_horizon[:BLOCK]
235         block_inc = rolling.increase[:BLOCK]
236         block_dec = rolling.decrease[:BLOCK]
237     else:

```

```

235         # 该时刻没有新预报，沿用 0:00 基准计划，不产生调整
236         block_plan = _slice_dispatch(baseline, node, block_stop)
237         block_forecast = base_pv[node:block_stop]
238         block_inc = np.zeros(BLOCK)
239         block_dec = np.zeros(BLOCK)
240
241         execution = execute_plan(
242             block_plan,
243             annual.load_kw[d, node:block_stop],
244             annual.pv_kw[d, node:block_stop],
245             soc_initial=node_state,
246             realtime_feedback=realtime_feedback,
247             storage=storage,
248         )
249         adjusted[node:block_stop] = execution.grid
250         charge[node:block_stop] = execution.charge
251         discharge[node:block_stop] = execution.discharge
252         soc[node:block_stop] = execution.soc
253         curtail[node:block_stop] = execution.curtail
254         emergency[node:block_stop] = execution.emergency
255         inc_acc[node:block_stop] = block_inc
256         dec_acc[node:block_stop] = block_dec
257
258         if node_index == 0 or node in nodes:
259             calibrator.observe(calibration, annual.pv_kw[d, node:block_stop], prices[d,
260                                     node:block_stop])
261             calibration_margins.append(calibration.margin_kw if calibrate else 0.0)
262             calibration_quantiles.append(calibration.quantile if calibrate else None)
263             block_forecast_mae.append(float(np.mean(np.abs(block_forecast - annual.pv_kw[d,
264                                     node:block_stop]))))
265             node_state = float(execution.soc[-1])
266
267         # 供后续节点使用的已观测信息（严格在决策与执行之后）
268         if load_mode == "causal" and node in node_load_decisions:
269             load_forecaster.observe_block(node_load_decisions[node], annual.load_kw[d,
270                                     node:block_stop])
271             load_forecaster.observe_partial(block_stop, annual.load_kw[d, :block_stop])
272
273         state = node_state
274         actual = ExecutionResult(adjusted.copy(), charge, discharge, soc, curtail, emergency
275                                 )
276
277         gp = baseline.grid
278         ga = adjusted
279         increase = np.maximum(ga - gp, 0.0)
280         decrease = np.maximum(gp - ga, 0.0)
281         settlement = settlement_cost(gp, ga, prices[d], settlement_mode)

```

```

278     if settlement_mode == "replacement":
279         lp_side = float(np.dot(prices[d], ga + 0.5 * inc_acc + 0.5 * dec_acc))
280     else:
281         lp_side = float(np.dot(prices[d], gp) + np.dot(prices[d], 0.5 * dec_acc + 1.5 *
                inc_acc))
282     emergency_cost = float(np.dot(5 * prices[d], emergency))
283     reversals = _reversal_count(revisions)
284     total_reversals += reversals
285
286     record = {
287         "day_index": d,
288         "date": annual.dates[d],
289         "soc_initial": initial,
290         "baseline_plan": baseline,
291         "adjusted_grid": adjusted,
292         "execution": actual,
293         "load_kw": annual.load_kw[d],
294         "actual_pv_kw": annual.pv_kw[d],
295         "load_plan_kw": load_plan_day,
296         "revisions": revisions,
297         "baseline_cost": float(np.dot(prices[d], gp)),
298         "settlement_cost": settlement,
299         "emergency_cost": emergency_cost,
300         "total_cost": settlement + emergency_cost,
301         "adjustment_cost_yuan": settlement - float(np.dot(prices[d], gp)) if
                settlement_mode == "additive" else settlement - float(np.dot(prices[d], gp))
                ,
302         "linearization_gap": abs(settlement - lp_side),
303         "up_energy": float(increase.sum()),
304         "down_energy": float(decrease.sum()),
305         "reversal_slots": reversals,
306         "forecast_mae": float(np.mean(block_forecast_mae)),
307         "calibration_margins": calibration_margins,
308         "calibration_quantiles": calibration_quantiles,
309     }
310     records.append(record)
311
312     balance = ga + annual.pv_kw[d] * DT_HOURS + discharge + emergency - annual.load_kw[d]
                ] * DT_HOURS - charge - curtail
313     previous = np.r_[initial, soc[:-1]]
314     soc_res = soc - (previous + storage.eta_ch * charge - discharge / storage.eta_dis)
315     verify.append(
316         (
317             float(np.max(np.abs(balance))),
318             float(np.max(np.abs(soc_res))),
319             float(soc.min()),
320             float(soc.max()),

```



```

321         float(np.max(np.minimum(charge, discharge))),
322     )
323 )
324
325 # ---- 日终观测回填 -----
326 if load_mode == "causal":
327     load_forecaster.commit_day(annual.load_kw[d], d)
328 if price_mode == "causal":
329     price_forecaster.commit_day(prices[d], d)
330 else:
331     price_forecaster.commit_day(prices[d], d)
332
333 delivery = records[31:] if max_days == 365 else []
334 metrics = {
335     "case_tag": tag,
336     "days_simulated": max_days,
337     "warmup_days": 31 if max_days == 365 else None,
338     "delivery_days": len(delivery),
339     "downscale_method": downscale,
340     "calibration_enabled": calibrate,
341     "update_nodes": list(nodes),
342     "update_nodes_label": "S" + str(len(nodes) - 1),
343     "settlement_mode": settlement_mode,
344     "load_mode": load_mode,
345     "price_mode": price_mode,
346     "realtime_feedback": bool(realtime_feedback),
347     "terminal_policy": terminal_policy,
348     "storage": storage.as_dict(),
349     "price_source": "attachment1" if price_matrix is None else "attachment4",
350     "load_information": (
351         "Attachment 2 same-day true load curve (perfect-information benchmark)"
352         if load_mode == "perfect"
353         else "Online causal forecast from days strictly before d"
354     ),
355     "pv_information": (
356         "Attachment 3 rolling forecasts with causal issue-specific calibration"
357         if calibrate
358         else "raw Attachment 3 forecasts"
359     )
360 + "; actual PV used only after block execution",
361     "price_information": (
362         "Attachment 4 same-day true price curve (perfect-information benchmark)"
363         if price_mode == "perfect"
364         else "Online causal price forecast from days strictly before d"
365     ),
366     "max_balance_residual": max(v[0] for v in verify),
367     "max_soc_residual": max(v[1] for v in verify),

```

```

368     "min_soc_kwh": min(v[2] for v in verify),
369     "max_soc_kwh": max(v[3] for v in verify),
370     "max_simultaneous_charge_discharge": max(v[4] for v in verify),
371     "replacement_settlement_formula": "min(Gp,Ga) + 0.5*(Gp-Ga)^+ + 1.5*(Ga-Gp)^+",
372     "additive_settlement_formula": "Gp + 0.5*(Gp-Ga)^+ + 1.5*(Ga-Gp)^+",
373     "cross_midnight": "24h optimize, execute current 6h block only; carry SOC only",
374 }
375 if delivery:
376     metrics.update(
377         {
378             "total_cost_yuan": float(sum(r["total_cost"] for r in delivery)),
379             "baseline_purchase_cost_yuan": float(sum(r["baseline_cost"] for r in
380                 delivery)),
381             "adjustment_settlement_cost_yuan": float(sum(r["settlement_cost"] for r in
382                 delivery)),
383             "emergency_cost_yuan": float(sum(r["emergency_cost"] for r in delivery)),
384             "plan_purchase_energy_kwh": float(sum(r["baseline_plan"].grid.sum() for r in
385                 delivery)),
386             "adjusted_purchase_energy_kwh": float(sum(r["adjusted_grid"].sum() for r in
387                 delivery)),
388             "emergency_energy_kwh": float(sum(r["execution"].emergency.sum() for r in
389                 delivery)),
390             "curtailment_energy_kwh": float(sum(r["execution"].curtail.sum() for r in
391                 delivery)),
392             "up_adjustment_energy_kwh": float(sum(r["up_energy"] for r in delivery)),
393             "down_adjustment_energy_kwh": float(sum(r["down_energy"] for r in delivery))
394             ,
395             "days_with_emergency": int(sum(r["execution"].emergency.sum() > 1e-7 for r
396                 in delivery)),
397             "emergency_interval_count": int(sum(_emergency_intervals(r["execution"].
398                 emergency) for r in delivery)),
399             "storage_charge_energy_kwh": float(sum(r["execution"].charge.sum() for r in
400                 delivery)),
401             "storage_discharge_energy_kwh": float(sum(r["execution"].discharge.sum() for
402                 r in delivery)),
403             "storage_throughput_kwh": float(
404                 sum(r["execution"].charge.sum() + r["execution"].discharge.sum() for r
405                     in delivery)
406             ),
407             "ending_soc_kwh": float(delivery[-1]["execution"].soc[-1]),
408             "direction_reversal_slots": int(sum(r["reversal_slots"] for r in delivery)),
409             "max_linearization_gap": float(max(r["linearization_gap"] for r in delivery)
410                 ),
411             "executed_block_forecast_mae_kw": float(np.mean([r["forecast_mae"] for r in
412                 delivery])),
413             "load_forecast_mae_kw": float(np.mean([row["mae_kw"] for row in load_rows
414                 [31:]])) if load_rows else None,

```

```

400         "load_forecast_rmse_kw": float(np.mean([row["rmse_kw"] for row in load_rows
401           [31:]])) if load_rows else None,
402         "price_forecast_mae": float(np.mean([row["price_mae"] for row in price_rows
403           [31:]])) if price_rows else None,
404         "price_forecast_rmse": float(np.mean([row["price_rmse"] for row in
405           price_rows[31:]])) if price_rows else None,
406         "runtime_seconds": float(
407             sum(r["baseline_plan"].solve_seconds for r in delivery)
408         ),
409     }
410 )
411 metrics["passed"] = bool(
412     metrics["max_balance_residual"] < 1e-7
413     and metrics["max_soc_residual"] < 1e-7
414     and metrics["min_soc_kwh"] >= storage.soc_min - 1e-7
415     and metrics["max_soc_kwh"] <= storage.soc_max + 1e-7
416     and metrics["max_simultaneous_charge_discharge"] < 1e-5
417     and (not delivery or metrics["max_linearization_gap"] < 1e-6)
418 )
419
420 if write_outputs:
421     out_dir.mkdir(parents=True, exist_ok=True)
422     (out_dir / "metrics.json").write_text(json.dumps(metrics, ensure_ascii=False, indent
423       =2), encoding="utf-8")
424     _write_csv(out_dir / "load_forecast_log.csv", load_rows)
425     if price_rows:
426         _write_csv(out_dir / "price_forecast_log.csv", price_rows)
427     if delivery:
428         fields = [
429             "date", "slot", "load_kw", "actual_pv_kwh", "load_plan_kw", "price", "
430               baseline_grid_kwh",
431             "adjusted_grid_kwh", "charge_actual_kwh", "discharge_actual_kwh", "
432               soc_actual_kwh",
433             "curtail_actual_kwh", "emergency_kwh", "revision_0", "revision_6", "
434               revision_12", "revision_18",
435         ]
436         with (out_dir / "solution.csv").open("w", newline="", encoding="utf-8-sig") as f:
437             :
438             import csv as _csv
439
440             writer = _csv.DictWriter(f, fieldnames=fields)
441             writer.writeheader()
442             for r in delivery:
443                 for t in range(144):
444                     rev = r["revisions"][t, :]
445                     writer.writerow(
446                         {

```

```

439         "date": r["date"].isoformat(),
440         "slot": t + 1,
441         "load_kw": r["load_kw"][t],
442         "actual_pv_kw": r["actual_pv_kw"][t],
443         "load_plan_kw": r["load_plan_kw"][t],
444         "price": prices[r["day_index"], t],
445         "baseline_grid_kwh": r["baseline_plan"].grid[t],
446         "adjusted_grid_kwh": r["adjusted_grid"][t],
447         "charge_actual_kwh": r["execution"].charge[t],
448         "discharge_actual_kwh": r["execution"].discharge[t],
449         "soc_actual_kwh": r["execution"].soc[t],
450         "curtail_actual_kwh": r["execution"].curtail[t],
451         "emergency_kwh": r["execution"].emergency[t],
452         "revision_0": rev[0],
453         "revision_6": rev[1],
454         "revision_12": rev[2],
455         "revision_18": rev[3],
456     }
457 )
458 fields_d = [
459     "date", "soc_initial", "soc_terminal", "baseline_cost", "settlement_cost", "
460     emergency_cost",
461     "total_cost", "up_energy", "down_energy", "reversal_slots", "forecast_mae",
462 ]
463 _write_csv(
464     out_dir / "daily_metrics.csv",
465     [
466         {
467             "date": r["date"].isoformat(),
468             "soc_initial": r["soc_initial"],
469             "soc_terminal": r["execution"].soc[-1],
470             **{k: r[k] for k in fields_d[3:]},
471         }
472         for r in delivery
473     ],
474     fields_d,
475 )
476 check = export_result3(template_path(template_name, data_root), out_dir /
477     template_name, delivery)
478 (out_dir / "xlsx_verification.json").write_text(json.dumps(check, ensure_ascii=
479     False, indent=2), encoding="utf-8")
480 metrics["xlsx_passed"] = check["passed"]
481 (out_dir / "metrics.json").write_text(json.dumps(metrics, ensure_ascii=False,
482     indent=2), encoding="utf-8")
483 env = {
484     "python": sys.version,
485     "platform": platform.platform(),

```

```

482         "numpy": np.__version__,
483         "scipy": scipy.__version__,
484         "algorithm": f"HiGHS LP, {settlement_mode} settlement with explicit (inc, dec)
            split",
485         "case_tag": tag,
486     }
487     (out_dir / "environment.json").write_text(json.dumps(env, ensure_ascii=False, indent
            =2), encoding="utf-8")
488     return {"metrics": metrics, "records": records}
489
490
491 if __name__ == "__main__":
492     print(json.dumps(run_q3()["metrics"], ensure_ascii=False, indent=2))

```

B.11 问题四求解 (q4.py)

Listing 11:

```

1  from __future__ import annotations
2
3  import json
4  from pathlib import Path
5
6  from .io_data import DEFAULT_DATA_ROOT, read_attachment4
7  from .q2 import run_q2
8  from .q3 import run_q3
9
10 ROOT = Path(__file__).resolve().parents[1]
11 OUTPUT = ROOT / "outputs"
12
13
14 def run_q4(
15     *,
16     data_root: Path = DEFAULT_DATA_ROOT,
17     max_days: int = 365,
18     price_mode_q2: str = "causal",
19     price_mode_q3: str = "causal",
20     load_mode: str = "causal",
21     settlement_mode: str = "replacement",
22     update_nodes: tuple[int, ...] = (0, 36, 72, 108),
23     realtime_feedback: bool = True,
24     write_outputs: bool = True,
25     tag_prefix: str = "",
26 ) -> dict:
27     """问题四：把固定典型日电价替换为附件 4 的波动电价，分别重算问题二、问题三。
28

```

```

29 price_mode = "perfect" 优化时已知当日真实电价 (perfect_price benchmark);
30     = "causal" 只用决策时刻之前已发生的电价滚动预测未来电价。
31 """
32 annual_price = read_attachment4(data_root)
33 prices = annual_price.price
34
35 suffix = f"--{tag_prefix}" if tag_prefix else ""
36 out2 = OUTPUT / f"q4-2{suffix}"
37 out3 = OUTPUT / f"q4-3{suffix}"
38 q4_2 = run_q2(
39     data_root=data_root,
40     max_days=max_days,
41     price_matrix=prices,
42     output_dir=out2,
43     template_name="result4-2.xlsx",
44     write_outputs=write_outputs,
45     load_mode=load_mode,
46     price_mode=price_mode_q2,
47     realtime_feedback=realtime_feedback,
48     tag=f"q4-2 {tag_prefix}".strip(),
49 )
50 q4_3 = run_q3(
51     data_root=data_root,
52     max_days=max_days,
53     price_matrix=prices,
54     output_dir=out3,
55     template_name="result4-3.xlsx",
56     write_outputs=write_outputs,
57     load_mode=load_mode,
58     price_mode=price_mode_q3,
59     settlement_mode=settlement_mode,
60     update_nodes=update_nodes,
61     realtime_feedback=realtime_feedback,
62     tag=f"q4-3 {tag_prefix}".strip(),
63 )
64
65 c2 = q4_2["metrics"].get("total_cost_yuan")
66 c3 = q4_3["metrics"].get("total_cost_yuan")
67 comparison = {
68     "q4_2_total_cost_yuan": c2,
69     "q4_3_total_cost_yuan": c3,
70     "rolling_saving_yuan": None if (c2 is None or c3 is None) else c2 - c3,
71     "rolling_saving_percent": None if (c2 is None or c3 is None) else 100 * (c2 - c3) /
72         c2,
73     "q4_2_emergency_energy_kwh": q4_2["metrics"].get("emergency_energy_kwh"),
74     "q4_3_emergency_energy_kwh": q4_3["metrics"].get("emergency_energy_kwh"),
75     "q4_2_price_mode": price_mode_q2,

```

```

75     "q4_3_price_mode": price_mode_q3,
76     "q4_2_price_mae": q4_2["metrics"].get("price_forecast_mae"),
77     "q4_3_price_mae": q4_3["metrics"].get("price_forecast_mae"),
78     "q4_2_passed": bool(q4_2["metrics"]["passed"]),
79     "q4_3_passed": bool(q4_3["metrics"]["passed"]),
80 }
81 comparison["passed"] = comparison["q4_2_passed"] and comparison["q4_3_passed"]
82 if write_outputs:
83     out2.mkdir(parents=True, exist_ok=True)
84     (out2 / "q4_comparison.json").write_text(
85         json.dumps(comparison, ensure_ascii=False, indent=2), encoding="utf-8"
86     )
87     return {"q4_2": q4_2, "q4_3": q4_3, "comparison": comparison}
88
89
90 if __name__ == "__main__":
91     print(json.dumps(run_q4()["comparison"], ensure_ascii=False, indent=2))

```

B.12 官方结果文件导出 (export.py)

Listing 12:

```

1  from __future__ import annotations
2  from copy import copy
3  from datetime import datetime, time
4  from pathlib import Path
5  from shutil import copy2
6  import numpy as np
7  import openpyxl
8  from .lp_core import DispatchResult
9
10 def export_result1(template: Path, destination: Path, result: DispatchResult) -> dict:
11     destination.parent.mkdir(parents=True, exist_ok=True); copy2(template, destination)
12     wb=openpyxl.load_workbook(destination); plan, storage=wb.worksheets[:2]
13     for row, value in enumerate(result.grid, start=2):
14         plan.cell(row, 2, float(value)); plan.cell(row, 2).number_format="0.0000"
15     for block in range(6):
16         lo, hi=block*24, (block+1)*24
17         storage.cell(block+2, 2, float(result.charge[lo:hi].sum()))
18         storage.cell(block+2, 3, float(result.discharge[lo:hi].sum()))
19     storage.cell(2, 5, 6000.0); storage.cell(3, 5, float(result.soc[-1])); wb.save(destination)
20     check=openpyxl.load_workbook(destination, data_only=True, read_only=True)
21     values=np.asarray([check.worksheets[0].cell(i, 2).value for i in range(2, 146)], dtype=
22                       float)
23     diff=float(np.max(np.abs(values-result.grid)))
24     return {"rows": len(values), "max_plan_roundtrip_diff": diff, "passed": bool(diff<1e-9)}

```

```

24
25 def _copy_row_style(ws,source_row:int,target_row:int,max_col:int)->None:
26     for col in range(1,max_col+1):
27         src=ws.cell(source_row,col); dst=ws.cell(target_row,col)
28         if src.has_style: dst._style=copy(src._style)
29         dst.alignment=copy(src.alignment); dst.protection=copy(src.protection)
30     ws.row_dimensions[target_row].height=ws.row_dimensions[source_row].height
31
32 def _segments(values:np.ndarray,tolerance:float=1e-7)->list[tuple[int,int,float]]:
33     result=[]; start=None
34     for idx,flag in enumerate(np.r_[np.asarray(values)>tolerance,False]):
35         if flag and start is None: start=idx
36         elif not flag and start is not None:
37             result.append((start,idx,float(np.sum(values[start:idx])))); start=None
38     return result
39
40 def _slot_time(slot:int)->str:
41     minutes=slot*10
42     return "24:00" if minutes==1440 else f"{minutes//60}:{minutes%60:02d}"
43
44 def export_result2(template:Path,destination:Path,days:list[dict])->dict:
45     if len(days)!=334: raise ValueError(f"result2 requires 334 days, got {len(days)}")
46     destination.parent.mkdir(parents=True,exist_ok=True); copy2(template,destination)
47     wb=openpyxl.load_workbook(destination); plan_ws,storage_ws,emergency_ws=wb.worksheets
48     [:3]
49     for idx,day in enumerate(days):
50         row=idx+2; plan=day["plan"]
51         plan_ws.cell(row,1,datetime.combine(day["date"],time()))
52         for slot,value in enumerate(plan.grid,start=2):
53             plan_ws.cell(row,slot,float(value)); plan_ws.cell(row,slot).number_format="
54                 0.0000"
55             plan_ws.cell(row,146,float(np.sum(plan.grid))); plan_ws.cell(row,147,float(day["
56                 plan_cost"]))
57             plan_ws.cell(row,146).number_format=plan_ws.cell(row,147).number_format="0.0000"
58     target=1+6*len(days)
59     if storage_ws.max_row<target: storage_ws.insert_rows(storage_ws.max_row+1,amount=target-
60         storage_ws.max_row)
61     periods=("0:00-4:00","4:00-8:00","8:00-12:00","12:00-16:00","16:00-20:00","20:00-24:00")
62     for idx,day in enumerate(days):
63         actual=day["execution"]
64         for block in range(6):
65             row=2+idx*6+block; _copy_row_style(storage_ws,2+block,row,6)
66             storage_ws.cell(row,1,datetime.combine(day["date"],time())) if block==0 else None
67             )
68             storage_ws.cell(row,2,periods[block]); lo,hi=block*24,(block+1)*24
69             storage_ws.cell(row,3,float(actual.charge[lo:hi].sum()))
70             storage_ws.cell(row,4,float(actual.discharge[lo:hi].sum()))

```



```

66         storage_ws.cell(row,5,time(0,0) if block==0 else ("24:00" if block==1 else None)
67         )
68         storage_ws.cell(row,6,float(day["soc_initial"]) if block==0 else (float(actual.
69         soc[-1]) if block==1 else None))
70         for col in (3,4,6): storage_ws.cell(row,col).number_format="0.0000"
71     if storage_ws.max_row>target: storage_ws.delete_rows(target+1,storage_ws.max_row-target)
72     rows=[]
73     for day in days:
74         segments=_segments(day["execution"].emergency)
75         if not segments: rows.append((datetime.combine(day["date"],time()),"无",0.0))
76         else:
77             for k,(start,end,total) in enumerate(segments):
78                 rows.append((datetime.combine(day["date"],time()) if k==0 else None,
79                 f"{_slot_time(start)}-{_slot_time(end)}",total))
80     target_e=1+len(rows)
81     if emergency_ws.max_row<target_e: emergency_ws.insert_rows(emergency_ws.max_row+1,amount
82     =target_e-emergency_ws.max_row)
83     for row,(date_value,period,total) in enumerate(rows,start=2):
84         _copy_row_style(emergency_ws,2,row,3)
85         emergency_ws.cell(row,1,date_value); emergency_ws.cell(row,2,period)
86         emergency_ws.cell(row,3,float(total)); emergency_ws.cell(row,3).number_format="
87         0.0000"
88     if emergency_ws.max_row>target_e: emergency_ws.delete_rows(target_e+1,emergency_ws.
89     max_row-target_e)
90     wb.save(destination)
91     check=openpyxl.load_workbook(destination,data_only=True,read_only=True)
92     plan_rows=list(check.worksheets[0].iter_rows(min_row=2,max_row=335,min_col=2,max_col
93     =145,values_only=True))
94     values=np.asarray(plan_rows,dtype=float)
95     expected=np.stack([d["plan"].grid for d in days]); diff=float(np.max(np.abs(values-
96     expected)))
97     last_row=3+6*(len(days)-1)
98     last_soc=float(next(check.worksheets[1].iter_rows(min_row=last_row,max_row=last_row,
99     min_col=6,max_col=6,values_only=True))[0])
100     return {"delivery_days":334,"plan_rows":values.shape[0],"plan_slots":values.shape[1],
101     "storage_rows":check.worksheets[1].max_row-1,"emergency_rows":check.worksheets
102     [2].max_row-1,
103     "max_plan_roundtrip_diff":diff,"last_terminal_soc":last_soc,
104     "passed":bool(values.shape==(334,144) and diff<1e-9 and check.worksheets[1].
105     max_row-1==2004)}
106
107 def export_result3(template:Path,destination:Path,days:list[dict])>->dict:
108     if len(days)!=334: raise ValueError(f"result3 requires 334 days, got {len(days)}")
109     destination.parent.mkdir(parents=True,exist_ok=True); copy2(template,destination)
110     wb=openpyxl.load_workbook(destination); plan_ws,adjust_ws,storage_ws,emergency_ws=wb.
111     worksheets[:4]

```

```

102     for idx,day in enumerate(days):
103         row=idx+2; plan=day["baseline_plan"]; adjusted=day["adjusted_grid"]
104         for ws,values,cost in ((plan_ws,plan.grid,day["baseline_cost"]), (adjust_ws,adjusted,
105             day["settlement_cost"])):
106             ws.cell(row,1,datetime.combine(day["date"],time()))
107             for slot,value in enumerate(values,start=2):
108                 ws.cell(row,slot,float(value)); ws.cell(row,slot).number_format="0.0000"
109             ws.cell(row,146,float(np.sum(values))); ws.cell(row,147,float(cost))
110             ws.cell(row,146).number_format=ws.cell(row,147).number_format="0.0000"
111 target=1+6*len(days)
112 if storage_ws.max_row<target: storage_ws.insert_rows(storage_ws.max_row+1,amount=target-
113     storage_ws.max_row)
114 periods=("0:00-4:00","4:00-8:00","8:00-12:00","12:00-16:00","16:00-20:00","20:00-24:00")
115 for idx,day in enumerate(days):
116     actual=day["execution"]
117     for block in range(6):
118         row=2+idx*6+block; _copy_row_style(storage_ws,2+block,row,6)
119         storage_ws.cell(row,1,datetime.combine(day["date"],time()) if block==0 else None
120             )
121         storage_ws.cell(row,2,periods[block]); lo,hi=block*24,(block+1)*24
122         storage_ws.cell(row,3,float(actual.charge[lo:hi].sum()))
123         storage_ws.cell(row,4,float(actual.discharge[lo:hi].sum()))
124         storage_ws.cell(row,5,time(0,0) if block==0 else ("24:00" if block==1 else None)
125             )
126         storage_ws.cell(row,6,float(day["soc_initial"]) if block==0 else (float(actual.
127             soc[-1]) if block==1 else None))
128         for col in (3,4,6): storage_ws.cell(row,col).number_format="0.0000"
129 if storage_ws.max_row>target: storage_ws.delete_rows(target+1,storage_ws.max_row-target)
130 rows=[]
131 for day in days:
132     segments=_segments(day["execution"].emergency)
133     if not segments: rows.append((datetime.combine(day["date"],time()),"无",0.0))
134     else:
135         for k,(start,end,total) in enumerate(segments):
136             rows.append((datetime.combine(day["date"],time()) if k==0 else None,
137                 f"{_slot_time(start)}-{_slot_time(end)}",total))
138 target_e=1+len(rows)
139 if emergency_ws.max_row<target_e: emergency_ws.insert_rows(emergency_ws.max_row+1,amount
140     =target_e-emergency_ws.max_row)
141 for row,(date_value,period,total) in enumerate(rows,start=2):
142     _copy_row_style(emergency_ws,2,row,3); emergency_ws.cell(row,1,date_value)
143     emergency_ws.cell(row,2,period); emergency_ws.cell(row,3,float(total))
144     emergency_ws.cell(row,3).number_format="0.0000"
145 if emergency_ws.max_row>target_e: emergency_ws.delete_rows(target_e+1,emergency_ws.
146     max_row-target_e)
147 wb.save(destination)
148 check=openpyxl.load_workbook(destination,data_only=True,read_only=True)

```

```

142 plan_values=np.asarray(list(check.worksheets[0].iter_rows(min_row=2,max_row=335,min_col
    =2,max_col=145,values_only=True)),dtype=float)
143 adjust_values=np.asarray(list(check.worksheets[1].iter_rows(min_row=2,max_row=335,
    min_col=2,max_col=145,values_only=True)),dtype=float)
144 expected_plan=np.stack([d["baseline_plan"].grid for d in days]); expected_adjust=np.
    stack([d["adjusted_grid"] for d in days])
145 return {"delivery_days":334,"plan_shape":list(plan_values.shape),"adjust_shape":list(
    adjust_values.shape),
146         "storage_rows":check.worksheets[2].max_row-1,"emergency_rows":check.worksheets
    [3].max_row-1,
147         "max_plan_roundtrip_diff":float(np.max(np.abs(plan_values-expected_plan))),
148         "max_adjust_roundtrip_diff":float(np.max(np.abs(adjust_values-expected_adjust)))
    ,
149         "passed":bool(plan_values.shape==(334,144) and adjust_values.shape==(334,144)
    and
150                             check.worksheets[2].max_row-1==2004 and
151                             np.max(np.abs(plan_values-expected_plan))<1e-9 and np.max(np.abs(
    adjust_values-expected_adjust))<1e-9)}

```

B.13 模型审计与消融实验总控 (ablation.py)

Listing 13:

```

1  """模型审计实验总控: Q2 原因拆解 / Q3 预报时点消融与结算口径 / Q4 价格信息边界 / P1 敏感性。
2
3  所有变体产物统一写入 MathModel/outputs/variants/model_audit_v2/, 不覆盖任何既有结果。
4  """
5  from __future__ import annotations
6
7  import json
8  import time
9  from pathlib import Path
10
11 import numpy as np
12
13 from .io_data import DEFAULT_DATA_ROOT, read_attachment1, read_attachment4
14 from .params import DEFAULT_STORAGE, EFFICIENCY_VARIANTS
15 from .q2 import run_q2
16 from .q3 import run_q3
17
18 ROOT = Path(__file__).resolve().parents[1]
19 VARIANTS = ROOT / "outputs" / "variants" / "model_audit_v2"
20
21 # Q2 原因拆解: 只改变一个因素
22 Q2_CASES = {
23     "A_perfectload_risk_feedback": dict(load_mode="perfect", pv_risk_mode="asymmetric",

```

```

        realtime_feedback=True),
24     "B_causalload_risk_feedback": dict(load_mode="causal", pv_risk_mode="asymmetric",
        realtime_feedback=True),
25     "C_causalload_point_feedback": dict(load_mode="causal", pv_risk_mode="point",
        realtime_feedback=True),
26     "D_causalload_risk_nofeedback": dict(load_mode="causal", pv_risk_mode="asymmetric",
        realtime_feedback=False),
27 }
28
29 Q3_CASES = {
30     "S0_0": dict(update_nodes=(0,)),
31     "S1_0_6": dict(update_nodes=(0, 36)),
32     "S2_0_6_12": dict(update_nodes=(0, 36, 72)),
33     "S3_0_6_12_18": dict(update_nodes=(0, 36, 72, 108)),
34 }
35
36 Q4_CASES = {
37     "q42_perfect_price": ("q2", dict(price_mode="perfect")),
38     "q42_causal_price": ("q2", dict(price_mode="causal")),
39     "q43_perfect_price": ("q3", dict(price_mode="perfect")),
40     "q43_causal_price": ("q3", dict(price_mode="causal")),
41 }
42
43 SEASONS = {
44     "spring": (3, 4, 5),
45     "summer": (6, 7, 8),
46     "autumn": (9, 10, 11),
47     "winter": (12, 1, 2),
48 }
49
50
51 def _season(month: int) -> str:
52     for name, months in SEASONS.items():
53         if month in months:
54             return name
55     raise ValueError(month)
56
57
58 def run_q2_ablation(*, max_days: int = 365, data_root: Path = DEFAULT_DATA_ROOT) -> dict:
59     out = VARIANTS / "q2"
60     summary = {}
61     for name, kwargs in Q2_CASES.items():
62         started = time.perf_counter()
63         result = run_q2(
64             data_root=data_root,
65             max_days=max_days,
66             output_dir=out / name,

```

```

67         template_name="result2.xlsx",
68         write_outputs=True,
69         tag=name,
70         **kwargs,
71     )
72     metrics = result["metrics"]
73     metrics["wall_seconds"] = time.perf_counter() - started
74     (out / name / "metrics.json").write_text(
75         json.dumps(metrics, ensure_ascii=False, indent=2), encoding="utf-8"
76     )
77     summary[name] = metrics
78     print(f"[Q2] {name}: total={metrics.get('total_cost_yuan')} emergency={metrics.get('emergency_energy_kwh')}")
79     _dump(out / "summary.json", summary)
80     return summary
81
82
83 def run_q3_ablation(
84     *, max_days: int = 365, data_root: Path = DEFAULT_DATA_ROOT, settlement_mode: str = "replacement"
85 ) -> dict:
86     # 两种结算口径的输出必须分目录，否则后跑的一轮会覆盖前一轮的
87     # metrics.json 与官方 result3.xlsx，导致论文与提交文件口径不一致。
88     out = VARIANTS / f"q3_{settlement_mode}"
89     summary = {}
90     for name, kwargs in Q3_CASES.items():
91         started = time.perf_counter()
92         result = run_q3(
93             data_root=data_root,
94             max_days=max_days,
95             output_dir=out / name,
96             template_name="result3.xlsx",
97             write_outputs=True,
98             settlement_mode=settlement_mode,
99             tag=name,
100             **kwargs,
101         )
102         metrics = result["metrics"]
103         metrics["wall_seconds"] = time.perf_counter() - started
104         (out / name / "metrics.json").write_text(
105             json.dumps(metrics, ensure_ascii=False, indent=2), encoding="utf-8"
106         )
107         # 季节分解
108         (out / name / "by_season.csv").write_text(_season_table(result["records"]), encoding="utf-8-sig")
109         summary[name] = metrics
110         print(f"[Q3-{settlement_mode}] {name}: total={metrics.get('total_cost_yuan')}")

```

```

111     _dump(out / "summary.json", summary)
112     return summary
113
114
115 def _season_table(records: list[dict]) -> str:
116     delivery = records[31:]
117     rows = ["season,days,total_cost_yuan,baseline_cost_yuan,emergency_energy_kwh,
118             up_energy_kwh,down_energy_kwh,curtailment_kwh"]
119     for season in ("spring", "summer", "autumn", "winter"):
120         subset = [r for r in delivery if _season(r["date"].month) == season]
121         if not subset:
122             continue
123         rows.append(
124             ", ".join(
125                 str(x)
126                 for x in [
127                     season,
128                     len(subset),
129                     round(sum(r["total_cost"] for r in subset), 4),
130                     round(sum(r["baseline_cost"] for r in subset), 4),
131                     round(sum(r["execution"].emergency.sum() for r in subset), 4),
132                     round(sum(r["up_energy"] for r in subset), 4),
133                     round(sum(r["down_energy"] for r in subset), 4),
134                     round(sum(r["execution"].curtail.sum() for r in subset), 4),
135                 ]
136             )
137         )
138     return "\n".join(rows) + "\n"
139
140 def run_q4_ablation(*, max_days: int = 365, data_root: Path = DEFAULT_DATA_ROOT) -> dict:
141     out = VARIANTS / "q4"
142     prices = read_attachment4(data_root).price
143     summary = {}
144     for name, (kind, kwargs) in Q4_CASES.items():
145         started = time.perf_counter()
146         if kind == "q2":
147             result = run_q2(
148                 data_root=data_root,
149                 max_days=max_days,
150                 price_matrix=prices,
151                 output_dir=out / name,
152                 template_name="result4-2.xlsx",
153                 write_outputs=True,
154                 load_mode="causal",
155                 tag=name,
156                 **kwargs,

```

```

157         )
158     else:
159         result = run_q3(
160             data_root=data_root,
161             max_days=max_days,
162             price_matrix=prices,
163             output_dir=out / name,
164             template_name="result4-3.xlsx",
165             write_outputs=True,
166             load_mode="causal",
167             settlement_mode="replacement",
168             update_nodes=(0, 36, 72, 108),
169             tag=name,
170             **kwargs,
171         )
172     metrics = result["metrics"]
173     metrics["wall_seconds"] = time.perf_counter() - started
174     (out / name / "metrics.json").write_text(
175         json.dumps(metrics, ensure_ascii=False, indent=2), encoding="utf-8"
176     )
177     summary[name] = metrics
178     print(f"[Q4] {name}: total={metrics.get('total_cost_yuan')} emergency={metrics.get('emergency_energy_kwh')}")
179     _dump(out / "summary.json", summary)
180     return summary
181
182
183 def run_sensitivity(*, max_days: int = 365, data_root: Path = DEFAULT_DATA_ROOT) -> dict:
184     out = VARIANTS / "sensitivity"
185     summary = {"terminal": {}, "efficiency": {}}
186     for policy in ("daily_cycle", "free"):
187         started = time.perf_counter()
188         result = run_q2(
189             data_root=data_root,
190             max_days=max_days,
191             output_dir=out / "terminal" / policy,
192             template_name="result2.xlsx",
193             write_outputs=True,
194             load_mode="causal",
195             pv_risk_mode="asymmetric",
196             realtime_feedback=True,
197             terminal_policy=policy,
198             tag=f"terminal_{policy}",
199         )
200     metrics = result["metrics"]
201     metrics["wall_seconds"] = time.perf_counter() - started
202     daily_soc = [float(r["execution"].soc[-1]) for r in result["records"][31:]]

```

```

203     metrics["daily_boundary_soc"] = {
204         "min": float(np.min(daily_soc)),
205         "max": float(np.max(daily_soc)),
206         "mean": float(np.mean(daily_soc)),
207         "std": float(np.std(daily_soc)),
208     }
209     (out / "terminal" / policy / "metrics.json").write_text(
210         json.dumps(metrics, ensure_ascii=False, indent=2), encoding="utf-8"
211     )
212     summary["terminal"][policy] = metrics
213     print(f"[P1-terminal] {policy}: total={metrics.get('total_cost_yuan')} endSOC={
214         metrics.get('ending_soc_kwh')}")
215
216 for name, storage in EFFICIENCY_VARIANTS.items():
217     started = time.perf_counter()
218     result = run_q2(
219         data_root=data_root,
220         max_days=max_days,
221         output_dir=out / "efficiency" / name,
222         template_name="result2.xlsx",
223         write_outputs=True,
224         load_mode="causal",
225         pv_risk_mode="asymmetric",
226         realtime_feedback=True,
227         storage=storage,
228         tag=f"efficiency_{name}",
229     )
230     metrics = result["metrics"]
231     metrics["wall_seconds"] = time.perf_counter() - started
232     (out / "efficiency" / name / "metrics.json").write_text(
233         json.dumps(metrics, ensure_ascii=False, indent=2), encoding="utf-8"
234     )
235     summary["efficiency"][name] = metrics
236     print(f"[P1-efficiency] {name}: total={metrics.get('total_cost_yuan')}")
237     _dump(out / "summary.json", summary)
238
239 def _dump(path: Path, payload: dict) -> None:
240     path.parent.mkdir(parents=True, exist_ok=True)
241     path.write_text(json.dumps(payload, ensure_ascii=False, indent=2, default=str), encoding
242                     ="utf-8")

```

B.14 未来信息泄漏 mutation 测试 (test_causality.py)

Listing 14:


```

1  """未来信息泄漏自动测试（任务书 §9）。
2
3  核心判据：在 causal 信息集模式下，人为篡改**决策时刻之后**才会发生的
4  真实负载 / 真实光伏 / 真实电价，决策时刻的优化输入必须完全不变。
5  """
6  from __future__ import annotations
7
8  import shutil
9  import tempfile
10 import unittest
11 from pathlib import Path
12
13 import numpy as np
14
15 from ..solve.io_data import read_attachment4
16 from ..solve.q2 import run_q2
17 from ..solve.q3 import run_q3
18 from .mutate import (
19     make_root,
20     mutate_attachment2_load,
21     mutate_attachment2_pv,
22     mutate_attachment4_price,
23 )
24
25 TARGET_DAY = 35
26 DAYS = 36
27 TOL = 1e-10
28
29
30 class CausalityTestBase(unittest.TestCase):
31     @classmethod
32     def setUpClass(cls) -> None:
33         cls._tmpdir = tempfile.mkdtemp(prefix="mathmodel_causal_")
34         cls.tmp = Path(cls._tmpdir)
35         cls.base = make_root(cls.tmp / "base")
36         cls.mut_load = make_root(cls.tmp / "mut_load")
37         mutate_attachment2_load(cls.mut_load, TARGET_DAY, 0, 10.0)
38         cls.mut_pv = make_root(cls.tmp / "mut_pv")
39         mutate_attachment2_pv(cls.mut_pv, TARGET_DAY, 0, 10.0)
40         cls.mut_both_after_6 = make_root(cls.tmp / "mut_both_after6")
41         mutate_attachment2_load(cls.mut_both_after_6, TARGET_DAY, 36, 10.0)
42         mutate_attachment2_pv(cls.mut_both_after_6, TARGET_DAY, 36, 10.0)
43         cls.mut_price_after_6 = make_root(cls.tmp / "mut_price_after6")
44         mutate_attachment4_price(cls.mut_price_after_6, TARGET_DAY, 36, 10.0)
45
46     @classmethod

```

```

47     def tearDownClass(cls) -> None:
48         shutil.rmtree(cls._tmpdir, ignore_errors=True)
49
50     @staticmethod
51     def _run_q2(root: Path, **kwargs):
52         return run_q2(data_root=root, max_days=DAYS, write_outputs=False, load_mode="causal"
53             , **kwargs)
54
55     @staticmethod
56     def _run_q3(root: Path, **kwargs):
57         return run_q3(data_root=root, max_days=DAYS, write_outputs=False, load_mode="causal"
58             , **kwargs)
59
60 class Q2LoadCausalityTest(CausalityTestBase):
61     def test_future_load_cannot_change_plan(self):
62         base = self._run_q2(self.base)
63         mutated = self._run_q2(self.mut_load)
64         b = base["records"][TARGET_DAY]["load_plan_kw"]
65         m = mutated["records"][TARGET_DAY]["load_plan_kw"]
66         self.assertLess(float(np.max(np.abs(b - m))), TOL, "第 d 天真实负载被篡改后 0:00 计
67             划输入发生变化")
68
69     def test_future_load_changes_execution_only(self):
70         """篡改真实负载应当改变紧急购电，说明该数据确实通过执行通道发挥作用。"""
71         base = self._run_q2(self.base)
72         mutated = self._run_q2(self.mut_load)
73         b = base["records"][TARGET_DAY]["execution"].emergency.sum()
74         m = mutated["records"][TARGET_DAY]["execution"].emergency.sum()
75         self.assertNotAlmostEqual(b, m, places=3)
76
77 class Q2PVCausalityTest(CausalityTestBase):
78     def test_future_pv_cannot_change_plan(self):
79         base = self._run_q2(self.base)
80         mutated = self._run_q2(self.mut_pv)
81         b = base["records"][TARGET_DAY]["pv_plan_kw"]
82         m = mutated["records"][TARGET_DAY]["pv_plan_kw"]
83         self.assertLess(float(np.max(np.abs(b - m))), TOL, "第 d 天真实光伏被篡改后 0:00 计
84             划输入发生变化")
85
86 class Q3NodeCausalityTest(CausalityTestBase):
87     def test_node_inputs_do_not_see_future_load_or_pv(self):
88         base = self._run_q3(self.base)
89         mutated = self._run_q3(self.mut_both_after_6)
90         for day in (TARGET_DAY,):

```

```

90         # 只冻结 6:00 这一次决策的输入；12:00/18:00 的预测可以合法使用 6:00-12:00
91         # 已经执行完的观测（这正是滚动在线预测的应有之义）。
92         b_load = base["records"][day]["load_plan_kw"][36:72]
93         m_load = mutated["records"][day]["load_plan_kw"][36:72]
94         self.assertLess(float(np.max(np.abs(b_load - m_load))), TOL, "6:00 负载预测输入
          受到未来真实负载影响")
95         b_block = base["records"][day]["adjusted_grid"][36:72]
96         m_block = mutated["records"][day]["adjusted_grid"][36:72]
97         self.assertLess(float(np.max(np.abs(b_block - m_block))), TOL, "6:00 优化结果受
          到未来真实负载/光伏影响")
98         b_base = base["records"][day]["baseline_plan"].grid
99         m_base = mutated["records"][day]["baseline_plan"].grid
100        self.assertLess(float(np.max(np.abs(b_base - m_base))), TOL, "0:00 基准计划受到
          未来真实负载/光伏影响")
101
102
103    class Q4PriceCausalityTest(CausalityTestBase):
104        def test_causal_price_ignores_future_realized_price(self):
105            prices = read_attachment4(self.base).price
106            base = self._run_q3(self.base, price_matrix=prices, price_mode="causal")
107            mutated = self._run_q3(self.mut_price_after_6, price_matrix=prices, price_mode="
          causal")
108            b = base["records"][TARGET_DAY]["adjusted_grid"][36:72]
109            m = mutated["records"][TARGET_DAY]["adjusted_grid"][36:72]
110            self.assertLess(float(np.max(np.abs(b - m))), TOL, "causal 电价模式下提前看到未来真
          实电价")
111
112        def test_perfect_price_does_see_future_realized_price(self):
113            """反证：perfect 模式下同一篡改确实会改变结果（说明测试本身有鉴别力）。"""
114            base = self._run_q3(self.base, price_matrix=read_attachment4(self.base).price,
          price_mode="perfect")
115            mutated = self._run_q3(
116                self.mut_price_after_6,
117                price_matrix=read_attachment4(self.mut_price_after_6).price,
118                price_mode="perfect",
119            )
120            b = base["records"][TARGET_DAY]["adjusted_grid"][36:72]
121            m = mutated["records"][TARGET_DAY]["adjusted_grid"][36:72]
122            self.assertGreater(float(np.max(np.abs(b - m))), 1e-6)
123
124
125    if __name__ == "__main__":
126        unittest.main()

```

B.15 结算口径单元测试（test_settlement.py）

Listing 15:

```

1  """Q3 结算口径单元测试（任务书 §10）。
2
3  replacement:  $C = p[\min(G_p, G_a) + 0.5(G_p - G_a)^+ + 1.5(G_a - G_p)^+]$ 
4  additive    :  $C = p \cdot G_p + 0.5p(G_p - G_a)^+ + 1.5p(G_a - G_p)^+$ 
5  同时验证 LP 线性化目标与直接回代公式一致 ( $gap < 1e-7$ )。
6  """
7  from __future__ import annotations
8
9  import unittest
10
11  import numpy as np
12
13  from ..solve.lp_core import settlement_cost, solve_adjustment_dispatch
14  from ..solve.params import DEFAULT_STORAGE
15
16
17  class ReplacementFormulaTest(unittest.TestCase):
18      def test_purchase_below_plan(self):
19          #  $G_p=900, G_a=700 \rightarrow 700 + 0.5 \cdot 200 = 800$ 
20          value = settlement_cost(np.array([900.0]), np.array([700.0]), np.array([1.0]), "
21                                  replacement")
22          self.assertAlmostEqual(value, 800.0, places=9)
23
24      def test_purchase_above_plan(self):
25          #  $G_p=700, G_a=900 \rightarrow 700 + 1.5 \cdot 200 = 1000$ 
26          value = settlement_cost(np.array([700.0]), np.array([900.0]), np.array([1.0]), "
27                                  replacement")
28          self.assertAlmostEqual(value, 1000.0, places=9)
29
30      def test_unchanged(self):
31          value = settlement_cost(np.array([800.0]), np.array([800.0]), np.array([1.0]), "
32                                  replacement")
33          self.assertAlmostEqual(value, 800.0, places=9)
34
35  class AdditiveFormulaTest(unittest.TestCase):
36      def test_purchase_below_plan(self):
37          #  $G_p=900, G_a=700 \rightarrow 900 + 0.5 \cdot 200 = 1000$ 
38          value = settlement_cost(np.array([900.0]), np.array([700.0]), np.array([1.0]), "
39                                  additive")
40          self.assertAlmostEqual(value, 1000.0, places=9)
41
42      def test_purchase_above_plan(self):
43          #  $G_p=700, G_a=900 \rightarrow 700 + 1.5 \cdot 200 = 1000$ 
44          value = settlement_cost(np.array([700.0]), np.array([900.0]), np.array([1.0]), "

```

```

        additive")
42     self.assertAlmostEqual(value, 1000.0, places=9)
43
44
45 def _case(seed: int, t: int = 24, mask_all: bool = True):
46     rng = np.random.default_rng(seed)
47     load = 800.0 + 200.0 * rng.random(t)
48     pv = 400.0 * rng.random(t)
49     price = 0.3 + 0.7 * rng.random(t)
50     baseline = np.maximum((load - pv) * (1 / 6), 0.0)
51     mask = np.ones(t, dtype=bool) if mask_all else np.r_[np.ones(t - 6, dtype=bool), np.
        zeros(6, dtype=bool)]
52     return load, pv, price, baseline, mask
53
54
55 def _lp_objective(result, price, baseline, mode: str) -> float:
56     """LP 侧线性化目标（只统计允许调整的时段）。"""
57     mask = result.adjustment_mask
58     p = price[mask]
59     if mode == "replacement":
60         return float(np.dot(p, result.dispatch.grid[mask] + 0.5 * result.increase[mask] +
            0.5 * result.decrease[mask]))
61     return float(np.dot(p, baseline[mask]) + np.dot(p, 0.5 * result.decrease[mask] + 1.5 *
        result.increase[mask]))
62
63
64 class LinearizationTest(unittest.TestCase):
65     """两种口径下 LP 线性化目标与按题意直接回代的差异必须 < 1e-7。"""
66
67     def _gap(self, mode: str, seed: int) -> float:
68         load, pv, price, baseline, mask = _case(seed)
69         result = solve_adjustment_dispatch(
70             load, pv, price, baseline, mask,
71             soc_initial=6000.0, soc_terminal=6000.0, settlement_mode=mode,
72         )
73         direct = settlement_cost(baseline, result.dispatch.grid, price, mode)
74         return abs(direct - _lp_objective(result, price, baseline, mode))
75
76     def test_replacement_linearization(self):
77         for seed in range(5):
78             self.assertLess(self._gap("replacement", seed), 1e-7, f"seed={seed}")
79
80     def test_additive_linearization(self):
81         for seed in range(5):
82             self.assertLess(self._gap("additive", seed), 1e-7, f"seed={seed}")
83
84     def test_two_modes_are_reoptimized_not_revalued(self):

```

```

85     """两种口径应给出各自最优的调度，不能只换算式计费。"""
86     for seed in range(5):
87         load, pv, price, baseline, mask = _case(seed)
88         kwargs = dict(soc_initial=6000.0, soc_terminal=6000.0)
89         replacement = solve_adjustment_dispatch(
90             load, pv, price, baseline, mask, settlement_mode="replacement", **kwargs
91         )
92         additive = solve_adjustment_dispatch(
93             load, pv, price, baseline, mask, settlement_mode="additive", **kwargs
94         )
95         best_replacement = settlement_cost(baseline, replacement.dispatch.grid, price, "
            replacement")
96         cross_replacement = settlement_cost(baseline, additive.dispatch.grid, price, "
            replacement")
97         best_additive = settlement_cost(baseline, additive.dispatch.grid, price, "
            additive")
98         cross_additive = settlement_cost(baseline, replacement.dispatch.grid, price, "
            additive")
99         # 容差取 LP 求解精度量级（主目标容差  $\max(1e-5, 1e-10/F)$ )
100        tol_r = 1e-6 * max(1.0, abs(best_replacement))
101        tol_a = 1e-6 * max(1.0, abs(best_additive))
102        self.assertLessEqual(best_replacement, cross_replacement + tol_r, f"seed={seed}"
            )
103        self.assertLessEqual(best_additive, cross_additive + tol_a, f"seed={seed}")
104
105
106    class StorageParamTest(unittest.TestCase):
107        def test_energy_limit(self):
108            self.assertAlmostEqual(DEFAULT_STORAGE.energy_limit, 5000.0 / 6.0, places=9)
109
110
111    if __name__ == "__main__":
112        unittest.main()

```

B.16 legacy 结果回归测试 (test_regression.py)

Listing 16:

```

1    """legacy 结果回归测试（任务书 §11）。
2
3    在 legacy 参数组合下（perfect 负载 / perfect 价格 / 实时反馈 ON / 非对称 PV 风险 /
4    replacement 结算 / daily_cycle 终端 / 不使用典型日先验），五类结果必须复现
5    Codex 工程的既有数值。
6    """
7    from __future__ import annotations
8

```

```

9 import json
10 import unittest
11 from pathlib import Path
12
13 from ..solve.io_data import read_attachment4
14 from ..solve.q1 import run_q1
15 from ..solve.q2 import run_q2
16 from ..solve.q3 import run_q3
17
18 FIXTURE = Path(__file__).with_name("legacy_metrics.json")
19 LEGACY = json.loads(FIXTURE.read_text(encoding="utf-8"))
20
21 # legacy 总费用尺度下的数值容差 (元)
22 ABS_TOL = 1e-3
23 REL_TOL = 1e-9
24
25
26 def _close(test: unittest.TestCase, label: str, new: float, old: float) -> None:
27     tolerance = max(ABS_TOL, REL_TOL * abs(old))
28     test.assertLess(
29         abs(new - old),
30         tolerance,
31         f"{label} 未复现 legacy 数值: new={new!r} legacy={old!r} diff={new - old!r}",
32     )
33
34
35 class LegacyRegressionTest(unittest.TestCase):
36     @classmethod
37     def setUpClass(cls) -> None:
38         cls.prices = read_attachment4().price
39
40     def test_q1_unchanged(self):
41         metrics = run_q1()["metrics"]
42         _close(self, "Q1 费用", metrics["cost_yuan"], LEGACY["q1"]["cost_yuan"])
43         _close(self, "Q1 购电量", metrics["grid_energy_kwh"], LEGACY["q1"]["grid_energy_kwh"])
44         _close(self, "Q1 充电量", metrics["charge_energy_kwh"], LEGACY["q1"]["charge_energy_kwh"])
45
46     def test_q2_legacy_benchmark(self):
47         metrics = run_q2(
48             max_days=365,
49             write_outputs=False,
50             load_mode="perfect",
51             pv_risk_mode="asymmetric",
52             realtime_feedback=True,
53             price_mode="perfect",

```

```

54         terminal_policy="daily_cycle",
55         use_typical_prior=False,
56     )["metrics"]
57     _close(self, "Q2 总费用", metrics["total_cost_yuan"], LEGACY["q2"]["total_cost_yuan"
58         ])
59     _close(self, "Q2 计划费用", metrics["plan_purchase_cost_yuan"], LEGACY["q2"]["
60         plan_purchase_cost_yuan"])
61     _close(self, "Q2 紧急电量", metrics["emergency_energy_kwh"], LEGACY["q2"]["
62         emergency_energy_kwh"])
63     _close(self, "Q2 弃光量", metrics["curtailment_energy_kwh"], LEGACY["q2"]["
64         curtailment_energy_kwh"])
65
66     def test_q3_legacy_benchmark(self):
67         metrics = run_q3(
68             max_days=365,
69             write_outputs=False,
70             downscale="linear",
71             calibrate=True,
72             update_nodes=(0, 36, 72, 108),
73             settlement_mode="replacement",
74             load_mode="perfect",
75             price_mode="perfect",
76             realtime_feedback=True,
77             terminal_policy="daily_cycle",
78             use_typical_prior=False,
79         )["metrics"]
80     _close(self, "Q3 总费用", metrics["total_cost_yuan"], LEGACY["q3"]["total_cost_yuan"
81         ])
82     _close(self, "Q3 基准计划费用", metrics["baseline_purchase_cost_yuan"], LEGACY["q3"
83         ]["baseline_purchase_cost_yuan"])
84     _close(self, "Q3 紧急电量", metrics["emergency_energy_kwh"], LEGACY["q3"]["
85         emergency_energy_kwh"])
86     _close(self, "Q3 弃光量", metrics["curtailment_energy_kwh"], LEGACY["q3"]["
87         curtailment_energy_kwh"])
88
89     def test_q4_legacy_benchmark(self):
90         metrics2 = run_q2(
91             max_days=365,
92             write_outputs=False,
93             price_matrix=self.prices,
94             load_mode="perfect",
95             pv_risk_mode="asymmetric",
96             realtime_feedback=True,
97             price_mode="perfect",
98             use_typical_prior=False,
99         )["metrics"]
100     _close(self, "Q4-2 总费用", metrics2["total_cost_yuan"], LEGACY["q4-2"]["

```



```

    total_cost_yuan"))
93 metrics3 = run_q3(
94     max_days=365,
95     write_outputs=False,
96     price_matrix=self.prices,
97     update_nodes=(0, 36, 72, 108),
98     settlement_mode="replacement",
99     load_mode="perfect",
100     price_mode="perfect",
101     realtime_feedback=True,
102     use_typical_prior=False,
103     )["metrics"]
104 _close(self, "Q4-3 总费用", metrics3["total_cost_yuan"], LEGACY["q4-3"]["
    total_cost_yuan"])
105
106
107 if __name__ == "__main__":
108     unittest.main()

```